

C++ Standard Library Issues Resolved Directly In Prague

Doc. no. P2117R0

Date: Revised 2020-02-14 at 16:23:28 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Immediate Issues

[1203^{\(i\)}](#). More useful rvalue stream insertion

Section: 29.7.5.5 [[ostream.rvalue](#)], 29.7.4.5 [[istream.rvalue](#)] **Status:** [Immediate](#) **Submitter:** Howard Hinnant **Opened:** 2009-09-06 **Last modified:** 2020-02-14

Priority: 2

View all other [issues](#) in [[ostream.rvalue](#)].

View all issues with [Immediate](#) status.

Discussion:

29.7.5.5 [[ostream.rvalue](#)] was created to preserve the ability to insert into (and extract from 29.7.4.5 [[istream.rvalue](#)]) rvalue streams:

```
template <class charT, class traits, class T>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>&& os, const T& x);
```

1 *Effects:* os << x

2 *Returns:* os

This is good as it allows code that wants to (for example) open, write to, and close an ofstream all in one statement:

```
std::ofstream("log file") << "Some message\n";
```

However, I think we can easily make this "rvalue stream helper" even easier to use. Consider trying to quickly create a formatted string. With the current spec you have to write:

```
std::string s = static_cast<std::ostringstream&>(std::ostringstream() << "i = " << i).str();
```

This will store "i = 10" (for example) in the string s. Note the need to cast the stream back to ostringstream& prior to using the member .str(). This is necessary because the inserter has cast the ostringstream down to a more generic ostream during the insertion process.

I believe we can re-specify the rvalue-inserter so that this cast is unnecessary. Thus our customer now has to only type:

```
std::string s = (std::ostringstream() << "i = " << i).str();
```

This is accomplished by having the rvalue stream inserter return an rvalue of the same type, instead of casting it down to the base class. This is done by making the stream generic, and constraining it to be an rvalue of a type derived from ios_base.

The same argument and solution also applies to the inserter. This code has been implemented and tested.

[2009 Santa Cruz:]

NAD Future. No consensus for change.

[LEWG Kona 2017]

Recommend Open: Design looks right.

[2018-05-25, Billy O'Neal requests this issue be reopened and provides P/R rebased against N4750]

Billy O'Neal requests this issue be reopened, as changing operator>> and operator<< to use perfect forwarding as described here is necessary to implement LWG [2534](#) which was accepted. Moreover, this P/R also resolves LWG [2498](#).

Previous resolution [SUPERSEDED]:

Change 29.7.4.5 [[istream.rvalue](#)]:

```
template <class charT, class traits Istream, class T>
basic_istream<charT, traits>& Istream&&
operator>>(basic_istream<charT, traits> Istream&& is, T& x);
```

1 Effects: is >> x

2 Returns: `std::move(is)`.

3 Remarks: This signature shall participate in overload resolution if and only if `Istream` is not an lvalue reference type and is derived from `ios_base`.

Change 29.7.5.5 [\[ostream.rvalue\]](#):

```
template <class charT, class traits Ostream, class T>
basic_ostream<charT, traits>& Ostream&&
operator<<(basic_ostream<charT, traits> Ostream&& os, const T& x);
```

1 Effects: os << x

2 Returns: `std::move(os)`.

3 Remarks: This signature shall participate in overload resolution if and only if `Ostream` is not an lvalue reference type and is derived from `ios_base`.

[2018-12-03, Ville comments]

The implementation in libstdc++ doesn't require derivation from `ios_base`, it requires convertibility to `basic_istream/basic_ostream`. This has been found to be important to avoid regressions with existing stream wrappers.

In libstdc++, the inserter/extractor also don't return a reference to the template parameter, they return a reference to the `basic_istream/basic_ostream` specialization the template parameter converts to. This was done in order to try and be closer to the earlier specification's return type, which specified `basic_ostream<charT, traits>&` and `basic_istream<charT, traits>&`. So we detected convertibility to (a type convertible to) those, and returned the result of that conversion. That doesn't seem to be necessary, and probably bothers certain chaining cases. Based on recent experiments, it seems that this return-type dance (as opposed to just returning what the p/r suggests) is unnecessary, and doesn't trigger any regressions.

[2019-01-20 Reflector prioritization]

Set Priority to 2

Previous resolution [SUPERSEDED]:

This resolution is relative to N4750.

Change 29.7.4.5 [\[istream.rvalue\]](#) as follows:

```
template <class charT, class traits Istream, class T>
basic_istream<charT, traits>& Istream&&
operator>>(basic_istream<charT, traits> Istream&& is, T& x);
```

-1- Effects: Equivalent to:

```
is >> std::forward<T>(x)
return std::move(is);
```

-2- Remarks: This function shall not participate in overload resolution unless the expression `is >> std::forward<T>(x)` is well-formed, `Istream` is not an lvalue reference type, and `Istream` is derived from `ios_base`.

Change 29.7.5.5 [\[ostream.rvalue\]](#):

```
template <class charT, class traits Ostream, class T>
basic_ostream<charT, traits>& Ostream&&
operator<<(basic_ostream<charT, traits> Ostream&& os, const T& x);
```

-1- Effects: As if by: os << x;

-2- Returns: `std::move(os)`.

-3- Remarks: This signature shall not participate in overload resolution unless the expression `os << x` is well-formed, `Ostream` is not an lvalue reference type, and `Ostream` is derived from `ios_base`.

[2019-03-17; Daniel comments and provides updated wording]

After discussion with Ville it turns out that the "derived from `ios_base`" approach works fine and no breakages were found in regression tests. As result of that discussion the wording was rebased to the most recent working draft and the "overload resolution participation" wording was replaced by a corresponding *Constraints* element.

[2020-02 Status to Immediate on Friday morning in Prague.]

Proposed resolution:

This wording is relative to [N4810](#).

1. Change 29.7.4.5 [\[istream.rvalue\]](#) as follows:

```
template <class charT, class traits Istream, class T>
basic_istream<charT, traits>& Istream&&
operator>>(basic_istream<charT, traits> Istream&& is, T&& x);
```

~~-?- Constraints: The expression is >> std::forward<T>(x) is well-formed and Istream is publicly and unambiguously derived from ios_base.~~

~~-1- Effects: Equivalent to:~~

```
is >> std::forward<T>(x);
return std::move(is);;
```

~~-2- Remarks: This function shall not participate in overload resolution unless the expression is >> std::forward<T>(x) is well-formed.~~

2. Change 29.7.5.5 [\[ostream.rvalue\]](#):

```
template <class charT, class traits Ostream, class T>
basic_ostream<charT, traits>& Ostream&&
operator<<(basic_ostream<charT, traits> Ostream&& os, const T& x);
```

~~-?- Constraints: The expression os << x is well-formed and Ostream is publicly and unambiguously derived from ios_base.~~

~~-1- Effects: As if by: os << x;~~

~~-2- Returns: std::move(os).~~

~~-3- Remarks: This signature shall not participate in overload resolution unless the expression os << x is well-formed.~~

[2859^{\(1\)}](#). Definition of *reachable* in [\[ptr.laundry\]](#) misses pointer arithmetic from pointer-interconvertible object

Section: 17.6.4 [\[ptr.laundry\]](#) Status: [Immediate](#) Submitter: Hubert Tong Opened: 2017-01-31 Last modified: 2020-02-14

Priority: 2

View all other [issues](#) in [\[ptr.laundry\]](#).

View all issues with [Immediate](#) status.

Discussion:

Given:

```
struct A { int x, y; };
A a[100];
```

The bytes which compose `a[3]` can be reached from `&a[2].x`: `reinterpret_cast<A*>(&a[2].x) + 1` points to `a[3]`, however, the definition of "reachable" in [\[ptr.laundry\]](#) does not encompass this case.

[2017-03-04, Kona]

Set priority to 2. Assign this (and [2860](#)) to Core.

[2017-08-14, CWG telecon note]

CWG is fine with the proposed resolution.

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4618](#).

1. Modify 17.6.4 [\[ptr.laundry\]](#) as indicated:

```
template <class T> constexpr T* launder(T* p) noexcept;

[...]
```

-3- Remarks: An invocation of this function may be used in a core constant expression whenever the value of its argument may be used in a core constant expression. A byte of storage `b` is reachable through a pointer value that points to an object `Y` if ~~there is an object `Z`, pointer-interconvertible with `Y`, such that `b` is within the storage occupied by `Z`, an object that is pointer-interconvertible with `Y`~~ or the immediately-enclosing array object if `Z` is an array element. The program is ill-formed if `T` is a function type or (possibly cv-qualified) void.

3018(i). `shared_ptr` of function type

Section: 20.11.3 [\[util.smartptr.shared\]](#) Status: [Immediate](#) Submitter: Agustín K-ballo Bergé Opened: 2017-09-13 Last modified: 2020-02-14

Priority: 3

View other [active issues](#) in [\[util.smartptr.shared\]](#).

View all other [issues](#) in [\[util.smartptr.shared\]](#).

View all issues with [Immediate](#) status.

Discussion:

`shared_ptr` has been designed to support use cases where it owns a pointer to function, and whose deleter does something with it. This can be used, for example, to keep a dynamic library loaded for as long as their exported functions are referenced.

Implementations have overlooked that the `T` in `shared_ptr<T>` can be a function type. It isn't immediately obvious from the standard, and it's not possible to tell from the wording that this is intentional.

[2017-11 Albuquerque Wednesday night issues processing]

Priority set to 3

Previous resolution [SUPERSEDED]:

This wording is relative to [N4687](#).

1. Edit 20.11.3 [\[util.smartptr.shared\]](#) as indicated:

[...]

-2- Specializations of `shared_ptr` shall be `CopyConstructible`, `CopyAssignable`, and `LessThanComparable`, allowing their use in standard containers. Specializations of `shared_ptr` shall be contextually convertible to `bool`, allowing their use in boolean expressions and declarations in conditions. ~~The template parameter `T` of `shared_ptr` may be an incomplete type.~~

?- The template parameter `T` of `shared_ptr` may be an incomplete type. `T*` shall be an object pointer type or a function pointer type.

[...]

[2020-02-13; Prague]

LWG would prefer to make the new constraint a Mandates-like thing.

Original resolution [SUPERSEDED]:

This wording is relative to [N4849](#).

1. Edit 20.11.3 [\[util.smartptr.shared\]](#) as indicated:

[...]

-2- Specializations of `shared_ptr` shall be `Cpp17CopyConstructible`, `Cpp17CopyAssignable`, and `Cpp17LessThanComparable`, allowing their use in standard containers. Specializations of `shared_ptr` shall be contextually convertible to `bool`, allowing their use in boolean expressions and declarations in conditions. ~~The template parameter `T` of `shared_ptr` may be an incomplete type.~~

?- The template parameter `T` of `shared_ptr` may be an incomplete type. The program is ill-formed unless `T*` is an object pointer type or a function pointer type.

[...]

[2020-02 Friday AM discussion in Prague.]

Marshall provides updated wording; status to Immediate

Proposed resolution:

This wording is relative to [N4849](#).

1. Edit 20.11.3 [\[util.smartptr.shared\]](#) as indicated:

[...]

-2- Specializations of `shared_ptr` shall be `Cpp17CopyConstructible`, `Cpp17CopyAssignable`, and `Cpp17LessThanComparable`, allowing their use in standard containers. Specializations of `shared_ptr` shall be contextually convertible to `bool`, allowing their use in boolean expressions and declarations in conditions. ~~The template parameter `T` of `shared_ptr` may be an incomplete type.~~

~~[-? The template parameter `T` of `shared_ptr` may be an incomplete type. [Note: `T` may be a function type. -- end note]~~

[...]

[3050](#)⁽ⁱ⁾. Conversion specification problem in `chrono::duration` constructor

Section: 27.5.1 [\[time.duration.cons\]](#), 27.5.5 [\[time.duration.nonmember\]](#) **Status:** [Immediate](#) **Submitter:** Barry Revzin **Opened:** 2018-01-22 **Last modified:** 2020-02-14

Priority: 3

View other [active issues](#) in [\[time.duration.cons\]](#).

View all other [issues](#) in [\[time.duration.cons\]](#).

View all issues with [Immediate](#) status.

Discussion:

The converting constructor in `std::chrono::duration` is currently specified as (27.5.1 [\[time.duration.cons\]](#) p1):

```
template<class Rep2>
constexpr explicit duration(const Rep2& r);
```

Remarks: This constructor shall not participate in overload resolution unless `Rep2` is implicitly convertible to `rep` and [...]

But the parameter is of type `Rep2 const`, so we should check that `Rep2 const` is implicitly convertible to `rep`, not just `Rep2`. This means that for a type like:

```
struct X { operator int64_t() /* not const */; };
```

`std::is_constructible_v<std::chrono::seconds, X>` is true, but actual construction will fail to compile.

Further analysis of sub-clause 27.5 [\[time.duration\]](#) revealed similar specification problems in some other functions.

[2018-06 Rapperswil Thursday issues processing]

P3; Status to Open

[2018-11 San Diego Thursday night issue processing]

Jonathan to provide updated wording; the underlying text has changed.

[2018-12-05 Jonathan provides new wording]

In San Diego Geoff noticed that the current WP does not use CR. Jonathan provides new wording consistent with the [editorial changes](#) that removed CR.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4713](#).

1. Modify 27.5.1 [\[time.duration.cons\]](#) as indicated:

```
template<class Rep2>
constexpr explicit duration(const Rep2& r);
```

-1- *Remarks:* This constructor shall not participate in overload resolution unless [is_convertible_v<const Rep2&, rep> is true](#) ~~`Rep2` is implicitly convertible to `rep`~~ and

(1.1) — `treat_as_floating_point_v<rep>` is true or

(1.2) — `treat_as_floating_point_v<Rep2>` is false.

[...]

-2- *Effects*: Constructs an object of type duration.

-3- *Postconditions*: count() == static_cast<rep>(r).

2. Modify 27.5.5 [time.duration.nonmember] as indicated:

```
template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator*(const duration<Rep1, Period>& d, const Rep2& s);
```

-4- *Remarks*: This operator shall not participate in overload resolution unless is_convertible v<const Rep2&, CR(Rep1, Rep2)> is true ~~Rep2 is implicitly convertible to CR(Rep1, Rep2).~~

[...]

```
template<class Rep1, class Rep2, class Period>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator*(const Rep1& s, const duration<Rep2, Period>& d);
```

-6- *Remarks*: This operator shall not participate in overload resolution unless is_convertible v<const Rep1&, CR(Rep1, Rep2)> is true ~~Rep1 is implicitly convertible to CR(Rep1, Rep2).~~

[...]

```
template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator/(const duration<Rep1, Period>& d, const Rep2& s);
```

-8- *Remarks*: This operator shall not participate in overload resolution unless is_convertible v<const Rep2&, CR(Rep1, Rep2)> is true ~~Rep2 is implicitly convertible to CR(Rep1, Rep2)~~ and Rep2 is not a specialization of duration.

[...]

```
template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator%(const duration<Rep1, Period>& d, const Rep2& s);
```

-11- *Remarks*: This operator shall not participate in overload resolution unless is_convertible v<const Rep2&, CR(Rep1, Rep2)> is true ~~Rep2 is implicitly convertible to CR(Rep1, Rep2)~~ and Rep2 is not a specialization of duration.

[...]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4791](#).

1. Modify 27.5.1 [time.duration.cons] as indicated:

```
template<class Rep2>
constexpr explicit duration(const Rep2& r);
```

-1- *Remarks*: This constructor shall not participate in overload resolution unless is_convertible v<const Rep2&, rep> is true ~~Rep2 is implicitly convertible to rep~~ and

(1.1) — treat_as_floating_point_v<rep> is true or

(1.2) — treat_as_floating_point_v<Rep2> is false.

[...]

-2- *Effects*: Constructs an object of type duration.

-3- *Postconditions*: count() == static_cast<rep>(r).

2. Modify 27.5.5 [time.duration.nonmember] as indicated:

```
template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator*(const duration<Rep1, Period>& d, const Rep2& s);
```

-4- *Remarks*: This operator shall not participate in overload resolution unless is_convertible v<const Rep2&, common_type_t<Rep1, Rep2>> is true ~~Rep2 is implicitly convertible to common_type_t<Rep1, Rep2>.~~

```

[...]

template<class Rep1, class Rep2, class Period>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator*(const Rep1& s, const duration<Rep2, Period>& d);

-6- Remarks: This operator shall not participate in overload resolution unless is_convertible_v<const Rep1&, common_type_t<Rep1, Rep2>> is true Rep1 is implicitly convertible to common_type_t<Rep1, Rep2>.

[...]

template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator/(const duration<Rep1, Period>& d, const Rep2& s);

-8- Remarks: This operator shall not participate in overload resolution unless is_convertible_v<const Rep2&, common_type_t<Rep1, Rep2>> is true Rep2 is implicitly convertible to common_type_t<Rep1, Rep2> and Rep2 is not a specialization of duration.

[...]

template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator%(const duration<Rep1, Period>& d, const Rep2& s);

-11- Remarks: This operator shall not participate in overload resolution unless is_convertible_v<const Rep2&, common_type_t<Rep1, Rep2>> is true Rep2 is implicitly convertible to common_type_t<Rep1, Rep2> and Rep2 is not a specialization of duration.

[...]

```

[2020-02-13, Prague]

Rebase to most recent working draft

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 27.5.1 [\[time.duration.cons\]](#) as indicated:

```

template<class Rep2>
constexpr explicit duration(const Rep2& r);

-1- Constraints: is_convertible_v<const Rep2&, rep> is true and

(1.1) — treat_as_floating_point_v<rep> is true or

(1.2) — treat_as_floating_point_v<Rep2> is false.

[...]

-2- Postconditions: count() == static_cast<rep>(r).

```

2. Modify 27.5.5 [\[time.duration.nonmember\]](#) as indicated:

```

template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator*(const duration<Rep1, Period>& d, const Rep2& s);

-4- Constraints: is_convertible_v<const Rep2&, common_type_t<Rep1, Rep2>> is true.

[...]

template<class Rep1, class Rep2, class Period>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator*(const Rep1& s, const duration<Rep2, Period>& d);

-6- Constraints: is_convertible_v<const Rep1&, common_type_t<Rep1, Rep2>> is true.

[...]

template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator/(const duration<Rep1, Period>& d, const Rep2& s);

```

-8- Constraints: `is_convertible_v<const Rep2&, common_type_t<Rep1, Rep2>>` is true and Rep2 is not a specialization of duration.

[...]

```
template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator%(const duration<Rep1, Period>& d, const Rep2& s);
```

-12- Constraints: `is_convertible_v<const Rep2&, common_type_t<Rep1, Rep2>>` is true and Rep2 is not a specialization of duration.

[...]

3141⁽ⁱ⁾. CopyConstructible doesn't preserve source values

Section: 18.6 [concepts.object] Status: [Immediate](#) Submitter: Casey Carter Opened: 2018-07-07 Last modified: 2020-02-14

Priority: 2

View all issues with [Immediate](#) status.

Discussion:

The design intent of the CopyConstructible concept has always been that the source of the constructions it requires be left unmodified. Before [P0541R1](#) reformulated the Constructible concept in terms of `is_constructible`, the wording which is now in 18.2 [concepts.equality]/5:

This document uses a notational convention to specify which expressions declared in a *requires-expression* modify which inputs: except where otherwise specified, [...] Operands that are constant lvalues or rvalues are required to not be modified.

indirectly enforced that requirement. Unfortunately, *nothing* in the current wording in 18.4.14 [concept.copyconstructible] enforces that requirement.

[2018-11 Reflector prioritization]

Set Priority to 2

Previous resolution: [SUPERSEDED]

This wording is relative to [N4762](#).

Change 18.4.14 [concept.copyconstructible] as indicated:

```
template<class T>
concept CopyConstructible =
    MoveConstructible<T> &&
    Constructible<T, T&> && ConvertibleTo<T&, T> &&
    Constructible<T, const T&> && ConvertibleTo<const T&, T> &&
    Constructible<T, const T> && ConvertibleTo<const T, T>;
```

-1- If T is an object type, then let v be an lvalue of type (possibly `const`) T or an rvalue of type `const T`. `CopyConstructible<T>` is satisfied only if

(1.1) — After the definition $T\ u = v$;, u is equal to v **and v is unmodified**.

(1.2) — $T(v)$ is equal to v **and does not modify v** .

[2020-02-13 Per LWG review, Casey provides updated P/R wording.]

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

Change 18.4.14 [concept.copyconstructible] as indicated:

```
template<class T>
concept copy_constructible =
    move_constructible<T> &&
    constructible_from<T, T&> && convertible_to<T&, T> &&
    constructible_from<T, const T&> && convertible_to<const T&, T> &&
    constructible_from<T, const T> && convertible_to<const T, T>;
```

-1- If T is an object type, then let v be an lvalue of type (possibly `const`) T or an rvalue of type `const T`. T models `copy_constructible` only if

(1.1) — After the definition $\tau u = v$; u is equal to v (18.2 [concepts.equality]) and v is not modified.

(1.2) — $\tau(v)$ is equal to v and does not modify v .

3150⁽ⁱ⁾. UniformRandomBitGenerator should validate min and max

Section: 26.6.2.3 [rand.req.urng] Status: Immediate Submitter: Casey Carter Opened: 2018-08-09 Last modified: 2020-02-14

Priority: 3

View other active issues in [rand.req.urng].

View all other issues in [rand.req.urng].

View all issues with Immediate status.

Discussion:

26.6.2.3 [rand.req.urng] paragraph 2 specifies axioms for the UniformRandomBitGenerator concept:

2 Let g be an object of type G . G models UniformRandomBitGenerator only if

(2.1) — both $G::\text{min}()$ and $G::\text{max}()$ are constant expressions (7.7 [expr.const]),

(2.2) — $G::\text{min}() < G::\text{max}()$,

(2.3) — $G::\text{min}() \leq g()$,

(2.4) — $g() \leq G::\text{max}()$, and

(2.5) — $g()$ has amortized constant complexity.

Bullets 2.1 and 2.2 are both compile-time requirements that ought to be validated by the concept.

[2018-08-20 Priority set to 3 after reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to N4791.

1. Modify 26.6.2.3 [rand.req.urng] as follows:

1 A uniform random bit generator g of type G is a function object returning unsigned integer values such that each value in the range of possible results has (ideally) equal probability of being returned. [Note: The degree to which g 's results approximate the ideal is often determined statistically.—end note]

```
template<auto> struct require-constant; // exposition-only

template<class G>
concept UniformRandomBitGenerator =
    Invocable<G>&& UnsignedIntegral<invoke_result_t<G>>&&
    requires {
        { G::min() } -> Same<invoke_result_t<G>>&&;
        { G::max() } -> Same<invoke_result_t<G>>&&;
        typename require-constant<G::min()>;
        typename require-constant<G::max()>;
        requires G::min() < G::max();
    };
```

2 Let g be an object of type G . G models UniformRandomBitGenerator only if

~~(2.1) — both $G::\text{min}()$ and $G::\text{max}()$ are constant expressions (7.7 [expr.const]),~~

~~(2.2) — $G::\text{min}() < G::\text{max}()$,~~

(2.3) — $G::\text{min}() \leq g()$,

(2.4) — $g() \leq G::\text{max}()$, and

(2.5) — $g()$ has amortized constant complexity.

3 A class G meets the uniform random bit generator requirements if G models UniformRandomBitGenerator, $\text{invoke_result_t}<G>$ is an unsigned integer type (6.8.1 [basic.fundamental]), and G provides a nested typedef-name result_type that denotes the same type as $\text{invoke_result_t}<G>$.

[2020-02-13; Prague]

LWG provided some improved wording.

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 26.6.2.3 [\[rand.req.urng\]](#) as follows:

1 A *uniform random bit generator* g of type G is a function object returning unsigned integer values such that each value in the range of possible results has (ideally) equal probability of being returned. [Note: The degree to which g 's results approximate the ideal is often determined statistically.—end note]

```
template<class G>
concept uniform_random_bit_generator =
    invocable<G&& && unsigned_integral<invoke_result_t<G&&>> &&
    requires {
        { G::min() } -> same_as<invoke_result_t<G&&>>;
        { G::max() } -> same_as<invoke_result_t<G&&>>;
        requires bool constant<(G::min() < G::max())>::value;
    };
```

-2- Let g be an object of type G . G models `uniform_random_bit_generator` only if

~~(2.1) — both $G::min()$ and $G::max()$ are constant expressions (7.7 [\[expr.const\]](#));~~

~~(2.2) — $G::min() < G::max()$;~~

(2.3) — $G::min() \leq g()$,

(2.4) — $g() \leq G::max()$, and

(2.5) — $g()$ has amortized constant complexity.

3 A class G meets the *uniform random bit generator* requirements if G models `uniform_random_bit_generator`, `invoke_result_t<G&&>` is an unsigned integer type (6.8.1 [\[basic.fundamental\]](#)), and G provides a nested *typedef-name* `result_type` that denotes the same type as `invoke_result_t<G&&>`.

[3175^{\(i\)}](#). The CommonReference requirement of concept `SwappableWith` is not satisfied in the example

Section: 18.4.9 [\[concept.swappable\]](#) Status: [Immediate](#) Submitter: Kostas Kyrimis Opened: 2018-12-14 Last modified: 2020-02-10

Priority: 1

View other [active issues](#) in `[concept.swappable]`.

View all other [issues](#) in `[concept.swappable]`.

View all issues with [Immediate](#) status.

Discussion:

The defect stems from the example found in sub-clause 18.4.9 [\[concept.swappable\]](#) p5:

```
[...]
template<class T, std::SwappableWith<T> U>
void value_swap(T&& t, U&& u) {
    ranges::swap(std::forward<T>(t), std::forward<U>(u));
}

[...]
namespace N {
    struct A { int m; };
    struct Proxy { A* a; };
    Proxy proxy(A& a) { return Proxy{ &a }; }

    void swap(A& x, Proxy p) {
        ranges::swap(x.m, p.a->m);
    }
    void swap(Proxy p, A& x) { swap(x, p); } // satisfy symmetry requirement
}

int main() {
    [...]
    N::A a1 = { 5 }, a2 = { -5 };
    value_swap(a1, proxy(a2)); // diagnostic manifests here(#1)
```

```
    assert(a1.m == -5 && a2.m == 5);
}
```

The call to `value_swap(a1, proxy(a2))` resolves to `[T = N::A&, U = N::Proxy]` The compiler will issue a diagnostic for #1 because:

1. `rvalue proxy(a2)` is not swappable
2. concept `SwappableWith<T, U>` requires `N::A` and `Proxy` to model `CommonReference<const remove_reference_t<T>&, const remove_reference_t<U>&>` It follows from the example that there is no common reference for `[T = N::A&, U = N::Proxy]`

[2019-06-20; Casey Carter comments and provides improved wording]

The purpose of the `CommonReference` requirements in the cross-type concepts is to provide a sanity check. The fact that two types satisfy a single-type concept, have a common reference type that satisfies that concept, and implement cross-type operations required by the cross-type flavor of that concept very strongly suggests the programmer intends them to model the cross-type concept. It's an opt-in that requires some actual work, so it's unlikely to be inadvertent.

The `CommonReference<const T&, const U&>` pattern makes sense for the comparison concepts which require that all variations of `const` and `value` category be comparable: we use `const lvalues` to trigger the "implicit expression variation" wording in 18.2 [concepts.equality]. `SwappableWith`, however, doesn't care about implicit expression variations: it only needs to witness that we can exchange the values denoted by two reference-y expressions `E1` and `E2`. This suggests that `CommonReference<decltype((E1)), decltype((E2))>` is a more appropriate requirement than the current `CommonReference<const remove_reference_t<...>&>` mess that was blindly copied from the comparison concepts.

We must change the definition of "exchange the values" in 18.4.9 [concept.swappable] — which refers to the common reference type — consistently.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4791](#).

1. Change 18.4.9 [concept.swappable] as indicated:

-3- [...]

```
template<class T>
concept Swappable = requires(T& a, T& b) { ranges::swap(a, b); };

template<class T, class U>
concept SwappableWith =
CommonReference<T, U>const remove_reference_t<T>&, const remove_reference_t<U>&> &&
requires(T&& t, U&& u) {
    ranges::swap(std::forward<T>(t), std::forward<T>(t));
    ranges::swap(std::forward<U>(u), std::forward<U>(u));
    ranges::swap(std::forward<T>(t), std::forward<U>(u));
    ranges::swap(std::forward<U>(u), std::forward<T>(t));
};
```

-4- [...]

-5- [Example: User code can ensure that the evaluation of swap calls is performed in an appropriate context under the various conditions as follows:

```
#include <cassert>
#include <concepts>
#include <utility>

namespace ranges = std::ranges;

template<class T, std::SwappableWith<T> U>
void value_swap(T&& t, U&& u) {
    ranges::swap(std::forward<T>(t), std::forward<U>(u));
}

template<std::Swappable T>
void lv_swap(T& t1, T& t2) {
    ranges::swap(t1, t2);
}

namespace N {
    struct A { int m; };
    struct Proxy {
        A* a;
        Proxy(A& a) : a(&a) {}
        friend void swap(Proxy&& x, Proxy&& y) {
            ranges::swap(x.a, y.a);
        }
    };
    Proxy proxy(A& a) { return Proxy{ &a }; }
    void swap(A& x, Proxy p) {
        ranges::swap(x.m, p.a->m);
    }
    void swap(Proxy p, A& x) { swap(x, p); } // satisfy symmetry requirement
}
```

```

int main() {
    int i = 1, j = 2;
    lv_swap(i, j);
    assert(i == 2 && j == 1);
    N::A a1 = { 5 }, a2 = { -5 };
    value_swap(a1, proxy(a2));
    assert(a1.m == -5 && a2.m == 5);
}

```

[2020-01-16 Priority set to 1 after discussion on the reflector.]

[2020-02-10 Move to Immediate Monday afternoon in Prague]

Proposed resolution:

This wording is relative to [N4820](#).

1. Change 18.4.9 [[concept.swappable](#)] as indicated:

-1- Let t_1 and t_2 be equality-preserving expressions that denote distinct equal objects of type T , and let u_1 and u_2 similarly denote distinct equal objects of type U . [Note: t_1 and u_1 can denote distinct objects, or the same object. — end note] An operation *exchanges the values* denoted by t_1 and u_1 if and only if the operation modifies neither t_2 nor u_2 and:

(1.1) — If T and U are the same type, the result of the operation is that t_1 equals u_2 and u_1 equals t_2 .

(1.2) — If T and U are different types ~~that model `CommonReference<const T&, const U&>` and~~ `CommonReference<decltype((t1)), decltype((u1))>` is modeled, the result of the operation is that $C(t_1)$ equals $C(u_2)$ and $C(u_1)$ equals $C(t_2)$ where C is `common_reference_t<const T&, const U&decltype((t1)), decltype((u1))>`.

-2- [...]

-3- [...]

```

template<class T>
concept Swappable = requires(T& a, T& b) { ranges::swap(a, b); };

template<class T, class U>
concept SwappableWith =
    CommonReference<T, Uconst remove_reference_t<T>&, const remove_reference_t<U>&> &&
    requires(T& t, U& u) {
        ranges::swap(std::forward<T>(t), std::forward<T>(t));
        ranges::swap(std::forward<U>(u), std::forward<U>(u));
        ranges::swap(std::forward<T>(t), std::forward<U>(u));
        ranges::swap(std::forward<U>(u), std::forward<T>(t));
    };

```

-4- [...]

-5- [Example: User code can ensure that the evaluation of swap calls is performed in an appropriate context under the various conditions as follows:

```

#include <cassert>
#include <concepts>
#include <utility>

namespace ranges = std::ranges;

template<class T, std::SwappableWith<T> U>
void value_swap(T&& t, U&& u) {
    ranges::swap(std::forward<T>(t), std::forward<U>(u));
}

template<std::Swappable T>
void lv_swap(T& t1, T& t2) {
    ranges::swap(t1, t2);
}

namespace N {
    struct A { int m; };
    struct Proxy {
        A* a;
        Proxy(A& a) : a{&a} {}
        friend void swap(Proxy x, Proxy y) {
            ranges::swap(*x.a, *y.a);
        }
    };
    Proxy proxy(A& a) { return Proxy{ &a }; }
    void swap(A& x, Proxy p) {
        ranges::swap(x.m, p.a->m);
    }
    void swap(Proxy p, A& x) { swap(x, p); } // satisfy symmetry requirement
}

```

```

int main() {
    int i = 1, j = 2;
    lv_swap(i, j);
    assert(i == 2 && j == 1);
    N::A a1 = { 5 }, a2 = { -5 };
    value_swap(a1, proxy(a2));
    assert(a1.m == -5 && a2.m == 5);
}

```

[3200^{\(i\)}](#). midpoint should not constrain τ is complete

Section: 25.9.15 [[numeric.ops.midpoint](#)] **Status:** [Immediate](#) **Submitter:** Paolo Torres **Opened:** 2019-04-10 **Last modified:** 2020-02-14

Priority: 2

View all issues with [Immediate](#) status.

Discussion:

The constraint of the midpoint overload in 25.9.15 [[numeric.ops.midpoint](#)]:

```

template<class T>
constexpr T* midpoint(T* a, T* b);

```

-4- *Constraints:* τ is a complete object type.

is incorrect. Paragraph 4 states τ is constrained to be a complete object type, however it does not seem to be possible to implement τ is complete. This means behavior is conditioned dependent on whether a type is complete, and this seems inconsistent with the library precedent.

[2019-06-12 Priority set to 2 after reflector discussion]

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4810](#).

1. Modify 25.9.15 [[numeric.ops.midpoint](#)] as indicated:

```

template<class T>
constexpr T* midpoint(T* a, T* b);

```

-4- *Constraints:* τ is an ~~complete~~ object type.

~~.- Mandates:~~ τ is a complete type.

[...]

[3201^{\(i\)}](#). lerp should be marked as noexcept

Section: 26.8.4 [[c.math.lerp](#)] **Status:** [Immediate](#) **Submitter:** Paolo Torres **Opened:** 2019-04-10 **Last modified:** 2020-02-14

Priority: 2

View all issues with [Immediate](#) status.

Discussion:

The overloads of `lerp` should be marked as `noexcept`, and this can be explained through the Lakos Rule. This function does not specify any undefined behaviour, and as such has no preconditions. This implies it has a wide contract, meaning it cannot throw, and thus can be marked as `noexcept`.

[2020-02 Moved to Immediate on Thursday afternoon in Prague.]

Proposed resolution:

This wording is relative to [N4810](#).

1. Modify 26.8.1 [[cmath.syn](#)], header `<cmath>` synopsis, as indicated:

```

// 26.8.4 [c.math.lerp], linear interpolation
constexpr float lerp(float a, float b, float t) noexcept;
constexpr double lerp(double a, double b, double t) noexcept;
constexpr long double lerp(long double a, long double b, long double t) noexcept;

```

2. Modify 26.8.4 [[c.math.lerp](#)] as indicated:

```
constexpr float lerp(float a, float b, float t) noexcept;
constexpr double lerp(double a, double b, double t) noexcept;
constexpr long double lerp(long double a, long double b, long double t) noexcept;
```

[3226](#)⁽ⁱ⁾. zoned_time constructor from string_view should accept zoned_time<Duration2, TimeZonePtr2>

Section: 27.11.7.2 [[time.zone.zonedtime.ctor](#)] Status: [Immediate](#) Submitter: Tomasz Kamiński Opened: 2019-06-23 Last modified: 2020-02-13

Priority: 2

View all other [issues](#) in [[time.zone.zonedtime.ctor](#)].

View all issues with [Immediate](#) status.

Discussion:

zoned_time constructors with string_view and another zoned_time are not accepting zoned_time instances that use a different time zone representation (TimeZonePtr parameter):

```
zoned_time(string_view name, const zoned_time<Duration>& zt);
zoned_time(string_view name, const zoned_time<Duration>& zt, choose);
```

This makes them inconsistent with the constructors from TimeZonePtr and zoned_time, that they delegate to:

```
template<class Duration2, class TimeZonePtr2>
zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& zt);
template<class Duration2, class TimeZonePtr2>
zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& zt, choose);
```

Furthermore it forces the creation of a temporary zoned_time object in case when the source uses the same TimeZonePtr, but different Duration.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4820](#).

1. Modify 27.11.7.1 [[time.zone.zonedtime.overview](#)], class template zoned_time synopsis, as indicated:

```
template<class Duration2, class TimeZonePtr2>
zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& zt);
template<class Duration2, class TimeZonePtr2>
zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& zt, choose);
```

2. Modify 27.11.7.2 [[time.zone.zonedtime.ctor](#)] as indicated:

```
template<class Duration2, class TimeZonePtr2>
zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& y);
```

-32- *Remarks:* This constructor does not participate in overload resolution unless zoned_time is constructible from the return type of traits::locate_zone(name) and zoned_time<Duration2, TimeZonePtr2>.

-33- *Effects:* Equivalent to construction with {traits::locate_zone(name), y}.

```
template<class Duration2, class TimeZonePtr2>
zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& y, choose c);
```

-34- *Remarks:* This constructor does not participate in overload resolution unless zoned_time is constructible from the return type of traits::locate_zone(name), zoned_time<Duration2, TimeZonePtr2>, and choose.

-35- *Effects:* Equivalent to construction with {traits::locate_zone(name), y, c}.

-36- [Note: The choose parameter has no effect. — end note]

[2020-02-13, Prague]

During LWG discussions it was suggested to rebase the wording to reduce the chances for confusion.

[2020-02 Status to Immediate on Thursday morning in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 27.11.7.1 [[time.zone.zonedtime.overview](#)], class template zoned_time synopsis, as indicated:

```
template<class Duration2, class TimeZonePtr2>
zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& zt);
template<class Duration2, class TimeZonePtr2>
zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& zt, choose);
```

2. Modify 27.11.7.2 [\[time.zone.zonedtime.ctor\]](#) as indicated:

```
template<class Duration2, class TimeZonePtr2>
zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& y);
```

-32- *Constraints:* zoned_time is constructible from the return type of traits::locate_zone(name) and zoned_time<Duration2, TimeZonePtr2>.

-33- *Effects:* Equivalent to construction with {traits::locate_zone(name), y}.

```
template<class Duration2, class TimeZonePtr2>
zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& y, choose c);
```

-34- *Constraints:* zoned_time is constructible from the return type of traits::locate_zone(name), zoned_time<Duration2, TimeZonePtr2>, and choose.

-35- *Effects:* Equivalent to construction with {traits::locate_zone(name), y, c}.

-36- [Note: The choose parameter has no effect. — end note]

3237⁽ⁱ⁾. LWG 3038 and 3190 have inconsistent PRs

Section: 20.12.3.2 [\[mem.poly allocator.mem\]](#) **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2019-07-18 **Last modified:** 2020-02-14

Priority: 2

View other [active issues](#) in [mem.poly.allocator.mem].

View all other [issues](#) in [mem.poly.allocator.mem].

View all issues with [Immediate](#) status.

Discussion:

Both LWG [3038](#) and LWG [3190](#) deal with how to respond to requests to allocate "n * sizeof(T)" bytes of memory when n * sizeof(T) is not sufficient storage for n objects of type T, i.e., when n > SIZE_MAX / sizeof(T). LWG [3038](#) changed polymorphic_allocator::allocate to throw length_error upon detecting this condition, whereas LWG [3190](#) changed allocator::allocate to throw bad_array_new_length. It's peculiar that two standard library components which allocate memory both detect this condition but handle it by throwing different exception types; for consistency, the two should be harmonized.

Reflector discussion of 3190 seemed to achieve consensus that bad_array_new_length was the better option. Unlike length_error, bad_array_new_length derives from bad_alloc so we can make this change without altering the invariant that allocation functions either succeed or throw an exception derived from bad_alloc.

Further, [P0339R6](#) "polymorphic_allocator<> as a vocabulary type" recently added the function template "template<class T> T* allocate_object(size_t n = 1);" to std::pmr::polymorphic_allocator, which is another instance of the "allocate memory for n objects of type T" pattern. 20.12.3.2 [\[mem.poly.allocator.mem\]](#) paragraph 8.1 specifies that allocate_object throws length_error when SIZE_MAX / sizeof(T) < n, presumably for consistency with std::pmr::polymorphic_allocator::allocate specified in paragraph 1. allocate_object's behavior should be consistent with allocator::allocate and polymorphic_allocator::allocate so we have a single means of communicating "request for allocation of unrepresentable size" errors in the Standard Library.

[2020-02 Moved to Immediate on Thursday afternoon in Prague.]

Proposed resolution:

This wording is relative to [N4820](#).

1. Modify 20.12.3.2 [\[mem.poly.allocator.mem\]](#) as indicated:

```
[[nodiscard]] Tp* allocate(size_t n);
```

-1- *Effects:* If SIZE_MAX / sizeof(Tp) < n, throws ~~length_error~~ bad_array_new_length. Otherwise equivalent to:

```
return static_cast<Tp*>(memory_rsrc->allocate(n * sizeof(Tp), alignof(Tp)));
```

```
[...]
```

```
template<class T>
T* allocate_object(size_t n = 1);
```

-8- *Effects:* Allocates memory suitable for holding an array of n objects of type T, as follows:

(8.1) — if `SIZE_MAX / sizeof(T) < n`, throws `length_error` ~~bad array new length~~.

(8.2) — otherwise equivalent to:

```
return static_cast<T*>(allocate_bytes(n*sizeof(T), alignof(T)));
```

3238⁽ⁱ⁾. Insufficiently-defined behavior of `std::function` deduction guides

Section: 20.14.16.2.1 [[func.wrap.func.con](#)] **Status:** [Immediate](#) **Submitter:** Louis Dionne **Opened:** 2019-07-17 **Last modified:** 2020-02-14

Priority: Not Prioritized

View other [active issues](#) in [[func.wrap.func.con](#)].

View all other [issues](#) in [[func.wrap.func.con](#)].

View all issues with [Immediate](#) status.

Discussion:

The following code is currently undefined behavior:

```
#include <functional>

struct R { };
struct f0 { R operator()() && { return {}; } };

int main() { std::function f = f0{}; }
```

The reason is that 20.14.16.2.1 [[func.wrap.func.con](#)]/12 says:

This deduction guide participates in overload resolution only if `&F::operator()` is well-formed when treated as an unevaluated operand. In that case, if `decltype(&F::operator())` is of the form `R(G::*)(A...) cv &opt noexceptopt` for a class type `G`, then the deduced type is `function<R(A...)>`.

However, it does not define the behavior when `&F::operator()` is well-formed but *not* of the required form (in the above example it's of the form `R(G::*)(A...) &&`, which is rvalue-reference qualified instead of optionally-lvalue-reference qualified). libc++'s implementation of the deduction guide SFINAE's out when either `&F::operator()` is not well-formed, or it is not of the required form. It seems like mandating that behavior in the Standard is the way to go.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4820](#).

1. Modify 20.14.16.2.1 [[func.wrap.func.con](#)] as indicated:

```
template<class F> function(F) -> function<see below>;
```

-12- Remarks: This deduction guide participates in overload resolution only if `&F::operator()` is well-formed when treated as an unevaluated operand, and its type is of the form `R(G::*)(A...) cv &opt noexceptopt` for a class type `G` and a sequence of types `A...`. In that case, ~~if `decltype(&F::operator())` is of the form `R(G::*)(A...) cv &opt noexceptopt` for a class type `G`, then~~ the deduced type is `function<R(A...)>`.

[2020-02-13; Prague]

LWG improves wording matching Marshall's Mandating paper.

[Status to Immediate on Friday in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 20.14.16.2.1 [[func.wrap.func.con](#)] as indicated:

[Drafting note: This edit should be used instead of the corresponding edit in [P1460](#)]

```
template<class F> function(F) -> function<see below>;
```

~~?- Constraints: `&F::operator()` is well-formed when treated as an unevaluated operand and `decltype(&F::operator())` is of the form `R(G::*)(A...) cv &opt noexceptopt` for a class type `G`.~~

-12- Remarks: This deduction guide participates in overload resolution only if `&F::operator()` is well formed when treated as an unevaluated operand. In that case, if `decltype(&F::operator())` is of the form `R(G::*)(A...)` cv `&opt` `noexceptopt` for a class type `G`, then `T` the deduced type is `function<R(A...)>`.

[3242^{\(i\)}](#). `std::format`: missing rules for `arg-id` in width and precision

Section: 20.20.2.2 [\[format.string.std\]](#) Status: [Immediate](#) Submitter: Richard Smith Opened: 2019-07-31 Last modified: 2020-02-13

Priority: 1

View other [active issues](#) in `[format.string.std]`.

View all other [issues](#) in `[format.string.std]`.

View all issues with [Immediate](#) status.

Discussion:

According to 20.20.2 [\[format.string\]](#) we have:

If the numeric `arg-ids` in a format string are `0`, `1`, `2`, ... in sequence, they can all be omitted (not just some) and the numbers `0`, `1`, `2`, ... will be automatically used in that order. A format string does not contain a mixture of automatic and manual indexing.

... but what does that mean in the presence of `arg-id` in width and precision? Can one replace

```
"{0:{1}} {2}"
```

with

```
"{:{} } {}"
```

? The grammar says the answer is no, because the `arg-id` in width is not optional, but the normative wording says the answer is yes.

Victor Zverovich:

That's a bug in the grammar. The `arg-id` should be optional in width and precision:

```
width      ::= nonzero-digit [integer] | '{' [arg-id] '}'
precision ::= integer | '{' [arg-id] '}'
```

[2020-02 Status to Immediate on Thursday morning in Prague.]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify the *width* and *precision* grammar in the syntax of format specifications of 20.20.2.2 [\[format.string.std\]](#) as indicated:

```
[...]
width:
    positive-integer
    { arg-idopt }
precision:
    . nonnegative-integer
    . { arg-idopt }
[...]
```

2. Modify 20.20.2.2 [\[format.string.std\]](#) as indicated:

-6- The `#` option causes [...]

?:- If `{ arg-idopt }` is used in a *width* or *precision*, the value of the corresponding formatting argument is used in its place. If the corresponding formatting argument is not of integral type, or its value is negative for *precision* or non-positive for *width*, an exception of type `format_error` is thrown.

-7- The *positive-integer* in *width* is a decimal integer defining the minimum field width. If *width* is not specified, there is no minimum field width, and the field width is determined based on the content of the field.

[3243^{\(i\)}](#). `std::format` and negative zeroes

Section: 20.20.2 [\[format.string\]](#) Status: [Immediate](#) Submitter: Richard Smith Opened: 2019-07-31 Last modified: 2020-02-14

Priority: 2

View other [active issues](#) in `[format.string]`.

View all other [issues](#) in [format.string].

View all issues with [Immediate](#) status.

Discussion:

What are these:

```
std::format("{} ", -0.0);
std::format("{: } ", -0.0);
std::format("{:~ } ", -0.0);
std::format("{: } ", -0.0);

with

"{: {} } {} "
```

A negative zero is not a negative number, so I think the answer for an implementation that supports signed zeroes is "0", "-0", "0", " 0". Is that the intent? (Note that this doesn't satisfy to_chars' round-trip guarantee.)

Or should the behavior actually be that "+" means "insert a leading + if to_chars' output does not start with -" and " " actually means "insert a leading space if to_chars' output does not start with -" (that is, the same as "%+f" and "% f") — so that the result of all four calls is "-0"?

Victor Zverovich:

The latter. std::format is specified in terms of to_chars and the intent is to have similar round trip guarantee here, so the result should be "-0", "-0", "-0", "-0". This has also been extensively discussed in LEWG and the outcome of that discussion was to print '-' for negative zeros.

So I think it should be clarified that '-' and space apply to negative numbers and negative zero.

[2019-08-17 Priority set to 2 based on reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

- 1. Modify the sign options Table [tab:format.sign] in 20.20.2.2 [\[format.string.std\]](#) as indicated:

Table 59: Meaning of sign options [tab:format.sign]	
Option	Meaning
'+'	Indicates that a sign should be used for both non-negative and negative numbers.
'-'	Indicates that a sign should be used only for negative numbers and negative zero (this is the default behavior).
space	Indicates that a leading space should be used for non-negative numbers other than negative zero , and a minus sign for negative numbers and negative zero .

[2020-02-13, Prague]

Rebased and some wording finetuning by LWG.

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

- 1. Modify the sign options Table [tab:format.sign] in 20.20.2.2 [\[format.string.std\]](#) as indicated:

Table 58: Meaning of sign options [tab:format.sign]	
Option	Meaning
'+'	Indicates that a sign should be used for both non-negative and negative numbers.
'-'	Indicates that a sign should be used only for negative numbers and negative zero only (this is the default behavior).
space	Indicates that a leading space should be used for non-negative numbers other than negative zero , and a minus sign for negative numbers and negative zero .

[3247^{\(i\)}](#). ranges::iter_move should perform ADL-only lookup of iter_move

Section: 23.3.3.1 [\[iterator.cust.move\]](#) Status: [Immediate](#) Submitter: Casey Carter Opened: 2019-07-29 Last modified: 2020-02-13

Priority: 2

View other [active issues](#) in [iterator.cust.move].

View all other [issues](#) in [iterator.cust.move].

View all issues with [Immediate](#) status.

Discussion:

The specification of the `ranges::iter_move` customization point in `[iterator.cust.move]` doesn't include a deleted poison pill overload. There is no `std::iter_move` to avoid, so such a poison pill is not needed. This is fine, except that it suggests that unqualified lookup for `iter_move` in the `iter_move(E)` expression in paragraph 1.1 should find declarations of `iter_move` in the global namespace via normal unqualified lookup, when the design intent is that only ADL be used to find overloads of `iter_move`.

Absent a more idiomatic wording to specify an ADL-only lookup, we can correct the problem by specifying the lookup in paragraph 1.1 is performed in a context that includes the declaration `"void iter_move();"` which has the desired effect.

[2020-02 Status to Immediate on Thursday morning in Prague.]

Proposed resolution:

This wording is relative to [N4820](#).

[Drafting Note: There's a drive-by fix here to change "valid" — which suggests runtime behavior which cannot be validated at compile time — to "well-formed".]

1. Modify 23.3.3.1 [\[iterator.cust.move\]](#) as indicated:

-1- The name `ranges::iter_move` denotes a customization point object (16.4.2.2.6 [\[customization.point.object\]](#)). The expression `ranges::iter_move(E)` for some subexpression `E` is expression-equivalent to:

(1.1) — `iter_move(E)`, if that expression is **valid well-formed when treated as an unevaluated operand**, with overload resolution performed in a context that does not include a declaration of `ranges::iter_move`; **but does include the declaration:**

`void iter_move();`

[...]

[3248^{\(i\)}](#). `std::format` #b, #B, #o, #x, and #X presentation types misformat negative numbers

Section: 20.20.2.2 [\[format.string.std\]](#) **Status:** [Immediate](#) **Submitter:** Richard Smith **Opened:** 2019-08-01 **Last modified:** 2020-02-14

Priority: 2

View other [active issues](#) in `[format.string.std]`.

View all other [issues](#) in `[format.string.std]`.

View all issues with [Immediate](#) status.

Discussion:

According to both the specification for '#' and the presentation types `b`, `B`, `o`, `x`, and `X` (which redundantly duplicate the rule for the relevant prefixes), the string `0b`, `0B`, `0`, `0x`, or `0X` is prefixed to the result of `to_chars`. That means:

```
std::format("{0:#b} {0:#B} {0:#o} {0:#x} {0:#X}", -1)
```

produces

```
"0b-1 0B-1 0-1 0x-1 0X-1"
```

I assume that's a bug?

(Additionally, if the `+` specifier is used, there appears to be no specification of where the sign is inserted into the output.)

Victor Zverovich:

Yes. I suggest replacing

adds the respective prefix `"0b"` (`"0B"`), `"0"`, or `"0x"` (`"0X"`) to the output value

with something like

inserts the respective prefix `"0b"` (`"0B"`), `"0"`, or `"0x"` (`"0X"`) to the output value after the sign, if any,

I think the duplicate wording in the presentation types `b`, `B`, `o`, `x`, and `X` can be dropped.

Regarding the `+` specifier problem: How about adding

The sign is inserted before the output of `to_chars`.

?

[2019-08-21 Priority set to 2 based on reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

1. Modify the *sign* options Table [tab:format.sign] in 20.20.2.2 [format.string.std] as indicated:

Table 59: Meaning of *sign* options [tab:format.sign]

Option	Meaning
'+'	Indicates that a sign should be used for both non-negative and negative numbers. The sign is inserted before the output of <code>to_chars</code> .
'-'	Indicates that a sign should be used only for negative numbers (this is the default behavior).
space	Indicates that a leading space should be used for non-negative numbers, and a minus sign for negative numbers.

2. Modify [format.string] as indicated:

-6 The `#` option causes the *alternate form* to be used for the conversion. This option is only valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified. For integral types, the alternate form ~~adds~~ inserts the base prefix (if any) specified in Table [tab:format.type.int] to the output value after the sign, if any. [...]

[2019-08-21; Victor Zverovich provides improved wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

1. Modify the *sign* options Table [tab:format.sign] in 20.20.2.2 [format.string.std] as indicated:

Table 59: Meaning of *sign* options [tab:format.sign]

Option	Meaning
'+'	Indicates that a sign should be used for both non-negative and negative numbers. The + sign is inserted before the output of <code>to_chars</code> for non-negative numbers other than the negative zero. [Note: For negative numbers and the negative zero the output of <code>to_chars</code> will already contain the sign so no additional transformation is performed. — end note]
'-'	Indicates that a sign should be used only for negative numbers (this is the default behavior).
space	Indicates that a leading space should be used for non-negative numbers, and a minus sign for negative numbers.

2. Modify [format.string] as indicated:

-6- The `#` option causes the *alternate form* to be used for the conversion. This option is only valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified. For integral types, the alternate form ~~adds~~ inserts the base prefix (if any) specified in Table [tab:format.type.int] to the output value after the sign character (possibly space) if there is one, or before the output of `to_chars` otherwise. [...]

[2020-02-13, Prague]

LWG made some improvements to the wording.

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify the *sign* options Table [tab:format.sign] in 20.20.2.2 [format.string.std] as indicated:

Table 58: Meaning of *sign* options [tab:format.sign]

Option	Meaning
'+'	Indicates that a sign should be used for both non-negative and negative numbers. The + sign is inserted before the output of <code>to_chars</code> for non-negative numbers other than negative zero. [Note: For negative numbers and negative zero the output of <code>to_chars</code> will already contain the sign so no additional transformation is performed. — end note]
'-'	Indicates that a sign should be used only for negative numbers (this is the default behavior).
space	Indicates that a leading space should be used for non-negative numbers, and a minus sign for negative numbers.

2. Modify [format.string] as indicated:

-6- The # option causes the *alternate form* to be used for the conversion. This option is **only** valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified, **and not otherwise**. For integral types, the alternate form **adds** the base prefix (if any) specified in Table [tab:format.type.int] **to** the output **value** **after the sign character (possibly space) if there is one, or before the output of to_chars otherwise**. [...]

[3250^{\(i\)}](#). `std::format: # (alternate form) for NaN and inf`

Section: 20.20.2.2 [format.string.std] **Status:** [Immediate](#) **Submitter:** Richard Smith **Opened:** 2019-08-05 **Last modified:** 2020-02-13

Priority: 0

View other [active issues](#) in [format.string.std].

View all other [issues](#) in [format.string.std].

View all issues with [Immediate](#) status.

Discussion:

We have:

"For floating-point numbers, the alternate form causes the result of the conversion to always contain a decimal-point character, even if no digits follow it."

So does that mean that infinity is formatted as "inf." and NaN as "nan."? (Or something like that? Where exactly do we add the decimal point in this case?) Or does this affect infinity but not NaN (because we can handwave that a NaN value is not a "floating-point number")?

I suspect that this should only cover finite floating-point numbers.

Victor Zverovich:

Right. So infinity and NaN should be still formatted as "inf" and "nan" without a decimal point.

[2020-02 Status to Immediate on Thursday morning in Prague.]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 20.20.2.2 [format.string.std] as indicated:

-6- [...] For floating-point types, the alternate form causes the result of the conversion **of finite values** to always contain a decimal-point character, even if no digits follow it. Normally, a decimal-point character appears in the result of these conversions only if a digit follows it. In addition, for g and G conversions, trailing zeros are not removed from the result.

[3251^{\(i\)}](#). `Are std::format alignment specifiers applied to string arguments?`

Section: 20.20.2 [format.string] **Status:** [Immediate](#) **Submitter:** Richard Smith **Opened:** 2019-08-02 **Last modified:** 2020-02-14

Priority: 2

View other [active issues](#) in [format.string].

View all other [issues](#) in [format.string].

View all issues with [Immediate](#) status.

Discussion:

We are told:

Formatting of objects of arithmetic types and `const void*` is done as if by calling `to_chars` (unless otherwise specified) and copying the output through the output iterator of the format context with additional padding and adjustments as specified by the format specifiers.

... but there is no corresponding rule for strings. Is an alignment specifier intended to be applied to strings or not? The wording as-is is ambiguous.

(The above also doesn't cover formatting `void*` or `std::nullptr_t`. Presumably at least those two should have the relevant adjustments applied to them!)

The wording never actually anywhere says that the `basic_format_args` are in any way involved in the formatting process, or how formatting actually happens. (The wording doesn't say that `basic_format_arg::handle::format` is ever called, for example.)

Victor Zverovich:

An alignment specifier is intended to be applied to strings as well, `void*` and `std::nullptr_t` are converted into `const void*` when constructing `basic_format_arg`.

The wording for `vformat` and similar functions says that `basic_format_args` is involved:

Returns: A string object holding the character representation of formatting arguments provided by `args` formatted according to specifications given in `fmt`.

but I admit that it is hand-wavy. Perhaps we could add something along the lines of

For each replacement field referring to the argument with index (`arg-id`) `i`, the `basic_format_arg` object referring to the argument is obtained via `args.get(i)` and the `parse` and `format` functions of the formatter specialization for the underlying argument type are called to parse the format specification and format the value.

to clarify how we format `args` (`basic_format_args`).

[2019-08-21 Priority set to 2 based on reflector discussion]

[2019-08-21; Victor Zverovich suggests wording]

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 20.20.2.2 [\[format.string.std\]](#) as indicated:

-3- [The align specifier applies to all argument types](#). The meaning of the various alignment options is as specified in Table [\[tab:format.align\]](#). [Example: [...

[3252\(i\)](#). Parse locale's aware modifiers for commands are not consistent with POSIX spec

Section: 27.13 [\[time.parse\]](#) **Status:** [Immediate](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-08-06 **Last modified:** 2020-02-13

Priority: 2

View other [active issues](#) in [\[time.parse\]](#).

View all other [issues](#) in [\[time.parse\]](#).

View all issues with [Immediate](#) status.

Discussion:

The current specification of the locale modifiers for the parse flags in "[[tab:time.parse.spec](#)] Meaning of parse flags" is inconsistent with the [POSIX strptime specification](#):

- New flags `%0C`, `%0u` are added
- Flags `%0I`, `%0U`, `%0W` are missing

Per Howard's comment:

I only invented in a couple places for these flags, and none of them involved locale-dependent stuff. If we are inconsistent with POSIX/C on this, it's a bug. Rationale, `std::get_time` is already specified in terms of `strptime`, and we can't afford to be inventive with locale-dependent things.

Note that, due above, the inconsistency between [POSIX strptime specification](#) that supports `%0u` and `%0v` that are not handled by [strptime](#) should be (by design) reflected in "[[tab:time.format.spec](#)] Meaning of conversion specifiers" and "[[tab:time.parse.spec](#)] Meaning of parse flags" tables.

The `%d` modifier was addressed by LWG [3218](#).

[2020-02 Status to Immediate on Thursday morning in Prague.]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify Table 99 "Meaning of parse flags [[tab:time.parse.spec](#)]" in 27.13 [\[time.parse\]](#) as indicated:

Table 99: Meaning of parse flags [tab:time.parse.spec]	
Flag	Parsed value
[...]	

%C	The century as a decimal number. The modified command % <i>NC</i> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified commands %EC and %OC interpret the locale's alternative representation of the century.
[...]	
%I	The hour (12-hour clock) as a decimal number. The modified command % <i>NI</i> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command %OI interprets the locale's alternative representation.
[...]	
%u	The ISO weekday as a decimal number (1-7), where Monday is 1. The modified command % <i>Nu</i> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 1. Leading zeroes are permitted but not required. The modified command %Ou interprets the locale's alternative representation.
%U	The week number of the year as a decimal number. The first Sunday of the year is the first day of week 01. Days of the same year prior to that are in week 00. The modified command % <i>NU</i> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command %OU interprets the locale's alternative representation.
[...]	
%W	The week number of the year as a decimal number. The first Monday of the year is the first day of week 01. Days of the same year prior to that are in week 00. The modified command % <i>NW</i> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command %OW interprets the locale's alternative representation.
[...]	

3255(i). span's array constructor is too strict

Section: 22.7.3.2 [\[span.cons\]](#) Status: [Immediate](#) Submitter: Jean Guegant & Barry Revzin Opened: 2019-08-10 Last modified: 2020-02-14

Priority: 2

View other [active issues](#) in [\[span.cons\]](#).

View all other [issues](#) in [\[span.cons\]](#).

View all issues with [Immediate](#) status.

Discussion:

Barry Revzin:

From [StackOverflow](#):

This compiles:

```
std::vector<int*> v = {nullptr, nullptr};
std::span<const int* const> s{v};
```

This does not:

```
std::array<int*, 2> a = {nullptr, nullptr};
std::span<const int* const> s{a};
```

The problem is that span's constructors include

- A constructor template that takes any Container that is neither a raw array nor a `std::array`
- A constructor template that takes an `array<value_type, N>&`
- A constructor template that takes a `const array<value_type, N>&`

So the first is excluded, and the other two don't match. We can change the array constructor templates to take an `array<T, N>` with the requirement that `T(*)[]` is convertible to `ElementType(*)[]`?

Jean Guegant:

It is impossible to create a `std::span` from a `std::array<const T, X>` given the current set of constructors of `std::span` (22.7.3.2 [\[span.cons\]](#)):

```
std::array<const int, 4> a = {1, 2, 3, 4};
std::span<const int> s{a}; // No overload can be found.
std::span s{a}; // CTAD doesn't help either.
```

Both constructors accepting a `std::array` (22.7.3.2 [\[span.cons\]](#) p11) require the first template parameter of the `std::array` parameter to be `value_type`:

```
template<size_t N> constexpr span(array<value_type, N>& arr) noexcept;
template<size_t N> constexpr span(const array<value_type, N>& arr) noexcept;
```

`value_type` being defined as `remove_cv_t<ElementType>` — this constrains the first template parameter not to be `const`.

Both constructors accepting a generic Container (22.7.3.2 [\[span.cons\]](#) p14) have a constraint — (p14.3) Container is not a specialization of array — rejecting `std::array`.

While you can call `std::array<const T, X>::data` and `std::array<const T, X>::size` to manually create a `std::span`, we should, in my opinion, offer a proper overload for this scenario. Two reasons came to my mind:

1. `std::span` handles C-arrays and `std::arrays` in an asymmetric way. The constructor taking a C-array (22.7.3.2 [\[span.cons\]](#) p11) is using `element_type` and as such can work with `const T`:

```
const int a[] = {1, 2, 3, 4};
std::span<const int> s{a}; // It works
```

If a user upgrades her/his code from C-arrays to `std::arrays` and literally take the type `const T` and use it as the first parameter of `std::array`, he/she will face an error.

2. Even if the user is aware that `const std::array<T, X>` is more idiomatic than `std::array<const T, X>`, the second form may appear in the context of template instantiation.

At the time this issue is written `gsl::span`, from which `std::span` is partly based on, does not suffer from the same issue: Its constructor taking a generic `const Container&` does not constraint the Container not to be a `std::array` (although its constructor taking a generic `Container&` does). For the users willing to upgrade from `gsl::span` to `std::span`, this could be a breaking change.

[2019-09-01 Priority set to 2 based on reflector discussion]

[2020-02-13 Tim updates PR]

The previous PR's change to the raw array constructor is both 1) unnecessary and 2) incorrect; it prevents `span<const int>` from being initialized with an `int[42]` xvalue.

Previous resolution: [SUPERSEDED]

This wording is relative to [N4830](#).

The only change is to make the constructors templated on the element type of the array as well. We already have the right constraints in place. It's just that the 2nd constraint is trivially satisfied today by the raw array constructor and either always or never satisfied by the `std::array` one.

1. Modify 22.7.3.1 [\[span.overview\]](#), class template `span` synopsis, as indicated:

```
template<class ElementType, size_t Extent = dynamic_extent>
class span {
public:
    [...]
    // 22.7.3.2 \[span.cons\], constructors, copy, and assignment
    constexpr span() noexcept;
    constexpr span(pointer ptr, index_type count);
    constexpr span(pointer first, pointer last);
    template<class T, size_t N>
        constexpr span(ElementType (&arr)[N]) noexcept;
    template<class T, size_t N>
        constexpr span(array<Tvalue_type, N>& arr) noexcept;
    template<class T, size_t N>
        constexpr span(const array<Tvalue_type, N>& arr) noexcept;
    [...]
};
```

2. Modify 22.7.3.2 [\[span.cons\]](#) as indicated:

```
template<class T, size_t N>
    constexpr span(ElementType (&arr)[N]) noexcept;
template<class T, size_t N>
    constexpr span(array<Tvalue_type, N>& arr) noexcept;
template<class T, size_t N>
    constexpr span(const array<Tvalue_type, N>& arr) noexcept;
```

-11- Constraints:

(11.1) — `extent == dynamic_extent || N == extent` is true, and

(11.2) — `remove_pointer_t<decltype(data(arr))>(*)[]` is convertible to `ElementType(*)[]`.

-12- Effects: Constructs a span that is a view over the supplied array.

-13- Ensures: `size() == N` && `data() == data(arr)` is true.

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 22.7.3.1 [\[span.overview\]](#), class template span synopsis, as indicated:

```
template<class ElementType, size_t Extent = dynamic_extent>
class span {
public:
    [...]
    // 22.7.3.2 \[span.cons\], constructors, copy, and assignment
    constexpr span() noexcept;
    [...]
    template<size_t N>
        constexpr span(element_type (&arr)[N]) noexcept;
    template<class T, size_t N>
        constexpr span(array<Tvalue_type, N>& arr) noexcept;
    template<class T, size_t N>
        constexpr span(const array<Tvalue_type, N>& arr) noexcept;
    [...]
};
```

2. Modify 22.7.3.2 [\[span.cons\]](#) as indicated:

```
template<size_t N>
    constexpr span(element_type (&arr)[N]) noexcept;
template<class T, size_t N>
    constexpr span(array<Tvalue_type, N>& arr) noexcept;
template<class T, size_t N>
    constexpr span(const array<Tvalue_type, N>& arr) noexcept;
```

-11- Constraints:

(11.1) — extent == dynamic_extent || N == extent is true, and

(11.2) — remove_pointer_t<decltype(data(arr))>(*)[] is convertible to ElementType(*)[].

-12- Effects: Constructs a span that is a view over the supplied array.

-13- Postconditions: size() == N && data() == data(arr) **is true**.

[3260^{\(i\)}](#). year_month* arithmetic rejects durations convertible to years

Section: 27.8 [\[time.cal\]](#) Status: [Immediate](#) Submitter: Tomasz Kamiński Opened: 2019-08-15 Last modified: 2020-02-14

Priority: 2

View all issues with [Immediate](#) status.

Discussion:

Currently, the year_month* types (year_month, year_month_day) provide separate arithmetic operators with duration type of years or months. This is an intentional optimization that avoids performing modulo arithmetic in the former case.

However, these make the arithmetic of year_month* types with durations that are convertible to years (and by consequence to months) ambiguous. For example, the following code is ambiguous:

```
using decades = duration<int, ratio_multiply<ratio<10>, years::period>>;
auto ymd = 2001y/January/1d;
ymd += decades(1); // error, ambiguous
```

while less usual durations that are only convertible to months work correctly:

```
using decamonths = duration<int, ratio_multiply<ratio<10>, months::period>>;
auto ymd = 2001y/January/1d;
ymd += decamonths(1);
```

The example implementation resolves the issues by making sure that the years overload will be preferred in case of ambiguity, by declaring the months overload a function template with a default argument for its parameter (suggested by Tim Song):

```
template<class = unspecified>
constexpr year_month_weekday& operator+=(const months& m) noexcept;
constexpr year_month_weekday& operator+=(const years& m) noexcept;
```

[2019-09-14 Priority set to 2 based on reflector discussion]

[2019-09-14; Tomasz and Howard provide concrete wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

[Drafting note: Suggested wording below assumes that we can add a *Constraints*: to a signature where the constraint does not apply to a deduced template. We have examples of such constraints in other parts of the WD (e.g. 20.11.1.2.1 [\[unique.ptr.single.ctor\]](#)/p15, 20.11.1.2.3 [\[unique.ptr.single.assign\]](#)/p1). And we have the old form "does not participate ..." being used for non-deduced templates in several places as well (e.g. 20.4.2 [\[pairs.pair\]](#)/p5).

There are several ways of implementing such a constraint, such as adding a gratuitous template parameter.]

1. Modify 27.8.13.2 [\[time.cal.ym.members\]](#) as indicated:

```
constexpr year_month& operator+=(const months& dm) noexcept;
```

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-4- Effects: *this = *this + dm.

[...]

```
constexpr year_month& operator-=(const months& dm) noexcept;
```

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-6- Effects: *this = *this - dm.

[...]

2. Modify 27.8.13.3 [\[time.cal.ym.nonmembers\]](#) as indicated:

```
constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
```

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-3- Returns: A year_month value z such that z - ym == dm.

[...]

```
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
```

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-5- Returns: ym + dm.

```
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
```

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-6- Returns: ym - dm.

3. Modify 27.8.14.2 [\[time.cal.ymd.members\]](#) as indicated:

```
constexpr year_month_day& operator+=(const months& m) noexcept;
```

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-7- Effects: *this = *this + m.

[...]

```
constexpr year_month_day& operator-=(const months& m) noexcept;
```

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-9- Effects: *this = *this - m.

[...]

4. Modify 27.8.14.3 [\[time.cal.ymd.nonmembers\]](#) as indicated:

constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-3- Returns: (ymd.year() / ymd.month() + dm) / ymd.day().

[...]

constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-5- Returns: ymd + dm.

constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-6- Returns: ymd + (-dm).

5. Modify 27.8.15.2 [\[time.cal.ymdlast.members\]](#) as indicated:

constexpr year_month_day_last& operator+=(const months& m) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-2- Effects: *this = *this + m.

[...]

constexpr year_month_day_last& operator-=(const months& m) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-4- Effects: *this = *this - m.

[...]

6. Modify 27.8.15.3 [\[time.cal.ymdlast.nonmembers\]](#) as indicated:

constexpr year_month_day_last
operator+(const year_month_day_last& ymdl, const months& dm) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-3- Returns: (ymdl.year() / ymdl.month() + dm) / last.

constexpr year_month_day_last
operator+(const months& dm, const year_month_day_last& ymdl) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-4- Returns: ymdl + dm.

constexpr year_month_day_last
operator-(const year_month_day_last& ymdl, const months& dm) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-5- Returns: ymdl + (-dm).

7. Modify 27.8.16.2 [\[time.cal.ymwd.members\]](#) as indicated:

constexpr year_month_weekday& operator+=(const months& m) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-6- Effects: *this = *this + m.

[...]

constexpr year_month_weekday& operator-=(const months& m) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-8- Effects: *this = *this - m.

[...]

8. Modify 27.8.16.3 [\[time.cal.ymwd.nonmembers\]](#) as indicated:

constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const months& dm) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-2- Returns: (ymwd.year() / ymwd.month() + dm) / ymwd.weekday_indexed().

constexpr year_month_weekday operator+(const months& dm, const year_month_weekday& ymwd) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-3- Returns: ymwd + dm.

constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const months& dm) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-4- Returns: ymwd + (-dm).

9. Modify 27.8.17.2 [\[time.cal.ymwdlast.members\]](#) as indicated:

constexpr year_month_weekday_last& operator+=(const months& m) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-2- Effects: *this = *this + m.

[...]

constexpr year_month_weekday_last& operator-=(const months& m) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-4- Effects: *this = *this - m.

[...]

10. Modify 27.8.17.3 [\[time.cal.ymwdlast.nonmembers\]](#) as indicated:

constexpr year_month_weekday_last
operator+(const year_month_weekday_last& ymwdl, const months& dm) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-2- Returns: (ymwdl.year() / ymwdl.month() + dm) / ymwdl.weekday_last().

constexpr year_month_weekday_last
operator+(const months& dm, const year_month_weekday_last& ymwdl) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-3- Returns: ymwdl + dm.

constexpr year_month_weekday_last
operator-(const year_month_weekday_last& ymwdl, const months& dm) noexcept;

-?- Constraints: The argument supplied by the caller for the months parameter is not implicitly convertible to years.

-4- Returns: ymwdl + (-dm).

[2020-02-13, Prague]

Tim Song found a wording problem that we would like to resolve:

Given a class like

```
struct C : months {
    operator years();
};
```

The previous wording requires calls with a C argument to use the years overload, which would require implementation heroics since its conversion sequence to months is better than years.

[2020-02 Status to Immediate on Friday morning in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

[Drafting note: Suggested wording below assumes that we can add a *Constraints*: to a signature where the constraint does not apply to a deduced template. We have examples of such constraints in other parts of the WD (e.g. 20.11.1.2.1 [\[unique.ptr.single.ctor\]/p15](#), 20.11.1.2.3 [\[unique.ptr.single.assign\]/p1](#)). And we have the old form "does not participate ..." being used for non-deduced templates in several places as well (e.g. 20.4.2 [\[pairs.pair\]/p5](#)).

There are several ways of implementing such a constraint, such as adding a gratuitous template parameter.]

1. Modify 27.8.13.2 [\[time.cal.ym.members\]](#) as indicated:

```
constexpr year_month& operator+=(const months& dm) noexcept;
```

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-4- Effects: *this = *this + dm.

[...]

```
constexpr year_month& operator-=(const months& dm) noexcept;
```

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-6- Effects: *this = *this - dm.

[...]

2. Modify 27.8.13.3 [\[time.cal.ym.nonmembers\]](#) as indicated:

```
constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
```

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-3- Returns: A year_month value z such that z - ym == dm.

[...]

```
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
```

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-5- Returns: ym + dm.

```
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
```

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-6- Returns: ym + -dm.

3. Modify 27.8.14.2 [\[time.cal.ymd.members\]](#) as indicated:

```
constexpr year_month_day& operator+=(const months& m) noexcept;
```

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-7- Effects: *this = *this + m.

[...]

constexpr year_month_day& operator+=(const months& m) noexcept;

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-9- Effects: *this = *this - m.

[...]

4. Modify 27.8.14.3 [\[time.cal.ymd.nonmembers\]](#) as indicated:

constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-3- Returns: (ymd.year() / ymd.month() + dm) / ymd.day().

[...]

constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-5- Returns: ymd + dm.

constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-6- Returns: ymd + (-dm).

5. Modify 27.8.15.2 [\[time.cal.ymdlast.members\]](#) as indicated:

constexpr year_month_day_last& operator+=(const months& m) noexcept;

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-2- Effects: *this = *this + m.

[...]

constexpr year_month_day_last& operator+=(const months& m) noexcept;

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-4- Effects: *this = *this - m.

[...]

6. Modify 27.8.15.3 [\[time.cal.ymdlast.nonmembers\]](#) as indicated:

constexpr year_month_day_last
operator+(const year_month_day_last& ymdl, const months& dm) noexcept;

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-3- Returns: (ymdl.year() / ymdl.month() + dm) / last.

constexpr year_month_day_last
operator+(const months& dm, const year_month_day_last& ymdl) noexcept;

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-4- Returns: ymdl + dm.

constexpr year_month_day_last
operator-(const year_month_day_last& ymdl, const months& dm) noexcept;

-?- *Constraints:* If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-5- *Returns:* ymdl + (-dm).

7. Modify 27.8.16.2 [\[time.cal.ymwd.members\]](#) as indicated:

```
constexpr year_month_weekday& operator+=(const months& m) noexcept;
```

-?- *Constraints:* If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-6- *Effects:* *this = *this + m.

[...]

```
constexpr year_month_weekday& operator-=(const months& m) noexcept;
```

-?- *Constraints:* If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-8- *Effects:* *this = *this - m.

[...]

8. Modify 27.8.16.3 [\[time.cal.ymwd.nonmembers\]](#) as indicated:

```
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const months& dm) noexcept;
```

-?- *Constraints:* If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-2- *Returns:* (ymwd.year() / ymwd.month() + dm) / ymwd.weekday_indexed().

```
constexpr year_month_weekday operator+(const months& dm, const year_month_weekday& ymwd) noexcept;
```

-?- *Constraints:* If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-3- *Returns:* ymwd + dm.

```
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const months& dm) noexcept;
```

-?- *Constraints:* If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-4- *Returns:* ymwd + (-dm).

9. Modify 27.8.17.2 [\[time.cal.ymwdlast.members\]](#) as indicated:

```
constexpr year_month_weekday_last& operator+=(const months& m) noexcept;
```

-?- *Constraints:* If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-2- *Effects:* *this = *this + m.

[...]

```
constexpr year_month_weekday_last& operator-=(const months& m) noexcept;
```

-?- *Constraints:* If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-4- *Effects:* *this = *this - m.

[...]

10. Modify 27.8.17.3 [\[time.cal.ymwdlast.nonmembers\]](#) as indicated:

```
constexpr year_month_weekday_last  
operator+(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
```

-?- *Constraints:* If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [\[over.ics.rank\]](#)).

-2- *Returns:* (ymwdl.year() / ymwdl.month() + dm) / ymwdl.weekday_last().

```
constexpr year_month_weekday_last
operator+(const months& dm, const year_month_weekday_last& ymwdl) noexcept;
```

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [over.ics.rank]).

-3- Returns: ymwdl + dm.

```
constexpr year_month_weekday_last
operator-(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
```

-?- Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.4.3.2 [over.ics.rank]).

-4- Returns: ymwdl + (-dm).

3262⁽ⁱ⁾. Formatting of negative durations is not specified

Section: 27.12 [time.format] **Status:** [Immediate](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-08-18 **Last modified:** 2020-02-13

Priority: 2

View other [active issues](#) in [time.format].

View all other [issues](#) in [time.format].

View all issues with [Immediate](#) status.

Discussion:

The current specification of the formatting for `std::chrono::duration` and `std::hh_mm_ss` types is unclear in regards the the handling of negative values. To illustrate:

```
std::cout << std::format("%H:%M:%S", -10'000s); // prints either -02:46:40 or -02:-46:-40
```

The indented behavior (and currently implemented, see [here](#)) is to apply the sign once, before the leftmost converted field.

[2019-09-14 Priority set to 2 based on reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

[Drafting note: With the above clarification, the specification of the operator<< for `hh_mm_ss` may be simplified to `format("{:%T}", hms).`]

1. Modify 27.12 [time.format] as indicated:

-2- Each conversion specifier *conversion-spec* is replaced by appropriate characters as described in Table [tab:time.format.spec]. Some of the conversion specifiers depend on the locale that is passed to the formatting function if the latter takes one, or the global locale otherwise. If the formatted object does not contain the information the conversion specifier refers to, an exception of type `format_error` is thrown.

-?- The result of formatting a `std::chrono::duration` instance holding a negative value, or of an `hh_mm_ss` object `h` for which `h.is_negative()` is true, is equivalent to the output of the corresponding positive value, with a `-` character placed before the replacement of the leftmost conversion specifier.

[Example:

```
cout << format("%T", -10'000s); // prints: -02:46:40
cout << format("%H:%M:%S", -10'000s); // prints: -02:46:40
cout << format("minutes %M, hours %H, seconds %S", -10'000s); // prints: minutes -46, hours 02, seconds 40
```

— end example]

-3- Unless explicitly requested, [...]

2. Modify 27.9.3 [time.hms.nonmembers] as indicated:

```
template<class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const hh_mm_ss<Duration>& hms);
```

-1- Effects: Equivalent to:


```

return os << format(os.getloc(),
                    hms.is_negative() ? STATICALLY_WIDEN<charT>("{:%T}")
                                     : STATICALLY_WIDEN<charT>("{:%T}"),
                    abs(hms.to_duration()));

```

[2019-09-14; Howard improves wording]

[2020-02; Status set to Immediate after LWG discussion Thursday in Prague. (Minor example wording cleanup)]

Proposed resolution:

This wording is relative to [N4830](#).

[Drafting note: With the above clarification, the specification of the operator<< for hh_mm_ss may be simplified to `format("{:%T}", hms)`.]

1. Modify 27.12 [\[time.format\]](#) as indicated:

-2- Each conversion specifier *conversion-spec* is replaced by appropriate characters as described in Table [tab:time.format.spec]. Some of the conversion specifiers depend on the locale that is passed to the formatting function if the latter takes one, or the global locale otherwise. If the formatted object does not contain the information the conversion specifier refers to, an exception of type `format_error` is thrown.

-?- The result of formatting a `std::chrono::duration` instance holding a negative value, or of an `hh_mm_ss` object `h` for which `h.is_negative()` is true, is equivalent to the output of the corresponding positive value, with a `STATICALLY_WIDEN<charT>("-")` character sequence placed before the replacement of the leftmost conversion specifier.

[Example:

```

cout << format("{:%T}", -10'000s); // prints: -02:46:40
cout << format("{:%H:%M:%S}", -10'000s); // prints: -02:46:40
cout << format("{:minutes %M, hours %H, seconds %S}", -10'000s); // prints: minutes -46, hours 02, seconds 40

```

— end example]

-3- Unless explicitly requested, [...]

2. Modify 27.9.3 [\[time.hms.nonmembers\]](#) as indicated:

```

template<class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const hh_mm_ss<Duration>& hms);

```

-1- Effects: Equivalent to:

```

return os << format(os.getloc(),
                    hms.is_negative() ? STATICALLY_WIDEN<charT>("{:%T}")
                                     : STATICALLY_WIDEN<charT>("{:%T}"),
                    abs(hms.to_duration()));

```

[3269\(i\)](#). Parse manipulators do not specify the result of the extraction from stream

Section: 27.13 [\[time.parse\]](#) Status: [Immediate](#) Submitter: Tomasz Kamiński Opened: 2019-09-01 Last modified: 2020-02-14

Priority: 2

View other [active issues](#) in [time.parse].

View all other [issues](#) in [time.parse].

View all issues with [Immediate](#) status.

Discussion:

None of the parse manipulators for the chrono types specifies the result of the extraction from the stream, as consequence they cannot be chained with the other read operations (at least portably). For example the following code is not required to work:

```

std::chrono::sys_time s;
int x;
std::cin >> std::chrono::parse("%X", s) >> x;

```

[2019-10 Priority set to 2 after reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

[Drafting note: As a drive-by fix the *Remarks* element is also converted to a *Constraints* element. The wording integrates the resolution for LWG [3235](#).

1. Modify 27.13 [\[time.parse\]](#) as indicated:

-1- Each parse overload specified in this subclause calls `from_stream` unqualified, so as to enable argument dependent lookup (6.5.2 [\[basic.lookup.argdep\]](#)). In the following paragraphs, let `is` denote an object of type `basic_istream<charT, traits>` and let `I` be `basic_istream<charT, traits>&`, where `charT` and `traits` are template parameters in that context.

```
template<class charT, class traits, class Alloc, class Parsable>
unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp);
```

-2- ~~Remarks~~ **Constraints:** ~~This function shall not participate in overload resolution unless~~ The expression

`from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp)`

~~is a valid expression~~ well-formed when treated as an unevaluated operand.

-3- **Returns:** A manipulator such that, when extracted from a `basic_istream<charT, traits>` is, the expression `is >> parse(fmt, tp)` has type `I`, value is, and calls `from_stream(is, fmt.c_str(), tp)`.

```
template<class charT, class traits, class Alloc, class Parsable>
unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev);
```

-4- ~~Remarks~~ **Constraints:** ~~This function shall not participate in overload resolution unless~~ The expression

`from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp, addressof(abbrev))`

~~is a valid expression~~ well-formed when treated as an unevaluated operand.

-5- **Returns:** A manipulator such that, when extracted from a `basic_istream<charT, traits>` is, the expression `is >> parse(fmt, tp, abbrev)` has type `I`, value is, and calls `from_stream(is, fmt.c_str(), tp, addressof(abbrev))`.

```
template<class charT, class traits, class Alloc, class Parsable>
unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
          minutes& offset);
```

-6- ~~Remarks~~ **Constraints:** ~~This function shall not participate in overload resolution unless~~ The expression

`from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp,
 declval<basic_string<charT, traits, Alloc>*>().nullptr, &offset)`

~~is a valid expression~~ well-formed when treated as an unevaluated operand.

-7- **Returns:** A manipulator such that, when extracted from a `basic_istream<charT, traits>` is, the expression `is >> parse(fmt, tp, offset)` has type `I`, value is, and calls `from_stream(is, fmt.c_str(), tp, static_cast<basic_string<charT, traits, Alloc>*>(nullptr), &offset)`.

```
template<class charT, class traits, class Alloc, class Parsable>
unspecified
    parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
```

-8- ~~Remarks~~ **Constraints:** ~~This function shall not participate in overload resolution unless~~ The expression

`from_stream(declval<basic_istream<charT, traits>&>(),
 fmt.c_str(), tp, addressof(abbrev), &offset)`

~~is a valid expression~~ well-formed when treated as an unevaluated operand.

-9- **Returns:** A manipulator such that, when extracted from a `basic_istream<charT, traits>` is, the expression `is >> parse(fmt, tp, abbrev, offset)` has type `I`, value is, and calls `from_stream(is, fmt.c_str(), tp, addressof(abbrev), &offset)`.

[2020-02-13, Prague]

Issue wording has been rebased.

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 27.13 [\[time.parse\]](#) as indicated:

-1- Each parse overload specified in this subclause calls `from_stream` unqualified, so as to enable argument dependent lookup (6.5.2 [\[basic.lookup.argdep\]](#)). In the following paragraphs, let `is` denote an object of type `basic_istream<charT, traits>` [and let `I` be `basic\_istream<charT, traits>&`](#), where `charT` and `traits` are template parameters in that context.

```
template<class charT, class traits, class Alloc, class Parsable>
unspecified
parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp);
```

- 2- Constraints: [The expression](#)

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp)
```

[is a valid expression well-formed when treated as an unevaluated operand.](#)

- 3- Returns: A manipulator [such that, when extracted from a `basic\_istream<charT, traits>` is, the expression is `>> parse\(fmt, tp\)` has type `I`, value is, and](#) calls `from_stream(is, fmt.c_str(), tp)`.

```
template<class charT, class traits, class Alloc, class Parsable>
unspecified
parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
      basic_string<charT, traits, Alloc>& abbrev);
```

- 4- Constraints: [The expression](#)

```
from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp, addressof(abbrev))
```

[is a valid expression well-formed when treated as an unevaluated operand.](#)

- 5- Returns: A manipulator [such that, when extracted from a `basic\_istream<charT, traits>` is, the expression is `>> parse\(fmt, tp, abbrev\)` has type `I`, value is, and](#) calls `from_stream(is, fmt.c_str(), tp, addressof(abbrev))`.

```
template<class charT, class traits, class Alloc, class Parsable>
unspecified
parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
      minutes& offset);
```

- 6- Constraints: The expression

```
from_stream(declval<basic_istream<charT, traits>&>(),
            fmt.c_str(), tp,
            declval<basic_string<charT, traits, Alloc>*>(),
            &offset)
```

is well-formed when treated as an unevaluated operand.

- 7- Returns: A manipulator [such that, when extracted from a `basic\_istream<charT, traits>` is, the expression is `>> parse\(fmt, tp, offset\)` has type `I`, value is, and](#) calls

```
from_stream(is,
            fmt.c_str(), tp,
            static_cast<basic_string<charT, traits, Alloc>*>(nullptr),
            &offset)
```

```
template<class charT, class traits, class Alloc, class Parsable>
unspecified
parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
      basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
```

- 8- Constraints: [The expression](#)

```
from_stream(declval<basic_istream<charT, traits>&>(),
            fmt.c_str(), tp, addressof(abbrev), &offset)
```

[is a valid expression well-formed when treated as an unevaluated operand.](#)

- 9- Returns: A manipulator [such that, when extracted from a `basic\_istream<charT, traits>` is, the expression is `>> parse\(fmt, tp, abbrev, offset\)` has type `I`, value is, and](#) calls `from_stream(is, fmt.c_str(), tp, addressof(abbrev), &offset)`.

Priority: 2

View other active issues in [time.format].

View all other issues in [time.format].

View all issues with Immediate status.

Discussion:

%j represents the day number of the year when formatting and parsing time_points. It is also handy to interpret this flag consistently when formatting and parsing durations. That is if there is not enough information in the stream to represent a time_point, and if the target of the format/parse is a duration, %j represents a number of days.

```
#include <chrono>
#include <iostream>
#include <sstream>
#include <string>

int main()
{
    using namespace std;
    using namespace std::chrono;
    // Parse %j as number of days into a duration
    istringstream in{"222"};
    hours d;
    in >> parse("%j", d);
    cout << d << '\n';
    cout << format("{:%j}", d) << '\n';
}
```

Output:

```
5328h
222
```

[2019-10 Priority set to 2 after reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to N4830.

- 1. Modify "Table 98 — Meaning of conversion specifiers" [tab:time.format.spec] as indicated:

Table 98 — Meaning of conversion specifiers [tab:time.format.spec]	
Specifier	Replacement
	[...]
%j	The day of the year as a decimal number. Jan 1 is 001. If the result is less than three digits, it is left-padded with 0 to three digits. <u>If the type being formatted is a specialization of duration, it is formatted as a decimal number of days.</u>
	[...]

- 2. Modify "Table 99 — Meaning of parse flags" [tab:time.parse.spec] as indicated:

Table 99 — Meaning of parse flags [tab:time.parse.spec]	
Flag	Parsed value
	[...]
%j	The day of the year as a decimal number. Jan 1 is 1. The modified command %Nj specifies the maximum number of characters to read. If N is not specified, the default is 3. Leading zeroes are permitted but not required. <u>If the type being parsed is a specialization of duration, it is parsed as a decimal number of days.</u>
	[...]

[2020-02-13 After Thursday afternoon discussion in Prague, Marshall provides updated wording.]

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to N4830.

- 1. Modify "Table 98 — Meaning of conversion specifiers" [tab:time.format.spec] as indicated:

Table 98 — Meaning of conversion specifiers [tab:time.format.spec]

Specifier	Replacement
	[...]
%j	If the type being formatted is a specialization of duration, the decimal number of days without padding. Otherwise, the day of the year as a decimal number. Jan 1 is 001. If the result is less than three digits, it is left-padded with 0 to three digits.
	[...]

2. Modify "Table 99 — Meaning of parse flags" [tab:time.parse.spec] as indicated:

Table 99 — Meaning of parse flags [tab:time.parse.spec]

Flag	Parsed value
	[...]
%j	If the type being parsed is a specialization of duration, a decimal number of days. Otherwise, the day of the year as a decimal number. Jan 1 is 1. In either case, the modified command %Nj specifies the maximum number of characters to read. If N is not specified, the default is 3. Leading zeroes are permitted but not required.
	[...]

3282(i). subrange converting constructor should disallow derived to base conversions

Section: 24.5.3 [range.subrange] Status: [Immediate](#) Submitter: Eric Niebler Opened: 2019-09-10 Last modified: 2020-02-10

Priority: 1

View other [active issues](#) in [range.subrange].

View all other [issues](#) in [range.subrange].

View all issues with [Immediate](#) status.

Discussion:

The following code leads to slicing and general badness:

```
struct Base {};
struct Derived : Base {};
subrange<Derived*> sd;
subrange<Base*> sb = sd;
```

Traversal operations on iterators that are pointers do pointer arithmetic. If a Base* is actually pointing to a Derived*, then pointer arithmetic is invalid. subrange's constructors can easily flag this invalid code, and probably should.

The following PR incorporates the suggested fix to issue LWG [3281](#) I previously reported.

Suggested priority: P1, since it will be hard to fix this after C++20 ships.

[2019-10 Priority set to 1 and status to LEWG after reflector discussion]

[2019-10; Marshall comments]

This issue would resolve [US-285](#).

[2019-11 LEWG says OK; Status to Open. Friday PM discussion in Belfast. Casey to investigate and report back.]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

1. Modify 24.5.3 [range.subrange] as indicated:

```
namespace std::ranges {
    template<class From, class To>
    concept convertible-to-non-slicing = // exposition only
        convertible-to<From, To> &&
        !(is_pointer v<decay t<From>> &&
          is_pointer v<decay t<To>> &&
          not-same-as<remove_pointer t<decay t<From>>, remove_pointer t<decay t<To>>>);

    template<class T>
    concept pair-like = // exposition only
        [...];

    template<class T, class U, class V>
    concept pair-like-convertible-to = // exposition only
        !range<T> && pair-like<remove-reference_t<T>> &&
```

```

    requires(T&& t) {
        { get<0>(std::forward<T>(t)) } > convertible_to<U>;
        { get<1>(std::forward<T>(t)) } > convertible_to<V>;
    };

template<class T, class U, class V>
concept pair-like-convertible-from = // exposition only
    !range<T> && pair-like<T> &&
    constructible_from<T, U, V> &&
    convertible-to-non-slicing<U, tuple element t<0, T>> &&
    convertible_to<V, tuple element t<1, T>>;

[...]
```

```

[...]
```

```

template<input_or_output_iterator I, sentinel_for<I> S = I, subrange_kind K =
    sized_sentinel_for<S, I> ? subrange_kind::sized : subrange_kind::unsized>
    requires (K == subrange_kind::sized || !sized_sentinel_for<S, I>)
class subrange : public view_interface<subrange<I, S, K>> {
private:
    [...]
public:
    subrange() = default;

    constexpr subrange(convertible-to-non-slicing<I> auto i, S s) requires (!StoreSize);

    constexpr subrange(convertible-to-non-slicing<I> auto i, S s,
        make_unsigned_like_t(iter_difference_t<I>) n) requires (K == subrange_kind::sized);

    template<not-same-as<subrange> R>
        requires forwarding-range<R> &&
        convertible-to convertible-to-non-slicing<iterator_t<R>, I> &&
        convertible_to<sentinel_t<R>, S>
    constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);

    template<forwarding-range R>
        requires convertible_to<iterator_t<R>, I> && convertible_to<sentinel_t<R>, S>
    constexpr subrange(R&& r, make_unsigned_like_t(iter_difference_t<I>) n)
        requires (K == subrange_kind::sized)
        : subrange{ranges::begin(r), ranges::end(r), n}
    {}

    template<not-same-as<subrange> PairLike>
        requires pair-like-convertible-to<PairLike, I, S>
    constexpr subrange(PairLike&& r) requires (!StoreSize)
        : subrange{std::get<0>(std::forward<PairLike>(r)),
            std::get<1>(std::forward<PairLike>(r))}
    {}

    template<pair-like-convertible-to<I, S> PairLike>
    constexpr subrange(PairLike&& r, make_unsigned_like_t(iter_difference_t<I>) n)
        requires (K == subrange_kind::sized)
        : subrange{std::get<0>(std::forward<PairLike>(r)),
            std::get<1>(std::forward<PairLike>(r)), n}
    {}

    [...]
};

template<input_or_output_iterator I, sentinel_for<I> S>
subrange(I, S) -> subrange<I, S>;

[...]
```

2. Modify 24.5.3.1 [\[range.subrange.ctor\]](#) as indicated:

```

constexpr subrange(convertible-to-non-slicing<I> auto i, S s) requires (!StoreSize);

-1- Expects: [...]
```

```

[...]
```

```

constexpr subrange(convertible-to-non-slicing<I> auto i, S s,
    make_unsigned_like_t(iter_difference_t<I>) n) requires (K == subrange_kind::sized);

-2- Expects: [...]
```

```

[...]
```

```

template<not-same-as<subrange> R>
    requires forwarding-range<R> &&
    convertible-to convertible-to-non-slicing<iterator_t<R>, I> &&
    convertible_to<sentinel_t<R>, S>
constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);

-6- Effects: [...]
```

```

[...]
```

[2020-02-10; Prague]

The group identified minor problems that have been fixed in the revised wording.

[2020-02-10 Move to Immediate Monday afternoon in Prague]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.5.3 [\[range.subrange\]](#) as indicated:

```
namespace std::ranges {
    template<class From, class To>
        concept convertible-to-non-slicing = // exposition only
        convertible_to<From, To> &&
        !(is_pointer v<decay t<From>> &&
        is_pointer v<decay t<To>> &&
        not-same-as<remove_pointer t<decay t<From>>, remove_pointer t<decay t<To>>>);

    template<class T>
        concept pair-like = // exposition only
        [...]

    template<class T, class U, class V>
        concept pair-like-convertible-to = // exposition only
        !range<T> && pair-like<remove_reference_t<T>> &&
        requires(T&& t) {
            { get<0>(std::forward<T>(t)) } -> convertible_to<U>;
            { get<1>(std::forward<T>(t)) } -> convertible_to<V>;
        }

    template<class T, class U, class V>
        concept pair-like-convertible-from = // exposition only
        !range<T> && pair-like<T> &&
        constructible_from<T, U, V> &&
        convertible-to-non-slicing<U, tuple element t<0, T>> &&
        convertible_to<V, tuple element t<1, T>>;

    [...]
    [...]
    template<input_or_output_iterator I, sentinel_for<I> S = I, subrange_kind K =
        sized_sentinel_for<S, I> ? subrange_kind::sized : subrange_kind::unsized>
        requires (K == subrange_kind::sized || !sized_sentinel_for<S, I>)
        class subrange : public view_interface<subrange<I, S, K>> {
        private:
            [...]
        public:
            subrange() = default;

            constexpr subrange(convertible-to-non-slicing<I> auto i, S s) requires (!StoreSize);

            constexpr subrange(convertible-to-non-slicing<I> auto i, S s,
                               make_unsigned_like_t(iter_difference_t<I>) n) requires (K == subrange_kind::sized);

            template<not-same-as<subrange> R>
                requires forwarding-range<R> &&
                convertible-to-non-slicing<iterator_t<R>, I> &&
                convertible_to<sentinel_t<R>, S>
                constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);

            template<forwarding-range R>
                requires convertible-to-non-slicing<iterator_t<R>, I> &&
                convertible_to<sentinel_t<R>, S>
                constexpr subrange(R&& r, make_unsigned_like_t(iter_difference_t<I>) n)
                    requires (K == subrange_kind::sized)
                    : subrange{ranges::begin(r), ranges::end(r), n}
            {}

            template<not-same-as<subrange> PairLike>
                requires pair-like-convertible-to<PairLike, I, S>
                constexpr subrange(PairLike&& r) requires (!StoreSize)
                    : subrange{std::get<0>(std::forward<PairLike>(r)),
                               std::get<1>(std::forward<PairLike>(r))}
            {}

            template<pair-like-convertible-to<I, S> PairLike>
                constexpr subrange(PairLike&& r, make_unsigned_like_t(iter_difference_t<I>) n)
                    requires (K == subrange_kind::sized)
                    : subrange{std::get<0>(std::forward<PairLike>(r)),
                               std::get<1>(std::forward<PairLike>(r)), n}
            {}
        };

    [...]
};

template<input_or_output_iterator I, sentinel_for<I> S>
```

```
subrange(I, S) -> subrange<I, S>;
```

```
[...]  
}
```

2. Modify 24.5.3.1 [\[range.subrange.ctor\]](#) as indicated:

```
constexpr subrange(convertible-to-non-slicing<I> auto i, S s) requires (!StoreSize);
```

-1- *Expects*: [...]

[...]

```
constexpr subrange(convertible-to-non-slicing<I> auto i, S s,  
make_unsigned_like_t(iter_difference_t<I>) n) requires (K == subrange_kind::sized);
```

-2- *Expects*: [...]

[...]

```
template<not-same-as<subrange> R>  
requires forwarding_range<R> &&  
convertible-toconvertible-to-non-slicing<iterator_t<R>, I> &&  
convertible_to<sentinel_t<R>, S>  
constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);
```

-6- *Effects*: [...]

[...]

[3301](#)⁽ⁱ⁾. transform_view::iterator has incorrect iterator_category

Section: 24.7.5.3 [\[range.transform.iterator\]](#) **Status:** [Immediate](#) **Submitter:** Michel Morin **Opened:** 2019-10-03 **Last modified:** 2020-02-10

Priority: 1

View all issues with [Immediate](#) status.

Discussion:

When the transformation function returns an rvalue, transform_view::iterator cannot model *cpp17-forward-iterator*. However, similar to LWG [3291](#), the current wording on transform_view::iterator::iterator_category does not consider this.

As Casey Carter pointed out [here](#), the proposed wording below does not consider input_iterator that is not *cpp17-input-iterator* (this problem is not specific to the PR; it's pervasive in adapted iterators) and concepts-based determination would be a better fix for issues around iterator_category. But anyway, I consider this PR as a minimal fix at the moment.

[2019-10-31 Issue Prioritization]

Priority to 1 after reflector discussion.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

1. Modify 24.7.5.3 [\[range.transform.iterator\]](#) as indicated:

-2- iterator::iterator_category is defined as follows: Let C denote the type iterator_traits<iterator_t<Base>>::iterator_category.

(2.?)—If is_lvalue_reference v<iter_reference t<iterator t<Base>>> is true,

(2.2.?)— If C models derived_from<contiguous_iterator_tag>, then iterator_category denotes random_access_iterator_tag;

(2.2.?)— Otherwise, iterator_category denotes C.

(2.?)— Otherwise, iterator_category denotes input_iterator_tag.

[2019-11-06, Tim updates P/R based on Belfast LWG evening session discussion]

The check in the original P/R is incorrect; we want to check the transformation's result, not the base iterator.

[2020-02-10 Move to Immediate Monday afternoon in Prague]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 24.7.5.3 [\[range.transform.iterator\]](#) as indicated:

-2- [iterator::iterator_category](#) is defined as follows: Let `c` denote the type `iterator_traits<iterator_t<Base>>::iterator_category`.

[\(2.?\)](#) — If `is_lvalue_reference v<invoke result t<F&, range reference t<Base>>> is true`,

[\(2.?.?\)](#) — If `c` models `derived_from<contiguous_iterator_tag>`, then `iterator_category` denotes `random_access_iterator_tag`;

[\(2.?.?\)](#) — Otherwise, `iterator_category` denotes `c`.

[\(2.?\)](#) — Otherwise, `iterator_category` denotes `input_iterator_tag`.

[3314](#)⁽ⁱ⁾. Is stream insertion behavior locale dependent when `Period::type` is `micro`?

Section: 27.5.10 [\[time.duration.io\]](#) **Status:** [Immediate](#) **Submitter:** Tom Honermann **Opened:** 2019-11-04 **Last modified:** 2020-02-14

Priority: 2

View other [active issues](#) in [\[time.duration.io\]](#).

View all other [issues](#) in [\[time.duration.io\]](#).

View all issues with [Immediate](#) status.

Discussion:

27.5.10 [\[time.duration.io\]](#) states:

```
template<class charT, class traits, class Rep, class Period>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const duration<Rep, Period>& d);
```

[...]

-3- The units suffix depends on the type `Period::type` as follows:

[...]

[\(3.5\)](#) — Otherwise, if `Period::type` is `micro`, the suffix is `"μs"` (`"\u00b5\u0073"`).

[...]

[...]

-4- If `Period::type` is `micro`, but the character `U+00B5` cannot be represented in the encoding used for `charT`, the unit suffix `"us"` is used instead of `"μs"`.

[...]

Which encoding is intended by "the encoding used for `charT`"? There are two candidates:

1. The associated execution character set as defined by 5.3 [\[lex.charset\]](#) p3 used to encode character and string literals (e.g., the "execution wide-character set" for `wchar_t`).
2. The locale dependent character set used by the `std::locale` ctype and codecvt facets as specified in 28.4.1 [\[category.ctype\]](#), sometimes referred to as the "native character set".

The behavior should not be dependent on `locale` and should therefore be specified in terms of the execution character sets.

The execution character set is implementation defined and some implementations allow the choice of execution character set to be specified via a compiler option or determined based on the locale active when the compiler is run. For example, the Microsoft compiler, when run on a Windows system with regional language settings configured for "English (United States)", will use Windows-1252 for the execution character set, but allows this choice to be overridden with the `/execution-charset` compiler option. The Microsoft compiler might therefore use `"us"` by default, but `"μs"` when invoked with the `/execution-charset:utf-8` or `/execution-charset:.437` options. In the latter two cases, the string contents would contain `"\xb5\x73"` and `"\xe6\x73"` respectively (Unicode and Windows code page 437 map `μ` (U+00B5, MICRO SIGN) to different code points).

This resolution relies on the character set for the locale used at run-time being compatible with the execution character set if the produced string is to be displayed correctly when written to a terminal or console. This is a typical requirement for character and string literals but is more strongly relevant for this issue since `μ` lacks representation in many character sets. Additionally, if the stream is imbued with a `std::codecvt` facet, the facet must provide appropriate conversion support for behavior to be well defined.

[2019-11 Priority to 2 during Tuesday morning issue processing in Belfast.]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4835](#).

1. Modify 27.5.10 [\[time.duration.io\]](#) as indicated:

[Drafting note: "implementation's native character set" is used in 28.4.1.1 [\[locale ctype\]](#) and 28.4.1.4 [\[locale codecvt\]](#) to refer to the locale dependent character encoding.]

```
template<class charT, class traits, class Rep, class Period>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const duration<Rep, Period>& d);
```

[...]

- 3- The units suffix depends on the type `Period::type` as follows:

[...]

(3.5) — Otherwise, if `Period::type` is `micro`, the suffix is `"μs"` (`"\u00b5\u0073"`).

[...]

[...]

- 4- If `Period::type` is `micro`, but the character `U+00B5` cannot be represented in the encoding used, lacks representation in the execution character set for `charT`, the unit suffix `"us"` is used instead of `"μs"`. If `"μs"` is used but the implementation's native character set lacks representation for `U+00B5` and the stream is associated with a terminal or console, or if the stream is imbued with a `std::codecvt` facet that lacks conversion support for the character, then the result is unspecified.

[...]

[2019-11-12; Tom Honermann improves wording]

[2020-02 Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 27.5.10 [\[time.duration.io\]](#) as indicated:

```
template<class charT, class traits, class Rep, class Period>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const duration<Rep, Period>& d);
```

[...]

- 3- The units suffix depends on the type `Period::type` as follows:

[...]

(3.5) — Otherwise, if `Period::type` is `micro`, it is implementation-defined whether the suffix is `"μs"` (`"\u00b5\u0073"`) or `"us"`.

[...]

[...]

- 4- If `Period::type` is `micro`, but the character `U+00B5` cannot be represented in the encoding used for `charT`, the unit suffix `"us"` is used instead of `"μs"`.

[...]

[3328](#)⁽ⁱ⁾. Clarify that `std::string` is not good for UTF-8

Section: D.19 [\[depr.fs.path.factory\]](#) Status: [Immediate](#) Submitter: The Netherlands Opened: 2019-11-07 Last modified: 2020-02-12

Priority: 0

View all issues with [Immediate](#) status.

Discussion:

Addresses [NL 375](#)

Example in deprecated section implies that `std::string` is the type to use for utf8 strings.

[Example: A string is to be read from a database that is encoded in UTF-8, and used to create a directory using the native encoding for filenames:

```
namespace fs = std::filesystem;
std::string utf8_string = read_utf8_data();
fs::create_directory(fs::u8path(utf8_string));
```

Proposed change:

Add clarification that `std::string` is the wrong type for utf8 strings

Jeff Garland:

SG16 in Belfast: Recommend to accept with a modification to update the example in D.19 [\[depr.fs.path.factory\]](#) p4 to state that `std::u8string` should be preferred for UTF-8 data.

Rationale: The example code is representative of historic use of `std::filesystem::u8path` and should not be changed to use `std::u8string`. The recommended change is to a non-normative example and may therefore be considered editorial.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4835](#).

1. Modify D.19 [\[depr.fs.path.factory\]](#) as indicated:

-4- [Example: A string is to be read from a database that is encoded in UTF-8, and used to create a directory using the native encoding for filenames:

```
namespace fs = std::filesystem;
std::string utf8_string = read_utf8_data();
fs::create_directory(fs::u8path(utf8_string));
```

For POSIX-based operating systems with the native narrow encoding set to UTF-8, no encoding or type conversion occurs.

For POSIX-based operating systems with the native narrow encoding not set to UTF-8, a conversion to UTF-32 occurs, followed by a conversion to the current native narrow encoding. Some Unicode characters may have no native character set representation.

For Windows-based operating systems a conversion from UTF-8 to UTF-16 occurs. — *end example*]

[Note: The example above is representative of historic use of `filesystem::u8path`. New code should use `std::u8string` in place of `std::string`. — *end note*]

LWG Belfast Friday Morning

Requested changes:

- Historic => historical.
- Add missing `::` before `u8path`.
- Remove ISO rules forbidden 'should' in a note.
- Use language describing why new code should use the `u8string` constructor rather than preaching that new code should do something.

Billy O'Neal provides updated wording.

[2020-02 Moved to Immediate on Tuesday in Prague.]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify D.19 [\[depr.fs.path.factory\]](#) as indicated:

-4- [Example: A string is to be read from a database that is encoded in UTF-8, and used to create a directory using the native encoding for filenames:

```
namespace fs = std::filesystem;
std::string utf8_string = read_utf8_data();
fs::create_directory(fs::u8path(utf8_string));
```

For POSIX-based operating systems with the native narrow encoding set to UTF-8, no encoding or type conversion occurs.

For POSIX-based operating systems with the native narrow encoding not set to UTF-8, a conversion to UTF-32 occurs, followed by a conversion to the current native narrow encoding. Some Unicode characters may have no native character set representation.

For Windows-based operating systems a conversion from UTF-8 to UTF-16 occurs. — *end example*

[Note: The example above is representative of a historical use of `filesystem::u8path`. Passing a `std::u8string` to `path`'s constructor is preferred for an indication of UTF-8 encoding more consistent with `path`'s handling of other encodings. — end note]

3334⁽ⁱ⁾. `basic_osyncstream` move assignment and destruction calls `basic_syncbuf::emit()` twice

Section: 29.10.3 [[syncstream.osyncstream](#)] Status: [Immediate](#) Submitter: Tim Song Opened: 2019-11-06 Last modified: 2020-02-14

Priority: 3

View all issues with [Immediate](#) status.

Discussion:

These functions are specified to call `emit()`, which calls `emit()` on the `basic_syncbuf` and sets `badbit` if it fails. Then, the move assignment is specified to move-assign the `basic_syncbuf`, while the destructor implicitly needs to destroy the `basic_syncbuf` data member. This calls `emit()` on the `basic_syncbuf` again.

Is this intended?

[2020-02-13 Tim adds wording after discussion with Peter]

[2020-02 Status to Immediate Thursday afternoon in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

[Drafting note: There is no need to explicitly call `emit` at all in these functions; memberwise move-assignment/destruction is sufficient, so we can strike the specification entirely and rely on the wording in 16.4.2.3 [\[functions.within.classes\]](#). — end drafting note]

1. Edit 29.10.3.2 [[syncstream.osyncstream.cons](#)] as indicated:

~~`basic_osyncstream();`~~

~~`6 Effects: Calls emit(). If an exception is thrown from emit(), that exception is caught and ignored.`~~

2. Strike 29.10.3.3 [[syncstream.osyncstream.assign](#)]:

~~`basic_osyncstream& operator=(basic_osyncstream&& rhs) noexcept;`~~

~~`1 Effects: First, calls emit(). If an exception is thrown from emit(), that exception is caught and ignored. Move assigns sb from rhs.sb. [ Note: This disassociates rhs from its wrapped stream buffer ensuring destruction of rhs produces no output. — end note ]`~~

~~`2 Postconditions: nullptr == rhs.get_wrapped() is true. get_wrapped() returns the value previously returned by rhs.get_wrapped().`~~

3335⁽ⁱ⁾. Resolve C++20 NB comments US 273 and GB 274

Section: 24.2 [[ranges.syn](#)] Status: [Immediate](#) Submitter: United States/Great Britain Opened: 2019-11-08 Last modified: 2020-02-10

Priority: 1

View other [active issues](#) in [[ranges.syn](#)].

View all other [issues](#) in [[ranges.syn](#)].

View all issues with [Immediate](#) status.

Discussion:

Addresses [US 273/GB 274](#)

US 273:

`all_view` is not a view like the others. For the other view types, `foo_view{args...}` is a valid way to construct an instance of type `foo_view`. However, `all_view` is just an alias to the type of `view::all(arg)`, which could be one of several different types. `all_view` feels like the wrong name.

Proposed change:

Suggest renaming `all_view` to `all_t` and moving it into the `views::` namespace.

[GB 274](#):

Add `range_size_t`.

LEWG asked that `range_size_t` be removed from [P1035](#), as they were doing a good job of being neutral w.r.t whether or not size-types were signed or unsigned at the time. Now that we've got a policy on what size-types are, and that [P1522](#) and [P1523](#) have been adopted, it makes sense for there to be a `range_size_t`.

Proposed change:

Add to `[ranges.syn]`:

```
template<range R>
using range_difference_t = iter_difference_t<iterator_t<R>>;
template<size range R>
using range_size_t = decltype(ranges::size(declval<R>().));
```

David Olsen:

The proposed wording has been approved by LEWG and LWG in Belfast.

[2019-11-23 Issue Prioritization]

Priority to 1 after reflector discussion.

[2020-02-10 Move to Immediate Monday afternoon in Prague]

Proposed resolution:

This wording is relative to [N4835](#).

1. Change 24.2 [\[ranges.syn\]](#), header `<ranges>` synopsis, as indicated:

```
#include <initializer_list>
#include <iterator>

namespace std::ranges {
    [...]
    // 24.4.2 \[range.range\], ranges
    template<class T>
    concept range = see below;
    [...]
    template<range R>
        using range_difference_t = iter_difference_t<iterator_t<R>>;
    template<size range R>
        using range_size_t = decltype(ranges::size(declval<R>().));
    template<range R>
        using range_value_t = iter_value_t<iterator_t<R>>;
    [...]
    // 24.7.3.1 \[range.ref.view\], all view
    namespace views { inline constexpr unspecified all = unspecified; }
    inline constexpr unspecified all = unspecified;

    template<viewable_range R>
        using all_view = decltype(views::all(declval<R>().));
    [...]
}
```

2. Globally replace all occurrences of `all_view` with `views::all_t`. There are 36 occurrences in addition to the definition in the `<ranges>` synopsis that was changed above.

[3340^{\(i\)}](#). Formatting functions should throw on argument/format string mismatch in `{format.functions}`

Section: 20.20.3 [\[format.functions\]](#) **Status:** [Immediate](#) **Submitter:** Great Britain **Opened:** 2019-11-17 **Last modified:** 2020-02-13

Priority: Not Prioritized

View other [active issues](#) in `[format.functions]`.

View all other [issues](#) in `[format.functions]`.

View all issues with [Immediate](#) status.

Discussion:

Formatting functions don't allow throwing on incorrect arguments. `std::format` is only allowed to throw if `fmt` is not a format string, but the intention is it also throws for errors during formatting, e.g. there are fewer arguments than required by the format string.

Proposed change:

Allow exceptions even when the format string is valid. Possibly state the *Effects*: more precisely.

Victor Zverovich:

LEWG approved resolution of this NB comment as an LWG issue.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4835](#).

[*Drafting Note*: Depending on whether LWG [3336](#)'s wording has been accepted when this issue's wording has been accepted, two mutually exclusive options are prepared, depicted below by **Option A** and **Option B**, respectively.]

Option A (LWG [3336](#) has been accepted)

1. Change 20.20.2.1 [\[format.string.general\]](#) as follows:

-1- A *format string* [for arguments args](#) is a (possibly empty) sequence of *replacement fields*, *escape sequences*, and characters other than `{` and `}`. [...]

-2- The *arg-id* field specifies the index of the argument in `args` whose value is to be formatted and inserted into the output instead of the replacement field. [If there is no argument with the index *arg-id* in `args`, the string is not a format string](#). The optional *format-specifier* field explicitly specifies a format for the replacement value.

[...]

-5- The *format-spec* field contains *format specifications* that define how the value should be presented. Each type can define its own interpretation of the *format-spec* field. [If *format-spec* doesn't conform to the format specifications for the argument in `args` referred to by *arg-id*, the string is not a format string](#). [...]

2. Before 20.20.3 [\[format.functions\]](#) insert a new sub-clause as indicated:

[20.20.? Error reporting](#)[\[format.err.report\]](#)

[?- Formatting functions throw exceptions to report formatting and other errors. They throw format error if an argument `fmt` is passed that is not a format string for arguments `args` and propagate exceptions thrown by formatter specializations and iterator operations. Failure to allocate storage is reported by throwing an exception as described in 16.5.5.13 \[\\[res.on.exception.handling\\]\]\(#\).](#)

3. Modify 20.20.3 [\[format.functions\]](#) as indicated:

```
string vformat(string_view fmt, format_args args);
wstring vformat(wstring_view fmt, wformat_args args);
string vformat(const locale& loc, string_view fmt, format_args args);
wstring vformat(const locale& loc, wstring_view fmt, wformat_args args);
```

-6- [...]

-7- Throws: ~~format_error if `fmt` is not a format string~~ [As specified in \[\\[format.err.report\\]\]\(#\)](#).

[...]

```
template<class Out>
    Out vformat_to(Out out, string_view fmt, format_args_t<Out, char> args);
template<class Out>
    Out vformat_to(Out out, wstring_view fmt, format_args_t<Out, wchar_t> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, string_view fmt,
        format_args_t<Out, char> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, wstring_view fmt,
        format_args_t<Out, wchar_t> args);
```

[...]

-15- Throws: ~~format_error if `fmt` is not a format string~~ [As specified in \[\\[format.err.report\\]\]\(#\)](#).

[...]

```

template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                         string_view fmt, const Args&... args);
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                         wstring_view fmt, const Args&... args);
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                         const locale& loc, string_view fmt,
                                         const Args&... args);
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                         const locale& loc, wstring_view fmt,
                                         const Args&... args);

```

[...]

-21- Throws: `format_error` if `fmt` is not a format string As specified in [format.err.report].

[...]

```

template<class... Args>
    size_t formatted_size(string_view fmt, const Args&... args);
template<class... Args>
    size_t formatted_size(wstring_view fmt, const Args&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, string_view fmt, const Args&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, wstring_view fmt, const Args&... args);

```

[...]

-25- Throws: `format_error` if `fmt` is not a format string As specified in [format.err.report].

Option B (LWG 3336 has not been accepted)

1. Change 20.20.2.1 [format.string.general] as follows:

-1- A *format string* `for arguments args` is a (possibly empty) sequence of *replacement fields*, *escape sequences*, and characters other than `{` and `}`. [...]

-2- The *arg-id* field specifies the index of the argument in `args` whose value is to be formatted and inserted into the output instead of the replacement field. If there is no argument with the index *arg-id* in `args`, the string is not a format string. The optional *format-specifier* field explicitly specifies a format for the replacement value.

[...]

-5- The *format-spec* field contains *format specifications* that define how the value should be presented. Each type can define its own interpretation of the *format-spec* field. If *format-spec* doesn't conform to the format specifications for the argument in `args` referred to by *arg-id*, the string is not a format string. [...]

2. Modify 20.20.3 [format.functions] as indicated:

```

string vformat(string_view fmt, format_args args);
wstring vformat(wstring_view fmt, wformat_args args);
string vformat(const locale& loc, string_view fmt, format_args args);
wstring vformat(const locale& loc, wstring_view fmt, wformat_args args);

```

-6- [...]

-7- Throws: `format_error` if `fmt` is not a format string `for args`.

[...]

```

template<class Out>
    Out vformat_to(Out out, string_view fmt, format_args_t<Out, char> args);
template<class Out>
    Out vformat_to(Out out, wstring_view fmt, format_args_t<Out, wchar_t> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, string_view fmt,
                  format_args_t<Out, char> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, wstring_view fmt,
                  format_args_t<Out, wchar_t> args);

```

[...]

-15- Throws: `format_error` if `fmt` is not a format string `for args`.

[...]

```

template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,

```

```

        string_view fmt, const Args&... args);
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
        wstring_view fmt, const Args&... args);
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
        const locale& loc, string_view fmt,
        const Args&... args);
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
        const locale& loc, wstring_view fmt,
        const Args&... args);

[...]

-21- Throws: format_error if fmt is not a format string for args.

[...]

template<class... Args>
    size_t formatted_size(string_view fmt, const Args&... args);
template<class... Args>
    size_t formatted_size(wstring_view fmt, const Args&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, string_view fmt, const Args&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, wstring_view fmt, const Args&... args);

[...]

-25- Throws: format_error if fmt is not a format string for args.

```

[2020-02-12, Prague; LWG discussion]

Option A is the only one we look at to resolve LWG [3336](#) as well. During the discussions some wording refinements have been suggested that are integrated below.

Proposed resolution:

This wording is relative to [N4849](#).

1. Change 20.20.2.1 [\[format.string.general\]](#) as follows:

-1- A *format string* **for arguments args** is a (possibly empty) sequence of *replacement fields*, *escape sequences*, and characters other than `{` and `}`. [...]

-2- The *arg-id* field specifies the index of the argument in `args` whose value is to be formatted and inserted into the output instead of the replacement field. **If there is no argument with the index *arg-id* in `args`, the string is not a format string for args.** The optional *format-specifier* field explicitly specifies a format for the replacement value.

[...]

-5- The *format-spec* field contains *format specifications* that define how the value should be presented. Each type can define its own interpretation of the *format-spec* field. **If *format-spec* does not conform to the format specifications for the argument type referred to by *arg-id*, the string is not a format string for args.** [...]

2. Before 20.20.3 [\[format.functions\]](#) insert a new sub-clause as indicated:

20.20.2 Error reporting [\[format.err.report\]](#)

-2- Formatting functions throw `format_error` if an argument `fmt` is passed that is not a format string for args. They propagate exceptions thrown by operations of formatter specializations and iterators. Failure to allocate storage is reported by throwing an exception as described in 16.5.5.13 [\[res.on.exception.handling\]](#).

3. Modify 20.20.3 [\[format.functions\]](#) as indicated:

```

string vformat(string_view fmt, format_args args);
wstring vformat(wstring_view fmt, wformat_args args);
string vformat(const locale& loc, string_view fmt, format_args args);
wstring vformat(const locale& loc, wstring_view fmt, wformat_args args);

```

-6- [...]

-7- Throws: `format_error` if `fmt` is not a format string **As specified in [\[format.err.report\]](#).**

[...]

```

template<class Out>
    Out vformat_to(Out out, string_view fmt, format_args_t<Out, char> args);
template<class Out>

```



```

    Out vformat_to(Out out, wstring_view fmt, format_args_t<Out, wchar_t> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, string_view fmt,
        format_args_t<Out, char> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, wstring_view fmt,
        format_args_t<Out, wchar_t> args);

[...]

```

-15- Throws: `format_error` if `fmt` is not a format string As specified in [format.err.report].

[...]

```

template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
        string_view fmt, const Args&... args);
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
        wstring_view fmt, const Args&... args);
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
        const locale& loc, string_view fmt,
        const Args&... args);
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
        const locale& loc, wstring_view fmt,
        const Args&... args);

```

[...]

-21- Throws: `format_error` if `fmt` is not a format string As specified in [format.err.report].

[...]

```

template<class... Args>
    size_t formatted_size(string_view fmt, const Args&... args);
template<class... Args>
    size_t formatted_size(wstring_view fmt, const Args&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, string_view fmt, const Args&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, wstring_view fmt, const Args&... args);

```

[...]

-25- Throws: `format_error` if `fmt` is not a format string As specified in [format.err.report].

[3347^{\(i\)}](#). `std::pair<T, U>` now requires `T` and `U` to be less-than-comparable

Section: 20.4.3 [pairs.spec], 22.3.7.1 [array.overview] **Status:** [Immediate](#) **Submitter:** Jonathan Wakely **Opened:** 2019-12-03 **Last modified:** 2020-02-10

Priority: 1

View other [active issues](#) in [pairs.spec].

View all other [issues](#) in [pairs.spec].

View all issues with [Immediate](#) status.

Discussion:

[P1614R2](#) added `operator<=>` as a hidden friend to `std::pair`:

```

friend constexpr common_comparison_category_t<synth-three-way-result<T1>,
        synth-three-way-result<T2>>
    operator<=>(const pair& x, const pair& y) { see below }

```

That is not a function template, so is not a SFINAE context. If one or both of `synth-three-way-result<T1>` or `synth-three-way-result<T2>` is an invalid type then the declaration of `operator<=>` is ill-formed, and so the specialization `std::pair<T1, T2>` is ill-formed.

A similar problem exists for `std::array`.

There are at least two ways to fix this:

1. Constrain the function and delay the use of `synth-three-way-result` until we know it's valid.
2. Replace the hidden friend with a namespace-scope function template, so invalid `synth-three-way-result` types cause substitution failure.

The first option is somewhat hard to specify, because current policy is to avoid the use of requires-clauses in most of the library clauses. Even with a requires-clause, the potentially-invalid *synth-three-way-result* types cannot be used in the function declarator. Furthermore, the operator<==> for std::array is currently specified in Table [tab:container.opt] and so there's nowhere to add a *Constraints:* element.

The second option would partially revert the [P1614R2](#) changes for std::pair and std::array and bring them closer to what was in C++17. The main motivation for making operator== a hidden friend was to allow it to be defaulted, so that std::pair and std::array would be usable as non-type template parameters. Following the acceptance of [P1907](#) in Belfast it isn't necessary to default it, so we can go back to what was in C++17.

[2019-12-12 Issue Prioritization]

Priority to 1 after reflector discussion.

[2020-02-10 Move to Immediate Monday afternoon in Prague]

Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 20.2.1 [\[utility.syn\]](#) as indicated:

[Drafting note: This restores the pre-[P1614R2](#) operator== and uses operator<==> as replacement for operator<, operator<=, operator>, operator>=.]

[...]

```
// 20.4 \[pairs\], class template pair
template<class T1, class T2>
struct pair;

// 20.4.3 \[pairs.spec\], pair specialized algorithms
template<class T1, class T2>
constexpr bool operator==(const pair<T1, T2>&, const pair<T1, T2>&);
template<class T1, class T2>
constexpr common_comparison_category_t<synth-three-way-result<T1>,
synth-three-way-result<T2>>
operator<==>(const pair<T1, T2>&, const pair<T1, T2>&);

template<class T1, class T2>
constexpr void swap(pair<T1, T2>& x, pair<T1, T2>& y) noexcept(noexcept(x.swap(y)));
[...]
```

2. Modify 20.4.2 [\[pairs.pair\]](#) as indicated:

```
[...]
constexpr void swap(pair& p) noexcept(see below);

// 20.4.3 \[pairs.spec\], pair specialized algorithms
friend constexpr bool operator==(const pair&, const pair&) = default;
friend constexpr bool operator==(const pair& x, const pair& y)
requires (is_reference_v<T1> || is_reference_v<T2>)
[ return x.first == y.first && x.second == y.second; ]
friend constexpr common_comparison_category_t<synth three way result<T1>,
synth three way result<T2>>
operator<==>(const pair& x, const pair& y) [ see below ]
};

template<class T1, class T2>
pair(T1, T2) -> pair<T1, T2>;
[...]
```

3. Modify 20.4.3 [\[pairs.spec\]](#) as indicated:

20.4.3 Specialized algorithms [\[pairs.spec\]](#)

```
template<class T1, class T2>
constexpr bool operator==(const pair<T1, T2>& x, const pair<T1, T2>& y);

-?- Returns: x.first == y.first && x.second == y.second.

template<class T1, class T2>
friend constexpr
common_comparison_category_t<synth-three-way-result<T1>, synth-three-way-result<T2>>
operator<==>(const pair<T1, T2>& x, const pair<T1, T2>& y);

-1- Effects: Equivalent to:

if (auto c = synth-three-way(x.first, y.first); c != 0) return c;
return synth-three-way(x.second, y.second);
```

4. Modify 22.3.2 [\[array.syn\]](#) as indicated:

[Drafting note: This restores the pre-P1614R2 operator== and uses operator<=> as replacement for operator<, operator<=, operator>, operator>=.]

```
namespace std {  
  
// 22.3.7 [array], class template array  
template<class T, size_t N> struct array;  
  
template<class T, size_t N>  
constexpr bool operator==(const array<T, N>& x, const array<T, N>& y);  
template<class T, size_t N>  
constexpr synth-three-way-result<T>  
operator<=>(const array<T, N>& x, const array<T, N>& y);  
  
template<class T, size_t N>  
constexpr void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));  
[...]
```

5. Modify 22.3.7.1 [array.overview] as indicated:

[Drafting note: there is no need to add definitions of operator== and operator<=> to [array.spec] because they are defined by Table 71: Container requirements [tab:container.req] and Table 73: Optional container operations [tab:container.opt] respectively.]

```
[...]  
constexpr T * data() noexcept;  
constexpr const T * data() const noexcept;  
  
friend constexpr bool operator==(const array&, const array&) = default;  
friend constexpr synth-three-way-result<value_type>  
operator<=>(const array&, const array&);  
};  
  
template<class T, class... U>  
array(T, U...) -> array<T, 1 + sizeof...(U)>;  
[...]
```

3348⁽ⁱ⁾. __cpp_lib_unwrap_ref in wrong header

Section: 17.3.2 [version.syn], 20.14.5.6 [refwrap.unwrapref] **Status:** Immediate **Submitter:** Barry Revzin **Opened:** 2019-12-03 **Last modified:** 2020-02-14

Priority: 2

View other [active issues](#) in [version.syn].

View all other [issues](#) in [version.syn].

View all issues with [Immediate](#) status.

Discussion:

cpplearner points out in [this github comment](#) that:

Since unwrap_reference and unwrap_ref_decay are defined in <functional> ([functional.syn]), their feature test macro should also be defined there.

P1902R1 adds this feature test macro in <type_traits> instead. The feature test macro and the type traits should go into the same header: either both in <functional> or both in <type_traits>.

The smallest diff is just to move the macro into <functional>.

[2019-12-12 Issue Prioritization]

Priority to 2 after reflector discussion.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4842](#).

1. Modify 17.3.2 [version.syn] p2 as indicated:

```
[...]  
#define __cpp_lib_unordered_map_try_emplace 201411L // also in <unordered_map>  
#define __cpp_lib_unwrap_ref 201811L // also in <type_traits>functional>
```

```
#define __cpp_lib_variant 201606L // also in <variant>
[...]
```

[2020-02-13, Prague]

During LWG discussions it had been suggested that they considered it is an improvement to move the definitions of `unwrap_reference` and `unwrap_ref_decay` from `<functional>` to `<type_traits>`. This is what the alternative wording tries to accomplish.

[Status to Immediate on Thursday night in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 20.14.1 [\[functional.syn\]](#), header `<functional>` synopsis, as indicated:

```
namespace std {
[...]
```

~~```
template<class T> struct unwrap_reference;
template<class T> using unwrap_reference_t = typename unwrap_reference<T>::type;
template<class T> struct unwrap_ref_decay;
template<class T> using unwrap_ref_decay_t = typename unwrap_ref_decay<T>::type;
[...]
```~~

```
}
```

2. Delete sub-clause 20.14.5.6 [\[refwrap.unwrapref\]](#) completely, as indicated:

~~**20.14.5.6 Transformation type trait `unwrap_reference` [\[refwrap.unwrapref\]](#)**

```
template<class T>
struct unwrap_reference;

1 If T is a specialization reference_wrapper<X> for some type X, the member typedef type of unwrap_reference<T> is X&, otherwise it is T.

template<class T>
struct unwrap_ref_decay;

2 The member typedef type of unwrap_ref_decay<T> denotes the type unwrap_reference_t<decay_t<T>>.
```~~

3. Modify 20.15.2 [\[meta.type.synop\]](#), header `<type_traits>` synopsis, as indicated:

```
namespace std {
[...]
```

~~```
// 20.15.7.6 \[meta.trans.other\], other transformations
[...]
```~~

```
template<class T> struct underlying_type;
template<class Fn, class... ArgTypes> struct invoke_result;
template<class T> struct unwrap_reference;
template<class T> struct unwrap_ref_decay;

template<class T>
using type_identity_t = typename type_identity<T>::type;
[...]
```

~~```
template<class Fn, class... ArgTypes>
using invoke_result_t = typename invoke_result<Fn, ArgTypes...>::type;
template<class T>
using unwrap_reference_t = typename unwrap_reference<T>::type;
template<class T>
using unwrap_ref_decay_t = typename unwrap_ref_decay<T>::type;
template<class...>
using void_t = void;
[...]
```~~

```
}
```

4. Modify 20.15.7.6 [\[meta.trans.other\]](#), Table 55 — "Sign modifications" in `[tab:meta.trans.sign]` as indicated:

Table 52 — Other transformations `[tab:meta.trans.other]`

| Template                                                             | Comments                                                                                                                                                                                                  |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [...]                                                                |                                                                                                                                                                                                           |
| <del>template &lt;class T&gt;<br/>struct<br/>unwrap_reference;</del> | <del>If T is a specialization <code>reference_wrapper&lt;X&gt;</code> for some type X, the member typedef <code>type</code> of <code>unwrap_reference&lt;T&gt;</code> is X&amp;, otherwise it is T.</del> |
| <del>template &lt;class T&gt;<br/>struct<br/>unwrap_ref_decay;</del> | <del>The member typedef <code>type</code> of <code>unwrap_ref_decay&lt;T&gt;</code> denotes the type <code>unwrap_reference_t&lt;decay_t&lt;T&gt;&gt;</code>.</del>                                       |

5. Insert between 20.15.7.6 [\[meta.trans.other\]](#) p1 and p2 as indicated:

In addition to being available via inclusion of the `<type_traits>` header, the templates `unwrap_reference`, `unwrap_ref_decay`, `unwrap_reference_t`, and `unwrap_ref_decay_t` are available when the header `<functional>` (20.14.1 [functional.syn]) is included.

### 3352<sup>(i)</sup>. `strong_equality` isn't a thing

**Section:** 22.2.1 [container.requirements.general] **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2019-12-06 **Last modified:** 2020-02-12

**Priority:** 1

**View other [active issues](#)** in [container.requirements.general].

**View all other [issues](#)** in [container.requirements.general].

**View all issues with [Immediate](#) status.**

#### Discussion:

[tab:container.req] includes the row:

**Expression:** `i <=> j`

**Return Type:** `strong_ordering` if `X::iterator` meets the random access iterator requirements, otherwise `strong_equality`.

**Complexity:** constant

"`strong_equality`" is (now) a meaningless term that appears nowhere else in the working draft. Presumably we want to make the Return Type unconditionally `strong_ordering`, and require this expression to be valid only when `i` and `j` are random access iterators. It's not clear to me if the best way to do so would be to add a "Constraints" to the "Assertion/note/pre-/post-condition" column of this table row, or if we should pull the row out into yet another "conditionally-supported iterator requirements" table.

[2019-12-21 Issue Prioritization]

Priority to 1 after reflector discussion.

[2020-02-10, Prague; David Olsen provides new wording based upon Tim Songs suggestion]

[2020-02 Moved to Immediate on Tuesday in Prague.]

#### Proposed resolution:

Table 71 — Container requirements [tab:container.req]

| Expression                 | Return type                                                                                                                                    | Operational semantics | Assertion/note pre/post-condition                                                           | Complexity |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|---------------------------------------------------------------------------------------------|------------|
| [...]                      |                                                                                                                                                |                       |                                                                                             |            |
| <code>i &lt;=&gt; j</code> | <code>strong_ordering</code> if <code>X::iterator</code> meets the random access iterator requirements, otherwise <code>strong_equality</code> |                       | <b>Constraints:</b> <code>X::iterator</code> meets the random access iterator requirements. | constant   |
| [...]                      |                                                                                                                                                |                       |                                                                                             |            |

### 3354<sup>(i)</sup>. `has_strong_structural_equality` has a meaningless definition

**Section:** 20.15.4.3 [meta.unary.prop], 17.3.2 [version.syn] **Status:** [Immediate](#) **Submitter:** Daniel Krüger **Opened:** 2019-12-08 **Last modified:** 2020-02-13

**Priority:** 1

**View other [active issues](#)** in [meta.unary.prop].

**View all other [issues](#)** in [meta.unary.prop].

**View all issues with [Immediate](#) status.**

#### Discussion:

After the Belfast 2019 meeting with the acceptance of [P1907R1](#) the core language term "strong structural equality" has been removed and instead a more general definition of a "structural type" has been introduced that is suitable to be used as non-type template parameter. These changes have caused the current definition of the `has_strong_structural_equality` type trait (which had originally been introduced by [P1614R2](#) during the Cologne 2019 meeting) to become meaningless since it is currently defined as follows:

The type `T` has strong structural equality (11.11.1 [class.compare.default]).

Besides the now undefined term "strong structural equality", the reference to 11.11.1 [\[class.compare.default\]](#) doesn't make sense anymore, assuming that the trait definition is supposed to refer now to a type that can be used as non-type template parameter.

During library reflector discussions several informal naming suggestions has been mentioned, such as `is_structural[_type]`, `can_be_nttp`, `is_nontype_template_parameter_type`.

Albeit `is_structural_type` would come very near to the current core terminology, core experts have argued that the term "structural type" should be considered as a "core-internal" term that may easily be replaced post-C++20.

In addition to that definition and naming question of that type trait it should be discussed whether there should exist a specific feature macro for just this type trait, similar to the reason why we introduced the `__cpp_lib_has_unique_object_representations` test macro for the `has_unique_object_representations` type trait while voting in [P0258R2](#). The submitter of this issue believes that such a feature macro should be added and given that its final definition should be related to the date of the acceptance of this issue (and should *not* be related to any historically accepted papers such as [P1614R2](#)).

[2020-01 Priority set to 1 and questions to LEWG after review on the reflector.]

**Previous resolution [SUPERSEDED]:**

This wording is relative to [N4842](#).

1. Modify 17.3.2 [\[version.syn\]](#), header `<version>` synopsis, as indicated (The symbolic `?????L` represents the date to be defined by the project editor):

```
[...]
#define __cpp_lib_is_swappable 201603L // also in <type_traits>
#define __cpp_lib_is_template_parameter_type ?????L // also in <type_traits>
#define __cpp_lib_jthread 201911L // also in <stop_token>, <thread>
[...]
```

2. Modify 20.15.2 [\[meta.type.synop\]](#), header `<type_traits>` synopsis, as indicated:

```
[...]
template<class T> struct has_unique_object_representations;

template<class T> struct has_strong_structural_equalityis_template_parameter_type;

// 20.15.5 \[meta.unary.prop.query\], type property queries
[...]
template<class T>
 inline constexpr bool has_unique_object_representations_v
 = has_unique_object_representations<T>::value;

template<class T>
 inline constexpr bool has_strong_structural_equalityis_template_parameter_type_v
 = has_strong_structural_equalityis_template_parameter_type<T>::value;

// 20.15.5 \[meta.unary.prop.query\], type property queries
[...]
```

3. Modify 20.15.2 [\[meta.type.synop\]](#), Table 47 ([tab:meta.unary.prop]) — "Type property predicates" — as indicated:

| Table 47: Type property predicates [tab:meta.unary.prop]                                             |                                                                                                                                                                                           |                                                                                                                                 |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Template                                                                                             | Condition                                                                                                                                                                                 | Preconditions                                                                                                                   |
| ...                                                                                                  |                                                                                                                                                                                           |                                                                                                                                 |
| template<class T><br>struct<br><del>has_strong_structural_equality</del> is_template_parameter_type; | The type T <del>has strong structural equality</del> (11.11.1 <del>[class.compare.default]</del> ) can be used as non-type <del>template-parameter</del> (13.2 <del>[temp.param]</del> ). | <del>I shall be a complete type, cv-void, or an array of unknown bound.</del> If T is a class type, T shall be a complete type. |
| ...                                                                                                  |                                                                                                                                                                                           |                                                                                                                                 |

[2020-02, Prague]

LEWG looked at this and suggested to remove the existing trait `has_strong_structural_equality` for now until someone demonstrates which usecases exist that would justify its existence.

[2020-02-13, Prague]

Set to Immediate.

**Proposed resolution:**

This wording is relative to [N4849](#).

1. Modify 20.15.2 [\[meta.type.synop\]](#), header <type\_traits> synopsis, as indicated:

```
[...]
template<class T> struct has_unique_object_representations;

template<class T> struct has_strong_structural_equality;

// 20.15.5 \[meta.unary.prop.query\], type property queries
[...]
template<class T>
 inline constexpr bool has_unique_object_representations_v
 = has_unique_object_representations<T>::value;

template<class T>
 inline constexpr bool has_strong_structural_equality_v
 = has_strong_structural_equality<T>::value;

// 20.15.5 \[meta.unary.prop.query\], type property queries
[...]
```

2. Modify 20.15.2 [\[meta.type.synop\]](#), Table 47 ([tab:meta.unary.prop]) — "Type property predicates" — as indicated:

| Table 47: Type property predicates [tab:meta.unary.prop]                          |                                                                                                                 |                                                                                       |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Template                                                                          | Condition                                                                                                       | Preconditions                                                                         |
| ...                                                                               |                                                                                                                 |                                                                                       |
| <del>template&lt;class T&gt;<br/>struct<br/>has_strong_structural_equality;</del> | <del>The type T has strong<br/>structural<br/>equality (11.11.1 <a href="#">[class.compare.default]</a>);</del> | <del>T shall be a complete type,<br/>cv void, or an array of<br/>unknown bound.</del> |

**3355<sup>(i)</sup>. The memory algorithms should support move-only input iterators introduced by P1207**

**Section:** 20.10.11.5 [\[uninitialized.copy\]](#), 20.10.11.6 [\[uninitialized.move\]](#) **Status:** [Immediate](#) **Submitter:** Corentin Jabot **Opened:** 2019-11-12 **Last modified:** 2020-02-14

**Priority:** 2

**View all other [issues](#)** in [\[uninitialized.copy\]](#).

**View all issues with [Immediate](#) status.**

**Discussion:**

[P1207](#) introduced move-only input iterators but did not modify the specialized memory algorithms to support them.

*[2020-01 Priority set to 2 after review on the reflector.]*

*[2020-02 Status to Immediate on Friday morning in Prague.]*

**Proposed resolution:**

This wording is relative to [N4842](#).

1. Modify 20.10.11.5 [\[uninitialized.copy\]](#) as indicated:

```
namespace ranges {
 template<input_iterator I, sentinel_for<I> S1,
 no_throw_forward_iterator O, no_throw_sentinel<O> S2>
 requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
 uninitialized_copy_result<I, O>
 uninitialized_copy(I ifirst, S1 ilast, O ofirst, S2 olast);
 template<input_range IR, no_throw_forward_range OR>
 requires constructible_from<range_value_t<OR>, range_reference_t<IR>>
 uninitialized_copy_result<safe_iterator_t<IR>, safe_iterator_t<OR>>
 uninitialized_copy(IR&& in_range, OR&& out_range);
}

-4- Preconditions: [ofirst, olast) shall not overlap with [ifirst, ilast).

-5- Effects: Equivalent to:

 for (; ifirst != ilast && ofirst != olast; ++ofirst, (void)++ifirst) {
 ::new (voidify(*ofirst)) remove_reference_t<iter_reference_t<O>>(*ifirst);
 }
 return {std::move(ifirst), ofirst};

[...]
```

```
namespace ranges {
 template<input_iterator I, no-throw-forward-iterator O, no-throw-sentinel<O> S>
 requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
 uninitialized_copy_n_result<I, O>
 uninitialized_copy_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);
}
```

-9- *Preconditions*: [ofirst, olast) shall not overlap with [ifirst, n).

-10- *Effects*: Equivalent to:

```
auto t = uninitialized_copy(counted_iterator(ifirst, n),
 default_sentinel, ofirst, olast);
return {std::move(t.in).base(), t.out};
```

2. Modify 20.10.11.6 [\[uninitialized.move\]](#) as indicated:

```
namespace ranges {
 template<input_iterator I, sentinel_for<I> S1,
 no-throw-forward-iterator O, no-throw-sentinel<O> S2>
 requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
 uninitialized_move_result<I, O>
 uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast);
 template<input_range IR, no-throw-forward-range OR>
 requires constructible_from<range_value_t<OR>, range_rvalue_reference_t<IR>>
 uninitialized_move_result<safe_iterator_t<IR>, safe_iterator_t<OR>>
 uninitialized_move(IR&& in_range, OR&& out_range);
}
```

-3- *Preconditions*: [ofirst, olast) shall not overlap with [ifirst, ilast).

-4- *Effects*: Equivalent to:

```
for (; ifirst != ilast && ofirst != olast; ++ofirst, (void)++ifirst) {
 ::new (voidify(*ofirst))
 remove_reference_t<iter_reference_t<O>>(ranges::iter_move(*ifirst));
}
return {std::move(ifirst), ofirst};
```

[...]

```
namespace ranges {
 template<input_iterator I, no-throw-forward-iterator O, no-throw-sentinel<O> S>
 requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
 uninitialized_move_n_result<I, O>
 uninitialized_move_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);
}
```

-8- *Preconditions*: [ofirst, olast) shall not overlap with [ifirst, n).

-9- *Effects*: Equivalent to:

```
auto t = uninitialized_move(counted_iterator(ifirst, n),
 default_sentinel, ofirst, olast);
return {std::move(t.in).base(), t.out};
```

## [3358<sup>\(i\)</sup>](#). [\[span.cons\]](#) is mistaken that `to_address` can throw

**Section:** 22.7.3.2 [\[span.cons\]](#) **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2019-12-10 **Last modified:** 2020-02-14

**Priority:** 0

**View other** [active issues](#) in [\[span.cons\]](#).

**View all other** [issues](#) in [\[span.cons\]](#).

**View all issues with** [Immediate](#) status.

**Discussion:**

[\[span.cons\]](#) paragraphs 6 and 9:

*Throws*: When and what `to_address(first)` throws.

could equivalently be "*Throws*: Nothing." since all overloads of `std::to_address` are noexcept. However, paragraph 9 fails to account for the fact that paragraph 8:

*Effects*: Initializes `data_` with `to_address(first)` and `size_` with `last - first`.

must evaluate `last - first`.



[2020-01-14 Status set to Tentatively Ready after ten positive votes on the reflector.]

[2020-01-14; Daniel comments]

The fixed wording in 22.7.3.2 [\[span.cons\]](#) p9 depends on the no-throw-guarantee of integer-class conversions to integral types. This guarantee is specified by LWG [3367](#).

**Proposed resolution:**

This wording is relative to [N4842](#).

1. Modify 22.7.3.2 [\[span.cons\]](#) paragraphs 6 and 9 as indicated:

[Drafting note:

1. The missing paragraph number of the *Preconditions* element at p7/p8 has already been reported as [editorial issue](#)
2. The effective change to "Throws: Nothing." in p6 has already been [applied editorially](#).

]

```
template<class It>
constexpr span(It first, size_type count);
```

[...]

-4- *Preconditions*: [...]

-5- *Effects*: Initializes `data_` with `to_address(first)` and `size_` with `count`.

-6- *Throws*: ~~When and what `to_address(first)` throws~~ **Nothing**.

```
template<class It, class End>
constexpr span(It first, End last);
```

[...]

-?- *Preconditions*: [...]

-8- *Effects*: Initializes `data_` with `to_address(first)` and `size_` with `last - first`.

-9- *Throws*: When and what ~~`to_address(first)`~~ **`last - first`** throws.

---

## [3359<sup>\(i\)</sup>](#). <chrono> leap second support should allow for negative leap seconds

**Section:** 27.11.8 [\[time.zone.leap\]](#) **Status:** [Immediate](#) **Submitter:** Asher Dunn **Opened:** 2019-12-16 **Last modified:** 2020-02-14

**Priority:** 3

**View all issues with [Immediate](#) status.**

**Discussion:**

class `leap` (which is expected to be renamed by [P1981R0](#) to `leap_second`) defined in 27.11.8 [\[time.zone.leap\]](#) should include support for both positive leap seconds (23:59:60 added to UTC at a specified time) and negative leap seconds (23:59:59 removed from UTC at a specified time). While only positive leap seconds have been inserted to date, the definition of UTC allows for both.

Update 27.11.8 [\[time.zone.leap\]](#) to specify the value of leap seconds in addition to their insertion date, and update wording and examples in 27.7 [\[time.clock\]](#) and 27.12 [\[time.format\]](#) that involve leap seconds to account for both types of leap second.

[2020-01 Priority set to 3 after review on the reflector.]

**Previous resolution [SUPERSEDED]:**

This wording is relative to [N4842](#).

1. Modify 27.7.2.2 [\[time.clock.utc.members\]](#) as indicated:

```
template<class Duration>
static sys_time<common_type_t<Duration, seconds>>
to_sys(const utc_time<Duration>& u);
```

- 2- *Returns*: A `sys_time t`, such that `from_sys(t) == u` if such a mapping exists. Otherwise `u` represents a `time_point` during a **positive** leap second insertion, **the conversion counts that leap second as not**

inserted, and the last representable value of `sys_time` prior to the insertion of the leap second is returned.

```
template<class Duration>
static utc_time<common_type_t<Duration, seconds>>
from_sys(const sys_time<Duration>& t);
```

-3- *Returns:* A `utc_time` `u`, such that `u.time_since_epoch()` - `t.time_since_epoch()` is equal to the number sum of leap seconds that were inserted between `t` and 1970-01-01. If `t` is exactly the date of leap second insertion, then the conversion counts that leap second as inserted.

[...]

2. Modify 27.7.2.3 [\[time.clock.utc.nonmembers\]](#) as indicated:

```
template<class Duration>
leap_second_info get_leap_second_info(const utc_time<Duration>& ut);
```

-6- *Returns:* A `leap_second_info` where `is_leap_second` is true if `ut` is during a positive leap second insertion, and otherwise false. `elapsed` is the number sum of leap seconds between 1970-01-01 and `ut`. If `is_leap_second` is true, the leap second referred to by `ut` is included in the count.

3. Modify 27.7.3.1 [\[time.clock.tai.overview\]](#) as indicated:

-1- The clock `tai_clock` measures seconds since 1958-01-01 00:00:00 and is offset 10s ahead of UTC at this date. That is, 1958-01-01 00:00:00 TAI is equivalent to 1957-12-31 23:59:50 UTC. Leap seconds are not inserted into TAI. Therefore every time a leap second is inserted into UTC, UTC ~~falls another second behind~~ shifts another second with respect to TAI. For example by 2000-01-01 there had been 22 positive and 0 negative leap seconds inserted so 2000-01-01 00:00:00 UTC is equivalent to 2000-01-01 00:00:32 TAI (22s plus the initial 10s offset).

4. Modify 27.7.4.1 [\[time.clock.gps.overview\]](#) as indicated:

-1- The clock `gps_clock` measures seconds since the first Sunday of January, 1980 00:00:00 UTC. Leap seconds are not inserted into GPS. Therefore every time a leap second is inserted into UTC, UTC ~~falls another second behind~~ shifts another second with respect to GPS. Aside from the offset from 1958y/January/1 to 1980y/January/Sunday[1], GPS is behind TAI by 19s due to the 10s offset between 1958 and 1970 and the additional 9 leap seconds inserted between 1970 and 1980.

5. Modify 27.11.8.1 [\[time.zone.leap.overview\]](#) as indicated:

```
namespace std::chrono {
class leap {
public:
 leap(const leap&) = default;
 leap& operator=(const leap&) = default;

 // unspecified additional constructors

 constexpr sys_seconds date() const noexcept;
 constexpr seconds value() const noexcept;
};
}
```

-1- Objects of type `leap` representing the date and value of the leap second insertions are constructed and stored in the time zone database when initialized.

-2- *[Example:*

```
for (auto& l : get_tzdb().leaps)
 if (l <= 2018y/March/17d)
 cout << l.date() << " " << l.value() << '\n';
```

Produces the output:

```
1972-07-01 00:00:00: 1s
1973-01-01 00:00:00: 1s
1974-01-01 00:00:00: 1s
1975-01-01 00:00:00: 1s
1976-01-01 00:00:00: 1s
1977-01-01 00:00:00: 1s
1978-01-01 00:00:00: 1s
1979-01-01 00:00:00: 1s
1980-01-01 00:00:00: 1s
1981-07-01 00:00:00: 1s
1982-07-01 00:00:00: 1s
1983-07-01 00:00:00: 1s
1985-07-01 00:00:00: 1s
1988-01-01 00:00:00: 1s
1990-01-01 00:00:00: 1s
1991-01-01 00:00:00: 1s
1992-07-01 00:00:00: 1s
1993-07-01 00:00:00: 1s
1994-07-01 00:00:00: 1s
```

```

1996-01-01 00:00:00: 1s
1997-07-01 00:00:00: 1s
1999-01-01 00:00:00: 1s
2006-01-01 00:00:00: 1s
2009-01-01 00:00:00: 1s
2012-07-01 00:00:00: 1s
2015-07-01 00:00:00: 1s
2017-01-01 00:00:00: 1s

```

— end example]

6. Modify 27.11.8.2 [time.zone.leap.members] as indicated:

```
constexpr sys_seconds date() const noexcept;
```

-1- Returns: The date and time at which the leap second was inserted.

```
constexpr seconds value() const noexcept;
```

-?- Returns: The value of the leap second. Always +1s to indicate a positive leap second or -1s to indicate a negative leap second. All leap seconds inserted up through 2017 were positive leap seconds.

7. Modify 27.12 [time.format] as indicated:

```
template<class Duration, class charT>
struct formatter<chrono::utc_time<Duration>, charT>;
```

-7- Remarks: If %Z is used, it is replaced with *STATICALLY-WIDEN*<charT>("UTC"). If %z (or a modified variant of %z) is used, an offset of 0min is formatted. If the argument represents a time during a positive leap second insertion, and if a seconds field is formatted, the integral portion of that format is *STATICALLY-WIDEN*<charT>("60").

[2020-02-14; Prague]

LWG Review. Some wording improvements have been made and lead to revised wording.

#### Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 27.7.2.2 [time.clock.utc.members] as indicated:

```
template<class Duration>
static sys_time<common_type_t<Duration, seconds>>
to_sys(const utc_time<Duration>& u);
```

-2- Returns: A sys\_time t, such that from\_sys(t) == u if such a mapping exists. Otherwise u represents a time\_point during a positive leap second insertion, the conversion counts that leap second as not inserted, and the last representable value of sys\_time prior to the insertion of the leap second is returned.

```
template<class Duration>
static utc_time<common_type_t<Duration, seconds>>
from_sys(const sys_time<Duration>& t);
```

-3- Returns: A utc\_time u, such that u.time\_since\_epoch() - t.time\_since\_epoch() is equal to the numbersum of leap seconds that were inserted between t and 1970-01-01. If t is exactly the date of leap second insertion, then the conversion counts that leap second as inserted.

[...]

2. Modify 27.7.2.3 [time.clock.utc.nonmembers] as indicated:

```
template<class Duration>
leap_second_info get_leap_second_info(const utc_time<Duration>& ut);
```

-6- Returns: A leap\_second\_info, lsi, where lsi.is\_leap\_second is true if ut is during a positive leap second insertion, and otherwise false. lsi.elapsed is the numbersum of leap seconds between 1970-01-01 and ut. If lsi.is\_leap\_second is true, the leap second referred to by ut is included in the countsum.

3. Modify 27.7.3.1 [time.clock.tai.overview] as indicated:

-1- The clock tai\_clock measures seconds since 1958-01-01 00:00:00 and is offset 10s ahead of UTC at this date. That is, 1958-01-01 00:00:00 TAI is equivalent to 1957-12-31 23:59:50 UTC. Leap seconds are not inserted into TAI. Therefore every time a leap second is inserted into UTC, UTC falls another second behind shifts another second with respect to TAI. For example by 2000-01-01 there had been 22 positive and 0 negative leap seconds inserted so 2000-01-01 00:00:00 UTC is equivalent to 2000-01-01 00:00:32 TAI (22s plus the initial 10s offset).

4. Modify 27.7.4.1 [time.clock.gps.overview] as indicated:

-1- The clock `gps_clock` measures seconds since the first Sunday of January, 1980 00:00:00 UTC. Leap seconds are not inserted into GPS. Therefore every time a leap second is inserted into UTC, UTC ~~falls another second behind~~ **shifts another second with respect to** GPS. Aside from the offset from 1958y/January/1 to 1980y/January/Sunday[1], GPS is behind TAI by 19s due to the 10s offset between 1958 and 1970 and the additional 9 leap seconds inserted between 1970 and 1980.

5. Modify 27.11.8.1 [\[time.zone.leap.overview\]](#) as indicated:

```
namespace std::chrono {
 class leap {
 public:
 leap(const leap&) = default;
 leap& operator=(const leap&) = default;

 // unspecified additional constructors

 constexpr sys_seconds date() const noexcept;
 constexpr seconds value() const noexcept;
 };
}
```

-1- Objects of type `leap` representing the date **and value** of the leap second insertions are constructed and stored in the time zone database when initialized.

-2- *[Example:*

```
for (auto& l : get_tzdb().leaps)
 if (l <= 2018y/March/17d)
 cout << l.date() << ": " << l.value() << '\n';
```

Produces the output:

```
1972-07-01 00:00:00: 1s
1973-01-01 00:00:00: 1s
1974-01-01 00:00:00: 1s
1975-01-01 00:00:00: 1s
1976-01-01 00:00:00: 1s
1977-01-01 00:00:00: 1s
1978-01-01 00:00:00: 1s
1979-01-01 00:00:00: 1s
1980-01-01 00:00:00: 1s
1981-07-01 00:00:00: 1s
1982-07-01 00:00:00: 1s
1983-07-01 00:00:00: 1s
1985-07-01 00:00:00: 1s
1988-01-01 00:00:00: 1s
1990-01-01 00:00:00: 1s
1991-01-01 00:00:00: 1s
1992-07-01 00:00:00: 1s
1993-07-01 00:00:00: 1s
1994-07-01 00:00:00: 1s
1996-01-01 00:00:00: 1s
1997-07-01 00:00:00: 1s
1999-01-01 00:00:00: 1s
2006-01-01 00:00:00: 1s
2009-01-01 00:00:00: 1s
2012-07-01 00:00:00: 1s
2015-07-01 00:00:00: 1s
2017-01-01 00:00:00: 1s
```

— end example]

6. Modify 27.11.8.2 [\[time.zone.leap.members\]](#) as indicated:

```
constexpr sys_seconds date() const noexcept;
```

-1- *Returns:* The date and time at which the leap second was inserted.

```
constexpr seconds value() const noexcept;
```

**-?- Returns:** **+1s** to indicate a positive leap second or **-1s** to indicate a negative leap second. *[Note: All leap seconds inserted up through 2019 were positive leap seconds. — end note]*

7. Modify 27.12 [\[time.format\]](#) as indicated:

```
template<class Duration, class charT>
 struct formatter<chrono::utc_time<Duration>, charT>;
```

-7- *Remarks:* If `%Z` is used, it is replaced with `STATICALLY-WIDEN<charT>("UTC")`. If `%z` (or a modified variant of `%z`) is used, an offset of `0min` is formatted. If the argument represents a time during a **positive** leap second insertion, and if a seconds field is formatted, the integral portion of that format is `STATICALLY-WIDEN<charT>("60")`.

**Section:** 32.3.4 [\[stopsource\]](#) **Status:** [Immediate](#) **Submitter:** Tim Song **Opened:** 2020-01-03 **Last modified:** 2020-02-14

**Priority:** 0

**View all issues with** [Immediate](#) status.

#### Discussion:

Just like `stop_token` (see LWG [3254](#)), `stop_source` in 32.3.4 [\[stopsource\]](#) declares an `operator!=` friend that is unnecessary in light of the new core language rules and should be struck.

*[2020-01-14 Status set to Tentatively Ready after six positive votes on the reflector.]*

#### Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 32.3.4 [\[stopsource\]](#), class `stop_source` synopsis, as indicated:

```
namespace std {
 [...]

 class stop_source {
 public:
 [...]

 [[nodiscard]] friend bool
 operator==(const stop_source& lhs, const stop_source& rhs) noexcept;
[[nodiscard]] friend bool
operator!=(const stop_source& lhs, const stop_source& rhs) noexcept;
 friend void swap(stop_source& lhs, stop_source& rhs) noexcept;
 };
}
```

2. Modify 32.3.4.3 [\[stopsource.cmp\]](#) and 32.3.4.4 [\[stopsource.special\]](#) as indicated:

##### ~~32.3.4.3 Non-member functions~~~~Comparisons~~ [\[stopsource.nonmembers.cmp\]](#)

```
[[nodiscard]] bool operator==(const stop_source& lhs, const stop_source& rhs) noexcept;
```

-1- *Returns:* true if lhs and rhs have ownership of the same stop state or if both lhs and rhs do not have ownership of a stop state; otherwise false.

```
[[nodiscard]] bool operator!=(const stop_source& lhs, const stop_source& rhs) noexcept;
```

~~-2- *Returns:* !(lhs==rhs);~~

##### ~~32.3.4.4 Specialized algorithms~~ [\[stopsource.special\]](#)

```
friend void swap(stop_source& x, stop_source& y) noexcept;
```

-1- *Effects:* Equivalent to: `x.swap(y)`.

---

## [3363](#)<sup>(i)</sup>. `drop_while_view` should opt-out of `sized_range`

**Section:** 24.7.9.2 [\[range.drop.while.view\]](#) **Status:** [Immediate](#) **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2020-01-07 **Last modified:** 2020-02-12

**Priority:** 1

**View all issues with** [Immediate](#) status.

#### Discussion:

If `drop_while_view`'s `iterator_t` and `sentinel_t` model `forward_iterator` and `sized_sentinel_for`, it will incorrectly satisfy `sized_range` thanks to the `size` member function inherited from `view_interface`.

Because it has to compute its `begin()`, it can never model `sized_range` due to not meeting its non-amortized  $O(1)$  requirement.

*[2020-01-16 Priority set to 1 after discussion on the reflector.]*

*[2020-02-10; Prague 2020; Casey comments and provides alternative wording]*

The fundamental issue here is that both `ranges::size` and `view_interface::size` (it should be unsurprising that many of the "default" implementations of member `meow` in `view_interface` look just like fallback cases of the `ranges::meow` CPO) have a case that returns the difference of `ranges::end` and `ranges::begin`. If `begin` and `end` are amortized\*  $O(1)$  but not "true"  $O(1)$ , then the resulting `size` operation is amortized  $O(1)$  and not "true"  $O(1)$  as required by the `sized_range` concept. I don't believe we can or should fix this on a case by case basis, but we should instead relax the complexity requirement for `sized_range` to be handwavy-amortized  $O(1)$ .

## Previous resolution [SUPERSEDED]:

This wording is relative to [N4849](#).

1. Add the following specialization to 24.2 [\[ranges.syn\]](#):

```
// [drop.while.view], drop while view
template<view V, class Pred>
 requires input_range<V> && is_object_v<Pred> &&
 indirect_unary_predicate<const Pred, iterator_t<V>>
 class drop_while_view;

 template<view V, class Pred>
 inline constexpr bool disable_sized_range<drop_while_view<V, Pred>> = true;

namespace views { inline constexpr unspecified drop_while = unspecified; }
```

[2020-02 Moved to Immediate on Tuesday in Prague.]

## Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 24.4.3 [\[range.sized\]](#) as indicated:

-2- Given an lvalue t of type `remove_reference_t<T>`, T models `sized_range` only if

(2.1) — `ranges::size(t)` is **amortized**  $\mathcal{O}(1)$ , does not modify t, and is equal to `ranges::distance(t)`, and

[...]

~~3- [Note: The complexity requirement for the evaluation of `ranges::size` is non-amortized, unlike the case for the complexity of the evaluations of `ranges::begin` and `ranges::end` in the range concept. — end note]~~

## [3364](#)<sup>(i)</sup>. Initialize data members of ranges and their iterators

**Section:** 24.7.7.2 [\[range.take.while.view\]](#), 24.7.8.2 [\[range.drop.view\]](#), 24.7.9.2 [\[range.drop.while.view\]](#), 24.7.15.3 [\[range.elements.iterator\]](#) **Status:** [Immediate](#) **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2020-01-07 **Last modified:** 2020-02-14

**Priority:** 0

**View all other** [issues](#) in [\[range.take.while.view\]](#).

**View all issues with** [Immediate](#) status.

## Discussion:

Before [P1035](#) was accepted, no data member in [\[ranges\]](#) whose type could potentially be an aggregate or fundamental type was left without initializer. P1035 left some such data members without initializer, so it is possible to have them have indeterminate values. We propose restoring consistency.

[2020-01-14 Status set to Tentatively Ready after five positive votes on the reflector.]

## Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 24.7.7.2 [\[range.take.while.view\]](#) as follows:

```
class take_while_view : public view_interface<take_while_view<V, Pred>> {
 template<bool> class sentinel; // exposition only

 V base_ = V(); // exposition only
 semiregular_box<Pred> pred_; // exposition only

public:
```

2. Modify 24.7.8.2 [\[range.drop.view\]](#) as follows:

```
private:
 V base_ = V(); // exposition only
 range_difference_t<V> count_ = 0; // exposition only
};
```

3. Modify 24.7.9.2 [\[range.drop.while.view\]](#) as follows:

```
private:
 V base_ = V(); // exposition only
 semiregular-box<Pred> pred_; // exposition only
};
```

4. Modify 24.7.15.3 [\[range.elements.iterator\]](#) as follows:

```
class elements_view<V, N>::iterator { // exposition only
 using base_t = conditional_t<Const, const V, V>;
 friend iterator<!Const>;

 iterator_t<base_t> current_ = iterator_t<base_t>(.);
public:
```

### [3367<sup>\(i\)</sup>](#). Integer-class conversions should not throw

**Section:** 23.3.4.4 [\[iterator.concept.winc\]](#) **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2020-01-07 **Last modified:** 2020-02-14

**Priority:** 0

**View other [active issues](#)** in [\[iterator.concept.winc\]](#).

**View all other [issues](#)** in [\[iterator.concept.winc\]](#).

**View all issues with [Immediate](#)** status.

#### Discussion:

It's widely established that neither conversions of integral types to `bool` nor conversions between different integral types throw exceptions. These properties are crucial to supporting exception guarantees in algorithms, containers, and other uses of iterators and their difference types. Integer-class types must provide the same guarantees to support the same use cases as do integer types.

*[2020-01-14; Daniel comments]*

1. We probably need to think about providing the stronger guarantee that all integer-class operations are also `noexcept` in addition to the guarantee that they do not throw any exceptions.
2. The fixed wording in LWG [3358](#), 22.7.3.2 [\[span.cons\]](#) p9 depends on the no-throw-guarantee of integer-class conversions to integral types.

*[2020-01-25 Status set to Tentatively Ready after five positive votes on the reflector.]*

#### Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 23.3.4.4 [\[iterator.concept.winc\]](#) as indicated:

*[Drafting note: There's a bit of drive-by editing here to change occurrences of the meaningless "type is convertible to type" to "expression is convertible to type". Paragraph 7 only has drive-by edits. ]*

-6- ~~All Expressions of~~ integer-class types are explicitly convertible to ~~all any~~ integral types ~~and Expressions of integral type are both~~ implicitly and explicitly convertible ~~from all integral types to any integer-class type. Conversions between integral and integer-class types do not exit via an exception.~~

-7- ~~All Expressions E of~~ integer-class types ~~I~~ are contextually convertible to `bool` as if by `bool(aE != I(0))` ~~where a is an instance of the integral-class type I.~~

### [3369<sup>\(i\)</sup>](#). `span`'s deduction-guide for built-in arrays doesn't work

**Section:** 22.7.3.1 [\[span.overview\]](#) **Status:** [Immediate](#) **Submitter:** Stephan T. Lavavej **Opened:** 2020-01-08 **Last modified:** 2020-02-14

**Priority:** 0

**View other [active issues](#)** in [\[span.overview\]](#).

**View all other [issues](#)** in [\[span.overview\]](#).

**View all issues with [Immediate](#)** status.

#### Discussion:

[N4842](#) 22.7.3.1 [\[span.overview\]](#) depicts:

```
template<class T, size_t N>
span(T (&)[N]) -> span<T, N>;
```

This isn't constrained by 22.7.3.3 [span.deduct]. Then, 22.7.3.2 [span.cons]/10 specifies:

```
template<size_t N> constexpr span(element_type (&arr)[N]) noexcept;
template<size_t N> constexpr span(array<value_type, N>& arr) noexcept;
template<size_t N> constexpr span(const array<value_type, N>& arr) noexcept;
```

*Constraints:*

- extent == dynamic\_extent || N == extent is true, and
- remove\_pointer\_t<decltype(data(arr))>(\*)[] is convertible to ElementType(\*)[].

Together, these cause CTAD to behave unexpectedly. Here's a minimal test case, reduced from libcxx's test suite:

```
C:\Temp>type span_ctad.cpp
#include <stddef.h>
#include <type_traits>

inline constexpr size_t dynamic_extent = static_cast<size_t>(-1);

template <typename T, size_t Extent = dynamic_extent>
struct span {
 template <size_t Size>
 requires (Extent == dynamic_extent || Extent == Size)
#ifdef WORKAROUND_WITH_TYPE_IDENTITY_T
 span(std::type_identity_t<T> (&)[Size]) {}
#else
 span(T (&)[Size]) {}
#endif
};

template <typename T, size_t Extent>
#ifdef WORKAROUND_WITH_REQUIRES_TRUE
requires (true)
#endif
span(T (&)[Extent]) -> span<T, Extent>;

int main() {
 int arr[] = {1,2,3};
 span s{arr};
 static_assert(std::is_same_v<decltype(s), span<int, 3>>,
 "CTAD should deduce span<int, 3>.");
}

C:\Temp>cl /EHsc /nologo /W4 /std:c++latest span_ctad.cpp
span_ctad.cpp
span_ctad.cpp(26): error C2338: CTAD should deduce span<int, 3>.

C:\Temp>cl /EHsc /nologo /W4 /std:c++latest /DWORKAROUND_WITH_TYPE_IDENTITY_T span_ctad.cpp
span_ctad.cpp

C:\Temp>cl /EHsc /nologo /W4 /std:c++latest /DWORKAROUND_WITH_REQUIRES_TRUE span_ctad.cpp
span_ctad.cpp

C:\Temp>
```

(MSVC and GCC 10 demonstrate this behavior. Clang is currently affected by [LLVM#44484](https://llvm.org/bugs/show_bug.cgi?id=44484).)

Usually, when there's an explicit deduction-guide, we can ignore any corresponding constructor, because the overload resolution tiebreaker 12.4.3 [over.match.best]/2.10 prefers deduction-guides. However, this is a mental shortcut only, and it's possible for guides generated from constructors to out-compete deduction-guides during CTAD. That's what's happening here.

Specifically, the constructor is constrained, while the deduction-guide is not constrained. This activates the "more specialized" tiebreaker first (12.4.3 [over.match.best]/2.5 is considered before /2.10 for deduction-guides). That goes through 13.7.6.2 [temp.func.order]/2 and 13.5.4 [temp.constr.order] to prefer the more constrained overload.

(In the test case, this results in span<int, dynamic\_extent> being deduced. That's because the constructor allows T to be deduced to be int. The constructor's Size template parameter is deduced to be 3, but that's unrelated to the class's Extent parameter. Because Extent has a default argument of dynamic\_extent, CTAD succeeds and deduces span<int, dynamic\_extent>.)

There are at least two possible workarounds: we could alter the constructor to prevent it from participating in CTAD, or we could constrain the deduction-guide, as depicted in the test case. Either way, we should probably include a Note, following the precedent of 21.3.2.2 [string.cons]/12.

Note that there are also deduction-guides for span from std::array. However, the constructors take array<value\_type, N> with using value\_type = remove\_cv\_t<ElementType>; so that prevents the constructors from interfering with CTAD.

I'm currently proposing to alter the constructor from built-in arrays. An alternative resolution to constrain the deduction-guide would look like: *"Constraints: true. [Note: This affects class template argument deduction. — end note]"*

[2020-01-25 Status set to Tentatively Ready after seven positive votes on the reflector.]



### Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 22.7.3.1 [\[span.overview\]](#), class template span synopsis, as indicated:

```
namespace std {
template<class ElementType, size_t Extent = dynamic_extent>
class span {
public:
 [...]
 // 22.7.3.2 \[span.cons\], constructors, copy, and assignment
 constexpr span() noexcept;
 [...]
 template<size_t N>
 constexpr span(type_identity_t<element_type> (&arr)[N]) noexcept;
 [...]
};
[...]
```

2. Modify 22.7.3.2 [\[span.cons\]](#) as indicated:

```
template<size_t N> constexpr span(type_identity_t<element_type> (&arr)[N]) noexcept;
template<size_t N> constexpr span(array<value_type, N>& arr) noexcept;
template<size_t N> constexpr span(const array<value_type, N>& arr) noexcept;
```

-10- Constraints:

(10.1) — extent == dynamic\_extent || N == extent is true, and

(10.2) — remove\_pointer\_t<decltype(data(arr))>(\*)[] is convertible to ElementType(\*)[].

-11- Effects: Constructs a span that is a view over the supplied array. [\[Note: type\\_identity t affects class template argument deduction. — end note\]](#)

-12- Postconditions: size() == N && data() == data(arr).

---

## [3371<sup>\(i\)</sup>](#). visit\_format\_arg and make\_format\_args are not hidden friends

Section: 20.20.5.1 [\[format.arg\]](#) Status: [Immediate](#) Submitter: Tim Song Opened: 2020-01-16 Last modified: 2020-02-14

Priority: 0

View all other [issues](#) in [\[format.arg\]](#).

View all issues with [Immediate](#) status.

### Discussion:

After [P1965R0](#), friend function and function template declarations always introduce hidden friends under the new blanket wording in 16.5.5.6 [\[hidden.friends\]](#). However, 20.20.5.1 [\[format.arg\]](#) contains "exposition only" friend declarations of visit\_format\_arg and make\_format\_args, and those are not intended to be hidden. The only reason to have these declarations in the first place is because these function templates are specified using the exposition-only private data members of basic\_format\_arg, but that's unnecessary — for example, shared\_ptr's constructors are not exposition-only friends of enable\_shared\_from\_this, even though the former are shown as assigning to the latter's exposition-only weak\_this private data member (see 20.11.3.1 [\[util.smartptr.shared.const\]](#)p1).

*[2020-02-01 Status set to Tentatively Ready after five positive votes on the reflector.]*

### Proposed resolution:

This wording is relative to [N4842](#).

1. Edit 20.20.5.1 [\[format.arg\]](#), class template basic\_format\_arg synopsis, as indicated:

```
namespace std {
 template<class Context>
 class basic_format_arg {
 [...]

 template<class Visitor, class Ctx>
 friend auto visit_format_arg(Visitor&& vis,
 basic_format_arg<Ctx> arg); // exposition only

 template<class Ctx, class... Args>
 friend format_arg_store<Ctx, Args...>
 make_format_args(const Args&... args); // exposition only
 [...]
}
```

```
};
}
```

### [3372\(i\)](#). `vformat_to` should not try to deduce `Out` twice

Section: 20.20.3 [\[format.functions\]](#) Status: [Immediate](#) Submitter: Tim Song Opened: 2020-01-16 Last modified: 2020-02-14

Priority: 0

View other [active issues](#) in [\[format.functions\]](#).

View all other [issues](#) in [\[format.functions\]](#).

View all issues with [Immediate](#) status.

#### Discussion:

`vformat_to` currently deduces `Out` from its first and last arguments. This requires its last argument's type to be a specialization of `basic_format_args`, which notably prevents the use of *format-arg-store* arguments directly. This is unnecessary: we should only deduce from the first argument.

[2020-02-01 Status set to Tentatively Ready after six positive votes on the reflector.]

#### Proposed resolution:

This wording is relative to [N4842](#).

1. Edit 20.20.1 [\[format.syn\]](#), header `<format>` synopsis, as indicated:

```
namespace std {
 [...]
 template<class Out>
 Out vformat_to(Out out, string_view fmt, format_args_t<type_identity_t<Out>, char> args);
 template<class Out>
 Out vformat_to(Out out, wstring_view fmt, format_args_t<type_identity_t<Out>, wchar_t> args);
 template<class Out>
 Out vformat_to(Out out, const locale& loc, string_view fmt,
 format_args_t<type_identity_t<Out>, char> args);
 template<class Out>
 Out vformat_to(Out out, const locale& loc, wstring_view fmt,
 format_args_t<type_identity_t<Out>, wchar_t> args);
 [...]
}
```

2. Edit 20.20.3 [\[format.functions\]](#) p8 through p10 as indicated:

```
template<class Out, class... Args>
 Out format_to(Out out, string_view fmt, const Args&... args);
template<class Out, class... Args>
 Out format_to(Out out, wstring_view fmt, const Args&... args);

-8- Effects: Equivalent to:

 using context = basic_format_context<Out, decltype(fmt)::value_type>;
 return vformat_to(out, fmt, make_format_args<context>(args...));

template<class Out, class... Args>
 Out format_to(Out out, const locale& loc, string_view fmt, const Args&... args);
template<class Out, class... Args>
 Out format_to(Out out, const locale& loc, wstring_view fmt, const Args&... args);
```

-9- Effects: Equivalent to:

```
 using context = basic_format_context<Out, decltype(fmt)::value_type>;
 return vformat_to(out, loc, fmt, make_format_args<context>(args...));

template<class Out>
 Out vformat_to(Out out, string_view fmt, format_args_t<type_identity_t<Out>, char> args);
template<class Out>
 Out vformat_to(Out out, wstring_view fmt, format_args_t<type_identity_t<Out>, wchar_t> args);
template<class Out>
 Out vformat_to(Out out, const locale& loc, string_view fmt,
 format_args_t<type_identity_t<Out>, char> args);
template<class Out>
 Out vformat_to(Out out, const locale& loc, wstring_view fmt,
 format_args_t<type_identity_t<Out>, wchar_t> args);
```

-10- Let `charT` be `decltype(fmt)::value_type`.

**3373<sup>(i)</sup>. {to, from}\_chars\_result and format\_to\_n\_result need the "we really mean what we say" wording**

**Section:** 20.19.1 [[charconv.syn](#)], 20.20.1 [[format.syn](#)] **Status:** [Immediate](#) **Submitter:** Tim Song **Opened:** 2020-01-16 **Last modified:** 2020-02-14

**Priority:** 0

**View all other [issues](#)** in [[charconv.syn](#)].

**View all issues with [Immediate](#)** status.

**Discussion:**

To ensure that `to_chars_result`, `from_chars_result`, and `format_to_n_result` can be used in structured bindings, they need the special wording we use to negate the general library permission to add private data members and bases.

*[2020-02-01 Status set to Tentatively Ready after six positive votes on the reflector.]*

**Proposed resolution:**

This wording is relative to [N4842](#).

1. Add a paragraph at the end of 20.19.1 [[charconv.syn](#)] as follows:

-?- The types `to_chars_result` and `from_chars_result` have the data members and special members specified above. They have no base classes or members other than those specified.

2. Add a paragraph at the end of 20.20.1 [[format.syn](#)] as follows:

-1- The class template `format_to_n_result` has the template parameters, data members, and special members specified above. It has no base classes or members other than those specified.

**3374<sup>(i)</sup>. P0653 + P1006 should have made the other `std::to_address` overload `constexpr`**

**Section:** 20.10.4 [[pointer.conversion](#)] **Status:** [Immediate](#) **Submitter:** Billy O'Neal III **Opened:** 2020-01-14 **Last modified:** 2020-02-14

**Priority:** 0

**View all issues with [Immediate](#)** status.

**Discussion:**

While reviewing some interactions with [P0653](#) + [P1006](#), Billy discovered that one of the overloads was missing the `constexpr` tag. This might be a typo or a missed merge interaction between P0653 (which adds `to_address` with the pointer overload being `constexpr`) and P1006 (which makes `pointer_traits::pointer_to` `constexpr`). Mail was sent the LWG reflector, and Glen Fernandes, the author of P0653, indicates that this might have been an oversight.

*[2020-02-01 Status set to Tentatively Ready after seven positive votes on the reflector.]*

**Proposed resolution:**

This wording is relative to [N4842](#).

1. Modify 20.10.2 [[memory.syn](#)], header `<memory>` synopsis, as indicated:

```
[...]
// 20.10.4 [pointer.conversion], pointer conversion
template<class T>
 constexpr T* to_address(T* p) noexcept;
template<class Ptr>
 constexpr auto to_address(const Ptr& p) noexcept;
[...]
```

2. Modify 20.10.4 [[pointer.conversion](#)] as indicated:

```
template<class Ptr> constexpr auto to_address(const Ptr& p) noexcept;
```

-3- *Returns:* `pointer_traits<Ptr>::to_address(p)` if that expression is well-formed (see 20.10.3.3 [[pointer.traits.optmem](#)]), otherwise `to_address(p.operator->())`.

**3375<sup>(i)</sup>. decay in `viewable_range` should be `remove_cvref`**

**Section:** 24.4.5 [\[range.refinements\]](#) **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2020-01-14 **Last modified:** 2020-02-13

**Priority:** 0

**View other** [active issues](#) in [\[range.refinements\]](#).

**View all other** [issues](#) in [\[range.refinements\]](#).

**View all issues with** [Immediate](#) status.

#### Discussion:

The viewable\_range concept is defined in 24.4.5 [\[range.refinements\]](#) as:

```
template<class T>
concept viewable_range =
 range<T> && (safe_range<T> || view<decay_t<T>>);
```

Since neither pointer types, array types, nor function types model view, view<decay\_t<T>> here could be simplified to view<remove\_cvref\_t<T>>. The use of decay\_t is an artifact of the Ranges TS being based on C++14 which didn't have remove\_cvref\_t. [Note that the proposed change is not purely editorial since the difference is observable to subsumption.]

*[2020-02-01 Status set to Tentatively Ready after five positive votes on the reflector.]*

#### Previous resolution [SUPERSEDED]:

This wording is relative to [N4842](#).

1. Modify 24.4.5 [\[range.refinements\]](#) as indicated:

-4- The viewable\_range concept specifies the requirements of a range type that can be converted to a view safely.

```
template<class T>
concept viewable_range =
 range<T> && (safe_range<T> || view<decay_tremove_cvref<T>>);
```

*[2020-02-06 Casey provides a corrected P/R]*

... in response to Jonathan's observation that remove\_cvref<T> is both the wrong type *and* not what the discussion argues for.

*[2020-02 Status to Immediate on Thursday morning in Prague.]*

#### Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 24.4.5 [\[range.refinements\]](#) as indicated:

-4- The viewable\_range concept specifies the requirements of a range type that can be converted to a view safely.

```
template<class T>
concept viewable_range =
 range<T> && (safe_range<T> || view<decay_tremove_cvref_t<T>>);
```

---

### [3377<sup>\(i\)</sup>](#). `elements_view::iterator` befriends a specialization of itself

**Section:** 24.7.15.3 [\[range.elements.iterator\]](#) **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2020-01-18 **Last modified:** 2020-02-14

**Priority:** 0

**View all issues with** [Immediate](#) status.

#### Discussion:

The synopsis of the exposition-only class template `elements_view::iterator` in 24.7.15.3 [\[range.elements.iterator\]](#) includes the declaration "friend iterator<!const>;". We typically don't depict such friend relationships in the Library specification, leaving the choice of how to implement access to private data from external sources to implementer magic. For consistency, we should strike this occurrence from `elements_view::iterator`.

*[2020-02-08 Status set to Tentatively Ready after ten positive votes on the reflector.]*

#### Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 24.7.15.3 [\[range.elements.iterator\]](#), class template `elements_view::iterator` synopsis, as indicated:

```

namespace std::ranges {
 template<class V, size_t N>
 template<bool Const>
 class elements_view<V, N>::iterator { // exposition only
 using base_t = conditional_t<Const, const V, V>;
 friend iterator<Const>;

 iterator_t<base_t> current_;
 public:
 [...]
 };
 [...]
}

```

### 3379<sup>(i)</sup>. "safe" in several library names is misleading

**Section:** 24.2 [\[ranges.syn\]](#) **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2020-01-21 **Last modified:** 2020-02-13

**Priority:** 1

**View other** [active issues](#) in [\[ranges.syn\]](#).

**View all other** [issues](#) in [\[ranges.syn\]](#).

**View all issues with** [Immediate](#) status.

#### Discussion:

Various WG21 members have commented that the use of "safe" in the names `safe_range`, `enable_safe_range`, `safe_iterator_t`, and `safe_subrange_t` to mean "fairly unlikely to produce dangling iterators" is inappropriate. The term "safe" has very strong connotations in some application domains, and these names don't intend such connotations. We could avoid confusion by changing these names to avoid the use of "safe". The Ranges Illuminati has deemed "borrowed" to be more appropriate: it reflects the fact that such ranges often "borrow" iterators from an adapted range which allows them to not be lifetime-bound to the adapting range.

*[2020-02-08 Issue Prioritization]*

Priority to 1 after reflector discussion. This issue needs to be looked at by LEWG.

*[2020-02-13, Prague]*

Set to Immediate.

#### Proposed resolution:

This wording is relative to [N4849](#).

1. Replace all occurrences of `safe_range`, `enable_safe_range`, `safe_iterator_t`, and `safe_subrange_t` in the working draft with `borrowed_range`, `enable_borrowed_range`, `borrowed_iterator_t`, and `borrowed_subrange_t`.

### 3380<sup>(i)</sup>. `common_type` and comparison categories

**Section:** 20.15.7.6 [\[meta.trans.other\]](#) **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2020-01-23 **Last modified:** 2020-02-14

**Priority:** 0

**View other** [active issues](#) in [\[meta.trans.other\]](#).

**View all other** [issues](#) in [\[meta.trans.other\]](#).

**View all issues with** [Immediate](#) status.

#### Discussion:

There are two paragraphs in the the definition of `common_type`:

- Otherwise, if both `D1` and `D2` denote comparison category types (17.11.2.1 [\[cmp.categories.pre\]](#)), let `C` denote the common comparison type (11.11.3 [\[class.spaceship\]](#)) of `D1` and `D2`.
- Otherwise, if `decay_t<decltype(false ? declval<D1>() : declval<D2>())>` denotes a valid type, let `C` denote that type.

[P1614R2](#) added the first bullet so that `common_type_t<strong_equality, T>` would be the same type as `common_comparison_category_t<strong_equality, T>`; other cases are correctly handled by the second (pre-existing) bullet. After application of [P1959R0](#) in Belfast, `std::strong_equality` is no more. We can now strike the first bullet without changing the behavior of `common_type`.

*[2020-02-08 Status set to Tentatively Ready after seven positive votes on the reflector.]*

### Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 20.15.7.6 [\[meta.trans.other\]](#) as indicated:

-3- Note A: For the `common_type` trait applied to a template parameter pack `T` of types, the member type shall be either defined or not present as follows:

(3.1) — [...]

[...]

(3.3) — If `sizeof... (T)` is two, let the first and second types constituting `T` be denoted by `T1` and `T2`, respectively, and let `D1` and `D2` denote the same types as `decay_t<T1>` and `decay_t<T2>`, respectively.

(3.3.1) — If `is_same_v<T1, D1>` is false or `is_same_v<T2, D2>` is false, let `C` denote the same type, if any, as `common_type_t<D1, D2>`.

(3.3.2) — [Note: None of the following will apply if there is a specialization `common_type<D1, D2>`. — end note]

~~(3.3.3) — Otherwise, if both `D1` and `D2` denote comparison category types (17.11.2.1 [\[cmp.categories.pre\]](#)), let `C` denote the common comparison type (11.11.3 [\[class.spaceship\]](#)) of `D1` and `D2`.~~

(3.3.4) — Otherwise, if

`decay_t<decltype(false ? declval<D1>() : declval<D2>())>`

denotes a valid type, let `C` denote that type.

(3.3.5) — [...]

[...]

---

## [3381](#)<sup>(i)</sup>. `begin` and `data` must agree for `contiguous_range`

**Section:** 24.4.5 [\[range.refinements\]](#) **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2020-01-25 **Last modified:** 2020-02-10

**Priority:** 0

**View other [active issues](#)** in [\[range.refinements\]](#).

**View all other [issues](#)** in [\[range.refinements\]](#).

**View all issues with [Immediate](#) status.**

### Discussion:

The definition of the `contiguous_range` concept in 24.4.5 [\[range.refinements\]](#)/2 requires that `ranges::data(r)` be valid for a `contiguous_range` `r`, but fails to impose the obvious semantic requirement that `to_address(ranges::begin(r)) == ranges::data(r)`. In other words, `data` and `begin` must agree so that `[begin(r), end(r))` and the counted range `data(r) + [0, size(r))` (this is the new "counted range" specification syntax per [working draft issue 2932](#)) denote the same sequence of elements.

*[2020-02 Prioritized as IMMEDIATE Monday morning in Prague]*

### Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 24.4.5 [\[range.refinements\]](#) as indicated:

-2- `contiguous_range` additionally requires that the `ranges::data` customization point (24.3.11 [\[range.prim.data\]](#)) is usable with the range.

```
template<class T>
concept contiguous_range =
 random_access_range<T> && contiguous_iterator<iterator_t<T>>> &&
 requires(T& t) {
 { ranges::data(t) } -> same_as<add_pointer_t<range_reference_t<T>>>;
 };

```

-?- Given an expression `t` such that `decltype((t))` is `T&`, `T` models `contiguous_range` only if `(to_address(ranges::begin(t)) == ranges::data(t)).`

-3- The `common_range` concept [...]

---

### [3382\(i\)](#). NTTP for pair and array

Section: 20.4.2 [[pairs.pair](#)], 22.3.7 [[array](#)] Status: [Immediate](#) Submitter: Barry Revzin Opened: 2020-01-27 Last modified: 2020-02-14

Priority: 2

View other [active issues](#) in [[pairs.pair](#)].

View all other [issues](#) in [[pairs.pair](#)].

View all issues with [Immediate](#) status.

#### Discussion:

We had [this NB ballot issue](#), to ensure that `std::array` could be a NTTP. But after [P1907](#), we still need some kind of wording to ensure that `std::array` (and also `std::pair`) have no extra private members or base classes.

This is similar to LWG [3373](#) — maybe we just need to add:

The class template `pair/array` has the data members specified above. It has no base classes or data members other than those specified.

[2020-02 Prioritized as P2 Monday morning in Prague]

#### Previous resolution [SUPERSEDED]:

This wording is relative to [N4849](#).

1. Modify 20.4.2 [[pairs.pair](#)] as indicated:

-1- Constructors and member functions of `pair` do not throw exceptions unless one of the element-wise operations specified to be called for that operation throws an exception.

-2- The defaulted move and copy constructor, respectively, of `pair` is a constexpr function if and only if all required element-wise initializations for copy and move, respectively, would satisfy the requirements for a constexpr function.

-3- If `(is_trivially_destructible_v<T1> && is_trivially_destructible_v<T2>)` is true, then the destructor of `pair` is trivial.

-?- The class template `pair` has the data members specified above. It has no base classes or data members other than those specified.

2. Modify 22.3.7.1 [[array.overview](#)] as indicated:

-1- The header `<array>` defines a class template for storing fixed-size sequences of objects. An array is a contiguous container (22.2.1 [[container.requirements.general](#)]). An instance of `array<T, N>` stores `N` elements of type `T`, so that `size() == N` is an invariant.

-2- An array is an aggregate (9.4.1 [[dcl.init.aggr](#)]) that can be list-initialized with up to `N` elements whose types are convertible to `T`.

-3- An array meets all of the requirements of a container and of a reversible container (22.2 [[container.requirements](#)]), except that a default constructed array object is not empty and that `swap` does not have constant complexity. An array meets some of the requirements of a sequence container (22.2.3 [[sequence.reqmts](#)]). Descriptions are provided here only for operations on array that are not described in one of these tables and for operations where there is additional semantic information.

-?- The class template `array` has the data members specified in subclauses 22.3.7.1 [[array.overview](#)] and 22.3.7.5 [[array.zero](#)]. It has no base classes or data members other than those specified.

-4- [...]

[2020-02-13, Prague]

Tim Song and Tomasz were trying to come up with general wording that could be reused for both `pair` and `array` (and other types). They suggest that if it should be in scope for C++20, it would be better to provide non-general wording for `pair` and `array` (that is easier to get right).

For completeness (and future wording) the generalized wording is included. The definition of *structurally compatible with*:

The type `T` is *structurally compatible with subs*, if for the values `t1` and `t2` of type `T`:

- `T` is a structural type (13.2 [[temp.param](#)]) if the types of subobjects of `t1` designated by *subs* are all structural types.
- `t1` is template-argument-equivalent (13.6 [[temp.type](#)]) to `t2`, if and only if, for each subject designated by *subs*, the value of subobject of `t1` is template-argument-equivalent to the value of the corresponding subobject of `t2`.

Then changes for array/pair would then look like:

pair<T, U> is structurally compatible (<some-reference>) with first and second.

array<T, N> is structurally compatible with its elements (if any).

[2020-02 Status to Immediate on Friday morning in Prague.]

#### Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 20.4.2 [\[pairs.pair\]](#) as indicated:

-1- Constructors and member functions of pair do not throw exceptions unless one of the element-wise operations specified to be called for that operation throws an exception.

-2- The defaulted move and copy constructor, respectively, of pair is a constexpr function if and only if all required element-wise initializations for copy and move, respectively, would satisfy the requirements for a constexpr function.

-3- If (is\_trivially\_destructible\_v<T1> && is\_trivially\_destructible\_v<T2>) is true, then the destructor of pair is trivial.

**-?- pair<T, U> is a structural type (13.2 [\[temp.param\]](#)) if T and U are both structural types. Two values p1 and p2 of type pair<T, U> are template-argument-equivalent (13.6 [\[temp.type\]](#)) if and only if p1.first and p2.first are template-argument-equivalent and p1.second and p2.second are template-argument-equivalent.**

2. Modify 22.3.7.1 [\[array.overview\]](#) as indicated:

-1- The header <array> defines a class template for storing fixed-size sequences of objects. An array is a contiguous container (22.2.1 [\[container.requirements.general\]](#)). An instance of array<T, N> stores N elements of type T, so that size() == N is an invariant.

-2- An array is an aggregate (9.4.1 [\[dcl.init.aggr\]](#)) that can be list-initialized with up to N elements whose types are convertible to T.

-3- An array meets all of the requirements of a container and of a reversible container (22.2 [\[container.requirements\]](#)), except that a default constructed array object is not empty and that swap does not have constant complexity. An array meets some of the requirements of a sequence container (22.2.3 [\[sequence.reqmts\]](#)). Descriptions are provided here only for operations on array that are not described in one of these tables and for operations where there is additional semantic information.

**-?- array<T, N> is a structural type (13.2 [\[temp.param\]](#)) if T is a structural type. Two values a1 and a2 of type array<T, N> are template-argument-equivalent (13.6 [\[temp.type\]](#)) if and only if each pair of corresponding elements in a1 and a2 are template-argument-equivalent.**

-4- [...]

---

### [3383](#)<sup>(i)</sup>. §[time.zone.leap.nonmembers] sys\_seconds should be replaced with seconds

Section: 27.11.8.3 [\[time.zone.leap.nonmembers\]](#) Status: [Immediate](#) Submitter: Jiang An Opened: 2020-01-30 Last modified: 2020-02-12

Priority: 1

View all issues with [Immediate](#) status.

#### Discussion:

In [N4849](#) 27.11.8.3 [\[time.zone.leap.nonmembers\]](#)/12, the type template parameter Duration is constrained by three\_way\_comparable\_with<sys\_seconds>. However, since std::chrono::sys\_seconds is a time point type and Duration must be a duration type, they can never be compared directly via operator<=>.

I guess that the actual intent is comparing Duration with the duration type of std::chrono::sys\_seconds, i.e. std::chrono::seconds. And thus sys\_seconds should be replaced with seconds here.

[2020-02 Prioritized as P1 Monday morning in Prague]

#### Previous resolution [SUPERSEDED]:

This wording is relative to [N4849](#).

1. Modify 27.2 [\[time.syn\]](#), header <chrono> synopsis, as indicated:

```
namespace std {
 [...]
 namespace chrono {
 [...]
 template<three_way_comparable_with<sys_seconds> Duration>
 auto operator<=>(const leap& x, const sys_time<Duration>& y);
 }
}
```



```

 [...]
 }
 [...]
}

```

2. Modify 27.11.8.3 [\[time.zone.leap.nonmembers\]](#) as indicated:

```

template<three_way_comparable_with<sys_seconds> Duration>
constexpr auto operator<=>(const leap& x, const sys_time<Duration>& y) noexcept;

-12- Returns: x.date() <=> y.

```

[2020-02-10, Prague; Howard suggests alternative wording]

The below shown alternative wording does more directly describe the constrained code (by comparing time\_points), even though in the very end the code specifying the effects finally goes down to actually return `x.date().time_since_epoch() <=> y.time_since_epoch()` (thus comparing durations).

#### Previous resolution [SUPERSEDED]:

This wording is relative to [N4849](#).

1. Modify 27.2 [\[time.syn\]](#), header <chrono> synopsis, as indicated:

The synopsis does provide an additional drive-by fix to eliminate the mismatch of the constexpr and noexcept in declaration and prototype specification.

```

namespace std {
 [...]
 namespace chrono {
 [...]
 template<three_way_comparable_with<sys_seconds> class Duration>
 requires three_way_comparable_with<sys_seconds, sys_time<Duration>>
 constexpr auto operator<=>(const leap& x, const sys_time<Duration>& y) noexcept;
 [...]
 }
 [...]
}

```

2. Modify 27.11.8.3 [\[time.zone.leap.nonmembers\]](#) as indicated:

```

template<three_way_comparable_with<sys_seconds> class Duration>
 requires three_way_comparable_with<sys_seconds, sys_time<Duration>>
 constexpr auto operator<=>(const leap& x, const sys_time<Duration>& y) noexcept;

-12- Returns: x.date() <=> y.

```

[2020-02-11, Prague; Daniel suggests alternative wording]

During today's LWG discussion of this issue the observation was made that there also exists a mismatch regarding the noexcept specifier for both declarations, but for this second deviation a corresponding change does *not* seem to be a good drive-by fix candidate, because we have a function template here that allows supporting user-defined types, whose comparison may throw (Note that the corresponding operator<=> or other comparison function declarations of the duration and time\_point templates are *not* specified as noexcept function templates). The revised wording below does therefore intentionally *not* change the currently existing noexcept-specifier mismatch, but a separate issue should instead be opened for the general noexcept-specifier mismatches for all comparison function templates of `std::chrono::leap`. Daniel has volunteered to take care for this issue.

[2020-02 Moved to Immediate on Tuesday in Prague.]

#### Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 27.2 [\[time.syn\]](#), header <chrono> synopsis, as indicated:

[Drafting note: The synopsis does provide an additional drive-by fix to eliminate the mismatch of the constexpr in declaration and prototype specification, but does *not* so for a similar mismatch of the exception-specifications of both declarations.]

```

namespace std {
 [...]
 namespace chrono {
 [...]
 template<three_way_comparable_with<sys_seconds> class Duration>
 requires three_way_comparable_with<sys_seconds, sys_time<Duration>>
 constexpr auto operator<=>(const leap& x, const sys_time<Duration>& y);
 [...]
 }
 [...]
}

```

```

 }
 [...]
}

```

2. Modify 27.11.8.3 [\[time.zone.leap.nonmembers\]](#) as indicated:

```

template<three_way_comparable_with<sys_seconds>class Duration>
requires three_way_comparable_with<sys_seconds, sys_time<Duration>>
constexpr auto operator<=>(const leap& x, const sys_time<Duration>& y) noexcept;

-12- Returns: x.date() <=> y.

```

### [3384](#)<sup>(i)</sup>. `transform_view::sentinel` has an incorrect operator-

**Section:** 24.7.5.4 [\[range.transform.sentinel\]](#) **Status:** [Immediate](#) **Submitter:** Ville Voutilainen **Opened:** 2020-01-31 **Last modified:** 2020-02-10

**Priority:** 0

**View all issues with [Immediate](#) status.**

#### Discussion:

1. `transform_view::iterator` has an exposition-only member `current_` (24.7.5.3 [\[range.transform.iterator\]](#))
2. `transform_view::sentinel` has an exposition-only member `end_` (24.7.5.4 [\[range.transform.sentinel\]](#))
3. at 24.7.5.4 [\[range.transform.sentinel\]](#)/6 we have:

```

friend constexpr range_difference_t<Base>
operator-(const sentinel& y, const iterator<Const>& x)
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

Effects: Equivalent to: return x.end_ - y.current_;

```

`x` is an `iterator`, so it has `current_`, not `end_`. `y` is a `sentinel`, so it has `end_`, not `current_`.

[2020-02 Prioritized as IMMEDIATE Monday morning in Prague]

#### Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 24.7.5.4 [\[range.transform.sentinel\]](#) as indicated:

```

friend constexpr range_difference_t<Base>
operator-(const sentinel& y, const iterator<Const>& x)
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

-6- Effects: Equivalent to: return *y.end_ - yx.current_;

```

### [3385](#)<sup>(i)</sup>. `common_iterator` is not sufficiently constrained for non-copyable iterators

**Section:** 23.5.4.1 [\[common.iterator\]](#) **Status:** [Immediate](#) **Submitter:** Corentin Jabot **Opened:** 2020-01-31 **Last modified:** 2020-02-10

**Priority:** 0

**View all issues with [Immediate](#) status.**

#### Discussion:

We don't actually specify anywhere that non-copyable iterators do not have a `common_iterator` (and it would make little sense for them to as algorithms dealing with C++17 iterators are not expecting non-copyable things) As it stands `common_iterator` can be created from move only iterator but are then non-copyable themselves. [P1862](#) already constrains `common_view` in a similar fashion

[2020-02 Prioritized as IMMEDIATE Monday morning in Prague]

#### Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 23.2 [\[iterator.synopsis\]](#), header `<iterator>` synopsis, as indicated:

```

namespace std {
 [...]
 // 23.5.4 \[iterators.common\], common iterators
 template<input_or_output_iterator I, sentinel_for<I> S>

```

```

 requires (!same_as<I, S> && copyable<I>)
 class common_iterator;
 [...]
}

```

2. Modify 23.5.4.1 [\[common.iterator\]](#), class template `common_iterator` synopsis, as indicated:

```

namespace std {
 template<input_or_output_iterator I, sentinel_for<I> S>
 requires (!same_as<I, S> && copyable<I>)
 class common_iterator {
 public:
 [...]
 };
 [...]
}

```

### [3387<sup>\(i\)</sup>](#). §[\[range.reverse.view\]](#) `reverse_view<V>` unintentionally requires `range<const V>`

**Section:** 24.7.14.2 [\[range.reverse.view\]](#) **Status:** [Immediate](#) **Submitter:** Patrick Palka **Opened:** 2020-02-04 **Last modified:** 2020-02-10

**Priority:** 0

**View all issues with** [Immediate](#) status.

#### **Discussion:**

`reverse_view<V>` requires `bidirectional_range<V>`, but not `range<const V>`, which means that `iterator_t<const V>` might be an invalid type. The return types of the `begin()` `const` and `end()` `const` overloads make use of `iterator_t<const V>` in a non-SFINAE context, which means that instantiating `reverse_view<X>` is ill-formed unless `range<const X>` is satisfied.

Code like `x | views::filter(p) | views::reverse` fails to compile because `const filter_view<...>` does not model `range`, so `iterator_t<const filter_view<...>>` is invalid.

Either `range<const V>` needs to be in the class' requires-clause, or the return types of the `const`-qualified `begin()` and `end()` need to delay use of `iterator_t<const V>` until `range<const V>` is known to be satisfied.

Giving these overloads an `auto` return type means the type is determined when the member is called. The constraint `common_range<const V>` appropriately restricts the selection of these overloads, so they can only be called when the type is valid. This is what `cmcstl2` does. `range-v3` makes the `begin()` `const` and `end()` `const` members into function templates, so that they are SFINAE contexts.

This is related to [3347](#).

*[2020-02 Prioritized as IMMEDIATE Monday morning in Prague]*

#### **Proposed resolution:**

This wording is relative to [N4849](#).

1. Modify 24.7.14.2 [\[range.reverse.view\]](#) as indicated:

```

namespace std::ranges {
 template<view V>
 requires bidirectional_range<V>
 class reverse_view : public view_interface<reverse_view<V>> {
 [...]
 constexpr reverse_iterator<iterator_t<V>> begin();
 constexpr reverse_iterator<iterator_t<V>> begin() requires common_range<V>;
 constexpr reverse_iterator<iterator_t<const V>> auto begin() const
 requires common_range<const V>;

 constexpr reverse_iterator<iterator_t<V>> end();
 constexpr reverse_iterator<iterator_t<const V>> auto end() const
 requires common_range<const V>;
 [...]
 };
 [...]
}

[...]

constexpr reverse_iterator<iterator_t<V>> begin() requires common_range<V>;
constexpr reverse_iterator<iterator_t<const V>> auto begin() const
 requires common_range<const V>;

-5- Effects: Equivalent to: return make_reverse_iterator(ranges::end(base_));

constexpr reverse_iterator<iterator_t<V>> end();
constexpr reverse_iterator<iterator_t<const V>> auto end() const
 requires common_range<const V>;

```

### [3388<sup>\(i\)</sup>](#). view iterator types have ill-formed <=> operators

**Section:** 24.6.3.3 [\[range.iota.iterator\]](#), 24.7.5.3 [\[range.transform.iterator\]](#), 24.7.15.3 [\[range.elements.iterator\]](#) **Status:** [Immediate](#) **Submitter:** Jonathan Wakely **Opened:** 2020-02-07 **Last modified:** 2020-02-10

**Priority:** 0

**View other [active issues](#)** in [\[range.iota.iterator\]](#).

**View all other [issues](#)** in [\[range.iota.iterator\]](#).

**View all issues with [Immediate](#) status.**

#### Discussion:

24.6.3.3 [\[range.iota.iterator\]](#) and 24.7.5.3 [\[range.transform.iterator\]](#) and 24.7.15.3 [\[range.elements.iterator\]](#) all declare operator<=> similar to this:

```
friend constexpr compare_three_way_result_t<W> operator<=>(
 const iterator& x, const iterator& y)
 requires totally_ordered<W> && three_way_comparable<W>;
```

Similar to issue [3347](#) and issue [3387](#), this is ill-formed if three\_way\_comparable<W> is not satisfied, because compare\_three\_way\_result\_t<W> is invalid. This declaration is instantiated when the enclosing iterator type is instantiated, making any use of iota\_view<W, B>::iterator ill-formed when three\_way\_comparable<W> is not satisfied.

We can either add an exposition-only *safe-compare-three-way-result-t* alias that denotes void or std::nonesuch for spaceship-less types, so the declaration is valid (and then disabled by the constraints), or simply make them return auto.

[2020-02 Prioritized as IMMEDIATE Monday morning in Prague]

#### Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 24.6.3.3 [\[range.iota.iterator\]](#) as indicated:

```
namespace std::ranges {
 template<class W, class Bound>
 struct iota_view<W, Bound>::iterator {
 [...]
 friend constexpr compare_three_way_result_t<W> auto operator<=>(
 const iterator& x, const iterator& y)
 requires totally_ordered<W> && three_way_comparable<W>;
 [...]
 };
 [...]
}

[...]

friend constexpr compare_three_way_result_t<W> auto
 operator<=>(const iterator& x, const iterator& y)
 requires totally_ordered<W> && three_way_comparable<W>;

-19- Effects: Equivalent to: return x.value_ <=> y.value_;
```

2. Modify 24.7.5.3 [\[range.transform.iterator\]](#) as indicated:

```
namespace std::ranges {
 template<class V, class F>
 template<bool Const>
 class transform_view<V, F>::iterator {
 [...]
 friend constexpr compare_three_way_result_t<iterator_t<Base>> auto
 operator<=>(const iterator& x, const iterator& y)
 requires random_access_range<Base> && three_way_comparable<iterator_t<Base>>;
 [...]
 };
 [...]
}

[...]

friend constexpr compare_three_way_result_t<iterator_t<Base>> auto
 operator<=>(const iterator& x, const iterator& y)
 requires random_access_range<Base> && three_way_comparable<iterator_t<Base>>;

-19- Effects: Equivalent to: return x.current_ <=> y.current_;
```

3. Modify 24.7.15.3 [\[range.elements.iterator\]](#) as indicated:

```
namespace std::ranges {
 template<class V, size_t N>
 template<bool Const>
 class elements_view<V, N>::iterator {
 [...]
 friend constexpr compare_three_way_result_t<iterator_t<base_t>> auto
 operator<=>(const iterator& x, const iterator& y)
 requires random_access_range<base_t> && three_way_comparable<iterator_t<base_t>>;
 [...]
 };
 [...]
}

[...]

friend constexpr compare_three_way_result_t<iterator_t<base_t>> auto
 operator<=>(const iterator& x, const iterator& y)
 requires random_access_range<base_t> && three_way_comparable<iterator_t<base_t>>;

-18- Effects: Equivalent to: return x.current_ <=> y.current_;
```

---

### [3389\(i\)](#). A move-only iterator still does not have a counted\_iterator

**Section:** 23.5.6.2 [\[counted.iter.const\]](#) **Status:** [Immediate](#) **Submitter:** Patrick Palka **Opened:** 2020-02-07 **Last modified:** 2020-02-10

**Priority:** 0

**View all issues with** [Immediate](#) status.

#### **Discussion:**

[P1207R4](#) ("Movability of single-pass iterators") introduces the notion of a move-only non-forward iterator and makes some changes to the iterator adaptor `counted_iterator` in order to support move-only iterators.

The problem is that the constructor of `counted_iterator` (23.5.6.2 [\[counted.iter.const\]](#) p2) accepting such an iterator is specified as "Initializes current with i" which would attempt copy-constructing current from i instead of move-constructing it.

*[2020-02 Prioritized as IMMEDIATE Monday morning in Prague]*

#### **Proposed resolution:**

This wording is relative to [N4849](#).

1. Modify 23.5.6.2 [\[counted.iter.const\]](#) as indicated:

```
constexpr counted_iterator(I i, iter_difference_t<I> n);

-1- Preconditions: n >= 0.

-2- Effects: Initializes current with std::move(i) and length with n.
```

---

### [3390\(i\)](#). `make_move_iterator()` cannot be used to construct a `move_iterator` for a move-only iterator

**Section:** 23.5.3.8 [\[move.iter.nonmember\]](#) **Status:** [Immediate](#) **Submitter:** Patrick Palka **Opened:** 2020-02-07 **Last modified:** 2020-02-10

**Priority:** 0

**View other** [active issues](#) in `[move.iter.nonmember]`.

**View all other** [issues](#) in `[move.iter.nonmember]`.

**View all issues with** [Immediate](#) status.

#### **Discussion:**

[P1207R4](#) ("Movability of single-pass iterators") introduces the notion of a move-only non-forward iterator and makes some changes to the existing specification to realize that support.

The problem is that the specification of `make_move_iterator()` provided in 23.5.3.8 [\[move.iter.nonmember\]](#) p6 does attempt to construct a `move_iterator<Iterator>` with an lvalue of i instead of an rvalue, having the effect of copying it instead of moving it, thus preventing to accept move-only iterators.

*[2020-02 Prioritized as IMMEDIATE Monday morning in Prague]*

**Proposed resolution:**

This wording is relative to [N4849](#).

- 1. Modify 23.5.3.8 [\[move.iter.nonmember\]](#) as indicated:

```
template<class Iterator>
constexpr move_iterator<Iterator> make_move_iterator(Iterator i);

-6- Returns: move_iterator<Iterator>(std::move\(i\)).
```

**[3393\(i\)](#). Missing/incorrect feature test macro for coroutines**

**Section:** 17.3.2 [\[version.syn\]](#) **Status:** [Immediate](#) **Submitter:** Barry Revzin **Opened:** 2020-01-25 **Last modified:** 2020-02-12

**Priority:** 0

**View other** [active issues](#) in [\[version.syn\]](#).

**View all other** [issues](#) in [\[version.syn\]](#).

**View all issues with** [Immediate](#) status.

**Discussion:**

We have a policy, established in [P1353](#) (and needing to be added to SD-6):

In some cases a feature requires two macros, one for the language and one for the library. For example, the library does not want to define its three-way comparison operations unless the compiler supports the feature.

For end-users, it is suggested that they test only the library macro, as that will only be true if the language macro is also true. As a result, the language macros contain "impl" to distinguish them from the more general version that is expected to be set by the library. That paper added two papers of macros: one for <=> and one for destroying delete. We have a third such example in coroutines: there is library machinery that needs to be provided only when the compiler has language support for it, and the end user should just check the library macro.

**Previous resolution [SUPERSEDED]:**

This wording is relative to [N4849](#).

- 1. Modify 15.11 [\[cpp.predefined\]](#), Table [\[tab:cpp.predefined.ft\]](#), as indicated:

Table 18: Feature-test macros  
[\[tab:cpp.predefined.ft\]](#)

| Macro name                         | Value   |
|------------------------------------|---------|
| <a href="#">[...]</a>              |         |
| <code>__cpp_impl_coroutines</code> | 201902L |
| <a href="#">[...]</a>              |         |

- 2. Modify 17.3.2 [\[version.syn\]](#), header `<version>` synopsis, as indicated:

```
\[...\]
#define __cpp_lib_constexpr_vector 201907L // also in <vector>
#define __cpp_lib_coroutines 201902L // also in <coroutine>
#define __cpp_lib_destroying_delete 201806L // also in <new>
\[...\]
```

*[2020-02-11, Prague]*

LWG observed that the naming suggestion didn't follow naming conventions of SG-10 because of the plural form coroutines. The submitter agreed with that complaint, so the revised wording uses now the singular form.

*[2020-02 Moved to Immediate on Tuesday in Prague.]*

**Proposed resolution:**

This wording is relative to [N4849](#).

- 1. Modify 15.11 [\[cpp.predefined\]](#), Table [\[tab:cpp.predefined.ft\]](#), as indicated:

Table 18: Feature-test macros  
[\[tab:cpp.predefined.ft\]](#)

| Macro name            | Value |
|-----------------------|-------|
| <a href="#">[...]</a> |       |

|                       |         |
|-----------------------|---------|
| __cpp_impl_coroutines | 201902L |
| [...]                 |         |

2. Modify 17.3.2 [\[version.syn\]](#), header <version> synopsis, as indicated:

```
[...]
#define __cpp_lib_constexpr_vector 201907L // also in <vector>
#define __cpp_lib_coroutine 201902L // also in <coroutine>
#define __cpp_lib_destroying_delete 201806L // also in <new>
[...]
```

### 3395<sup>(i)</sup>. Definition for three-way comparison needs to be updated (US 152)

**Section:** 16.3.4 [\[defns.comparison\]](#) **Status:** [Immediate](#) **Submitter:** Jeff Garland **Opened:** 2020-02-10 **Last modified:** 2020-02-12

**Priority:** 1

**View all issues with** [Immediate](#) status.

**Discussion:**

**Addresses** [US 152](#)

This definition in 16.3.4 [\[defns.comparison\]](#) should be updated to accommodate the new 3-way comparison operator (7.6.8 [\[expr.spaceship\]](#)) as well.

*[2020-02 Moved to Immediate on Tuesday in Prague.]*

**Proposed resolution:**

This wording is relative to [N4849](#).

1. Modify 16.3.4 [\[defns.comparison\]](#) as indicated:

**comparison function**

operator function (12.6 [\[over.oper\]](#)) for any of the equality (7.6.10 [\[expr.eq\]](#)), ~~or~~ relational (7.6.9 [\[expr.rel\]](#)), or three-way comparison (7.6.8 [\[expr.spaceship\]](#)) operators

### 3396<sup>(i)</sup>. Clarify point of reference for `source_location::current()` (DE 169)

**Section:** 17.8.2.1 [\[support.srcloc.cons\]](#) **Status:** [Immediate](#) **Submitter:** Jens Maurer **Opened:** 2020-02-13 **Last modified:** 2020-02-14

**Priority:** 2

**View all issues with** [Immediate](#) status.

**Discussion:**

**Addresses** [DE 169](#)

The expectation of the note that a default argument expression involving `current()` causes a `source_location` to be constructed that refers to the site of a function call where that default argument is needed has no basis in normative text. In particular, 9.2.3.6 paragraph 5 seems to imply that the name "current" and its semantics are bound where it appears lexically in the function declaration.

Proposed change:

Add normative text to express the desired semantics.

*[2020-02 Moved to Immediate on Thursday afternoon in Prague.]*

**Proposed resolution:**

This wording is relative to [N4849](#).

1. Modify 17.8.2.1 [\[support.srcloc.cons\]](#) as indicated:

```
static constexpr source_location current() noexcept;
```

-1- [...]

-2- Remarks: ~~When a default member initializer is used to initialize a non-static data member, any calls to `current` to `current` that appears as a default member initializer (11.4 [\[class.mem\]](#)), or as a subexpression thereof, should correspond to the location of the constructor [definition](#) or aggregate initialization that ~~initializes the member~~ uses the default member~~

initializer. Any call to `current` that appears as a default argument (9.3.3.6 [dcl.fct.default]), or as a subexpression thereof, should correspond to the location of the invocation of the function that uses the default argument (7.6.1.2 [expr.call]).

~~3- [Note: When used as a default argument (9.3.3.6 [dcl.fct.default]), the value of the `source_location` will be the location of the call to `current` at the call site. — end note]~~

---

### [3397](#)<sup>(i)</sup>. `ranges::basic_istream_view::iterator` should not provide `iterator_category`

**Section:** 24.6.4.3 [range.istream.iterator] **Status:** [Immediate](#) **Submitter:** Tomasz Kamiński **Opened:** 2020-02-13 **Last modified:** 2020-02-14

**Priority:** 1

**View other [active issues](#)** in [range.istream.iterator].

**View all other [issues](#)** in [range.istream.iterator].

**View all issues with [Immediate](#) status.**

#### **Discussion:**

The `ranges::basic_istream_view::iterator` is a move-only type, and as such it does not meet the *Cpp17* iterator requirements, yet it does provide `iterator_category` (intended to be used for *Cpp17* iterators). We should provide `iterator_concept` instead.

[2020-02 Status to Immediate on Thursday night in Prague.]

#### **Proposed resolution:**

This wording is relative to [N4849](#).

1. Modify 24.6.4.3 [range.istream.iterator] as indicated:

```
namespace std::ranges {
 template<class Val, class CharT, class Traits>
 class basic_istream_view<Val, CharT, Traits>::iterator { // exposition only
 public:
 using iterator_categoryiterator_concept = input_iterator_tag;
 using difference_type = ptrdiff_t;
 using value_type = Val;

 iterator() = default;
 [...]
 };
}
```

---

### [3398](#)<sup>(i)</sup>. `tuple_element_t` is also wrong for `const subrange`

**Section:** 24.2 [ranges.syn] **Status:** [Immediate](#) **Submitter:** Casey Carter **Opened:** 2019-02-13 **Last modified:** 2020-02-14

**Priority:** 0

**View other [active issues](#)** in [ranges.syn].

**View all other [issues](#)** in [ranges.syn].

**View all issues with [Immediate](#) status.**

#### **Discussion:**

As currently specified, it uses the cv-qualified partial specialization, which incorrectly adds cv-qualification to the element type.

#### **Previous resolution [SUPERSEDED]:**

This wording is relative to [N4849](#).

1. Modify 24.2 [ranges.syn], header `<ranges>` synopsis, as indicated:

```
[...]
namespace std {
 namespace views = ranges::views;

 template<class I, class S, ranges::subrange_kind K>
 struct tuple_size<ranges::subrange<I, S, K>>
 : integral_constant<size_t, 2> {};
 template<class I, class S, ranges::subrange_kind K>
 struct tuple_element<0, ranges::subrange<I, S, K>> {
 using type = I;
 };
}
```



```

};
template<class I, class S, ranges::subrange_kind K>
struct tuple_element<1, ranges::subrange<I, S, K>> {
 using type = S;
};
template<class I, class S, ranges::subrange_kind K>
struct tuple_element<0, const ranges::subrange<I, S, K>> {
 using type = I;
};
template<class I, class S, ranges::subrange_kind K>
struct tuple_element<1, const ranges::subrange<I, S, K>> {
 using type = S;
};
}

```

2. Add the following wording to Annex D:

#### **D.? Deprecated subrange tuple interface [depr.ranges.syn]**

1 The header `<ranges>` (24.2 [ranges.syn]) has the following additions:

```

namespace std {
 template<size_t X, class I, class S, ranges::subrange_kind K>
 struct tuple_element<X, volatile ranges::subrange<I, S, K>> {};
 template<size_t X, class I, class S, ranges::subrange_kind K>
 struct tuple_element<X, const volatile ranges::subrange<I, S, K>> {};
}

```

[2020-02-14, Prague]

LWG decided to remove the volatile support, we shouldn't give the impression to support an idiom that wouldn't work.

#### **Proposed resolution:**

This wording is relative to [N4849](#).

1. Modify 24.2 [ranges.syn], header `<ranges>` synopsis, as indicated:

```

[...
namespace std {
 namespace views = ranges::views;

 template<class I, class S, ranges::subrange_kind K>
 struct tuple_size<ranges::subrange<I, S, K>>
 : integral_constant<size_t, 2> {};
 template<class I, class S, ranges::subrange_kind K>
 struct tuple_element<0, ranges::subrange<I, S, K>> {
 using type = I;
 };
 template<class I, class S, ranges::subrange_kind K>
 struct tuple_element<1, ranges::subrange<I, S, K>> {
 using type = S;
 };
 template<class I, class S, ranges::subrange_kind K>
 struct tuple_element<0, const ranges::subrange<I, S, K>> {
 using type = I;
 };
 template<class I, class S, ranges::subrange_kind K>
 struct tuple_element<1, const ranges::subrange<I, S, K>> {
 using type = S;
 };
}

```